CSE471: System Analysis and Design Project Report
Project Title: Workforce Enroll

Table of Contents

Introduction

Workforce Enroll is an innovative and user-friendly job portal designed to simplify the job search process for jobseekers and streamline candidate recruitment for companies. The platform offers a range of features aimed at enhancing the overall experience for both job seekers and employers, making it easy for them to connect and manage job applications effectively.

For **jobseekers**, Workforce Enroll provides a seamless experience, allowing them to easily search and filter jobs, apply with confidence, and track their application status. The application status is dynamically updated, with color-coded changes when the status is updated by employers, giving jobseekers instant feedback. Additionally, users can manage their profiles, create resumes with the built-in resume builder, and bookmark jobs of interest for future reference.

On the **company** side, Workforce Enroll stands out for its flexibility and efficiency. Employers can post jobs effortlessly through the intuitive job posting interface, manage applications, and make quick decisions on candidates. The job posting and management system includes unique features such as the ability to delete a job and restore it with all the associated applications or repost the job as new, effectively wiping out previous applications. Employers can also edit, update, or delete job posts permanently, giving them complete control over their recruitment process.

The **dashboard functionalities** are tailored for both **admins** and **companies**, allowing them to oversee job postings, applications, and candidate profiles .Admin can add more admins from Dashboard. For candidates, the **Admin Dashboard** provides tools to manage profiles and monitor job applications, while the **Company Dashboard** gives companies an overview of posted jobs, pending, accepted, and rejected applications, as well as the ability to track the number of job vacancies.

Application management is made easy, with companies able to accept, reject, and even reconsider previously rejected applications. The countdown feature for job deadlines ensures that users are alerted when the last day to apply arrives, while the detailed job information on pending, accepted, and rejected applications helps job seekers make informed decisions.

Overall, **Workforce Enroll** is a comprehensive and well-rounded platform that simplifies the job search and recruitment process, making it easier for jobseekers to find the right opportunities and for companies to identify top candidates, all within an intuitive and user-friendly interface.

Functional Requirements

Module 1 : USER AUTHENTICATION AND AUTHORIZATION

1. Users and Admins Registration
2. Login
3. Password Management
4.Admin Dashboard (Candidates)
5.Admin Dashboard (company)

Module 2 : Job Posting and Management

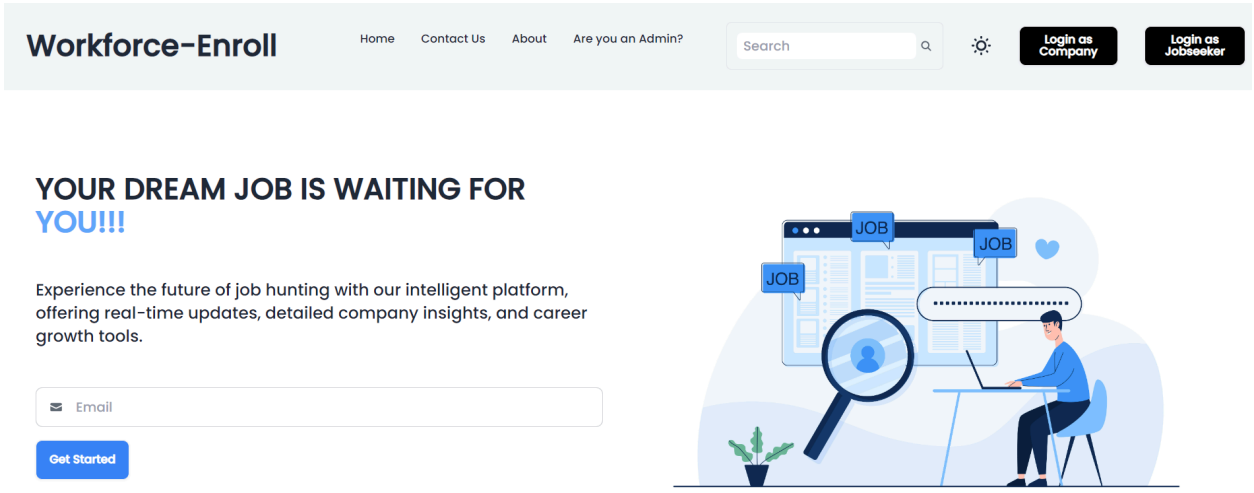
1. User Profile Handling
2. Resume builder
3. Job Posting Interface
4.Company Dashboard
5.Job Search and Filters

Extras:

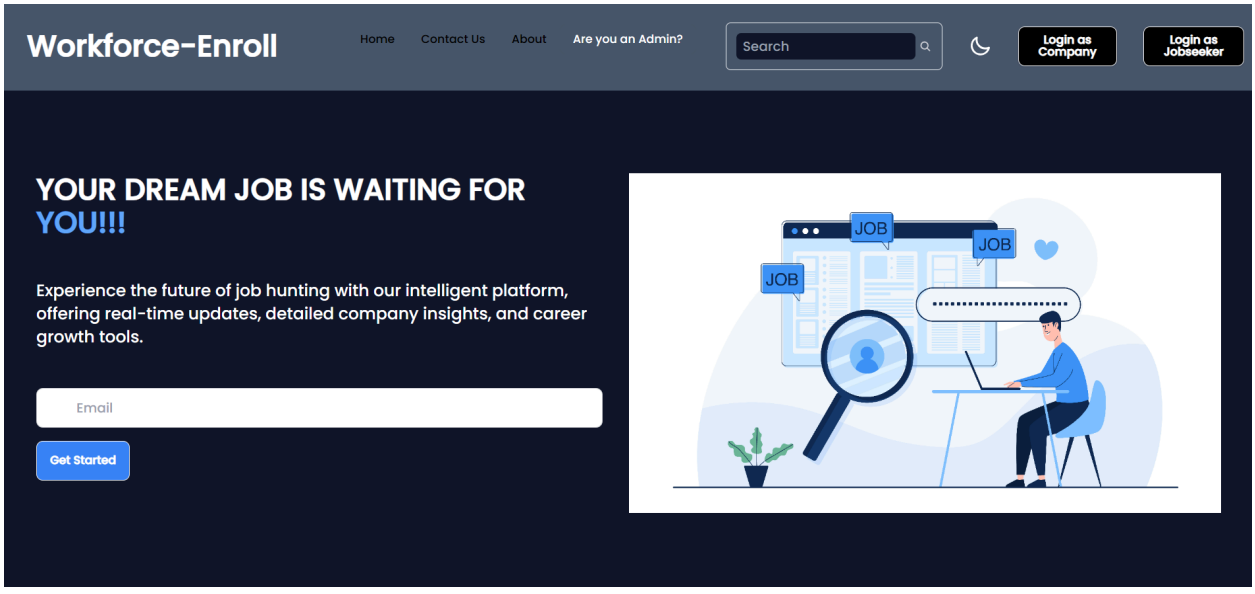
1.Deleting Jobs(Restore the existing job,Post the deleted job as new,edit,delete jobs permanently)

User Manual

Home Page: This is the home page view

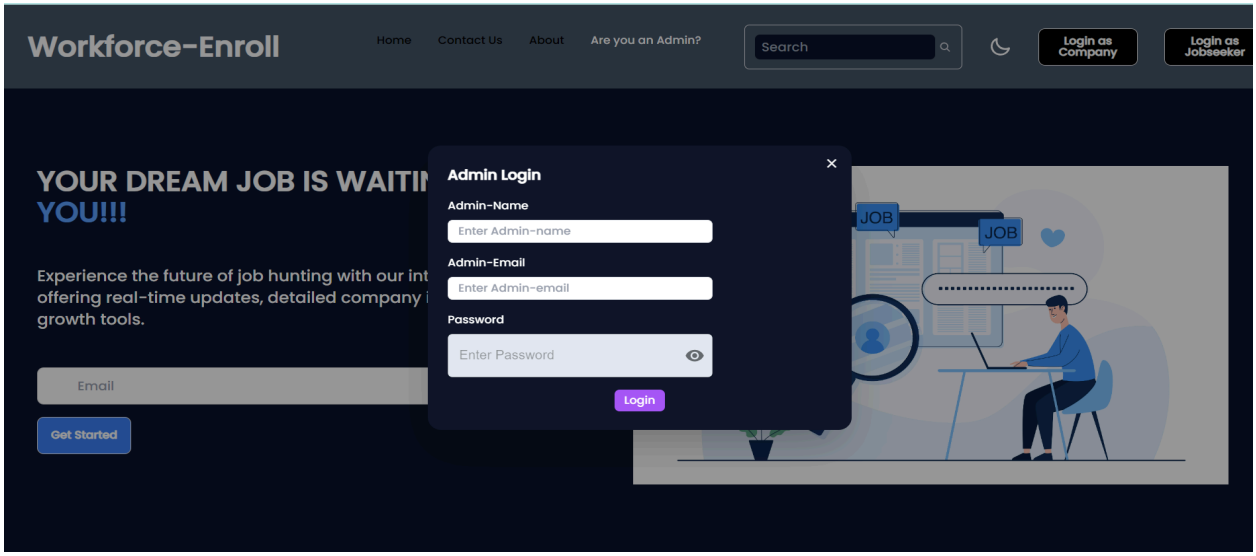


Every page has its dark mode.

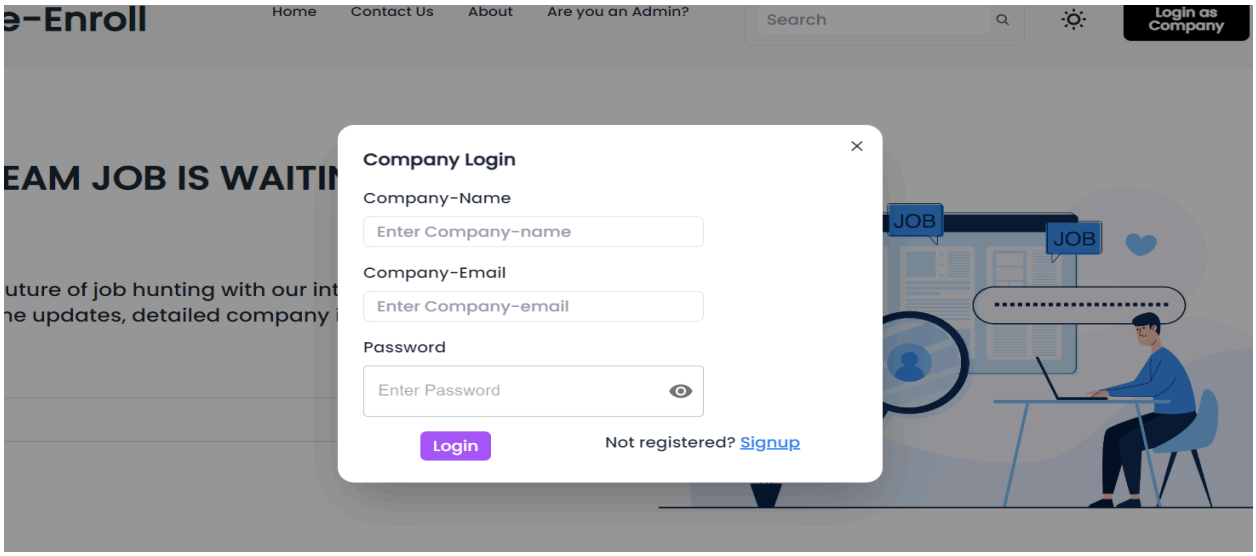


Login: All three entities(Admin,Company,Jobseeker) can log in. To show the color variation, some of the pages are in dark and some of them are in light mode.

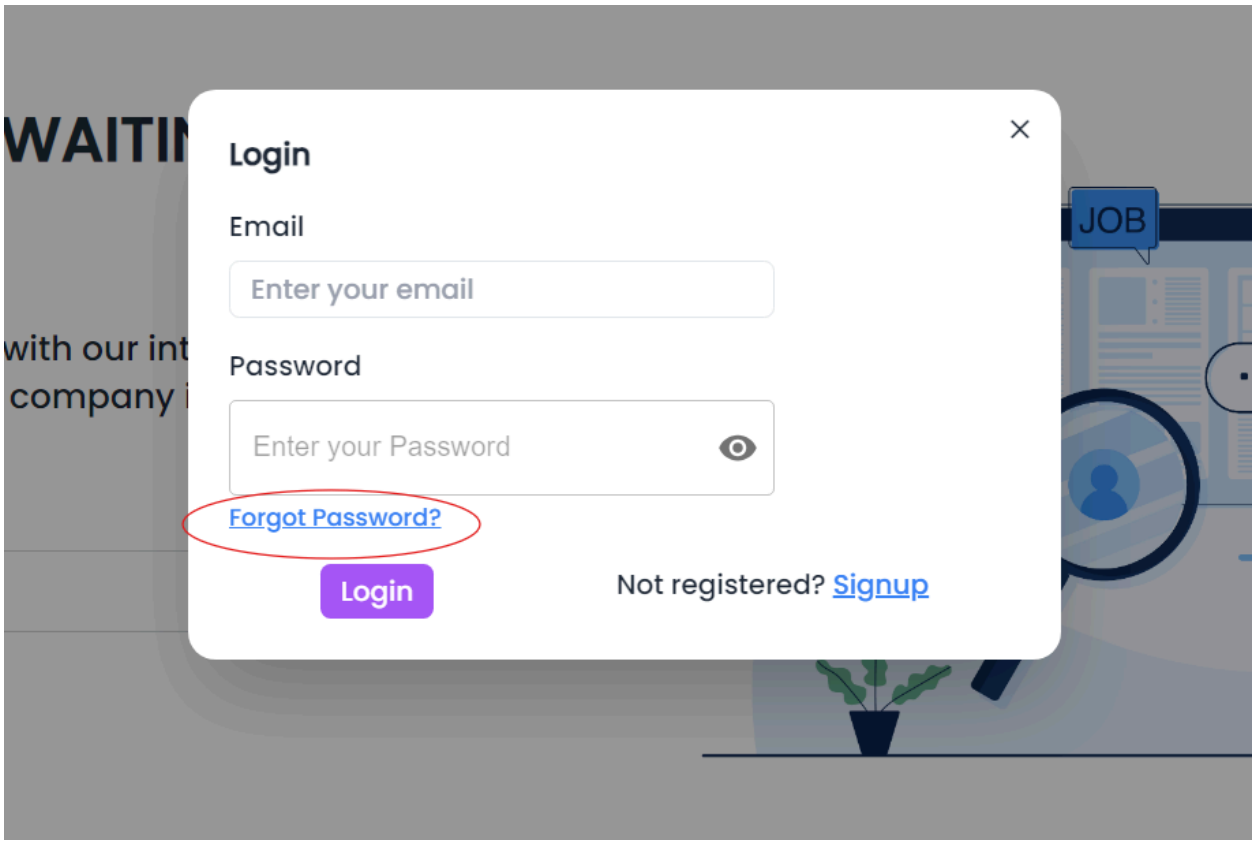
Admin Login:



Company Login:

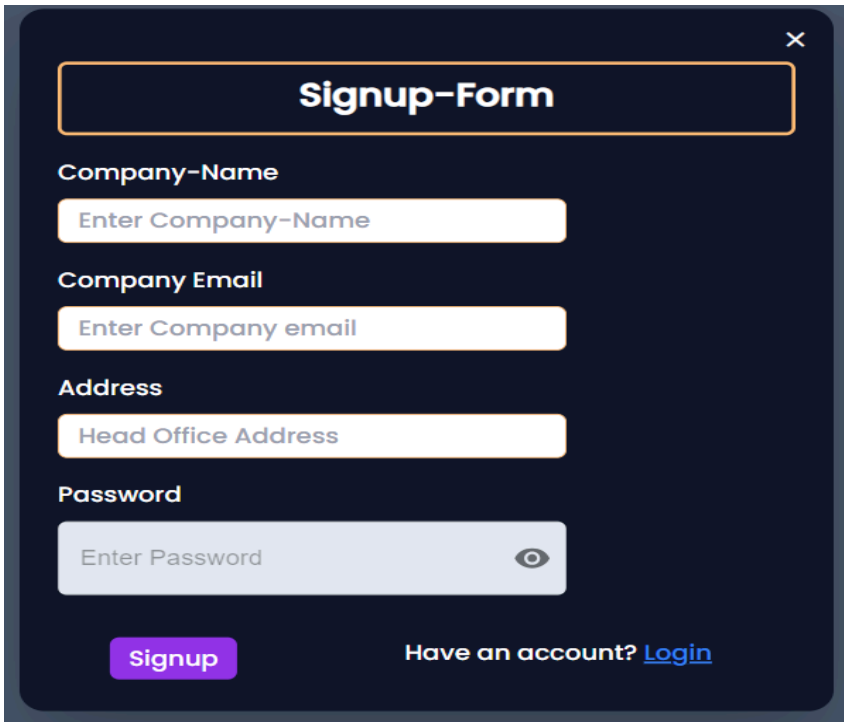


Jobseeker Login:

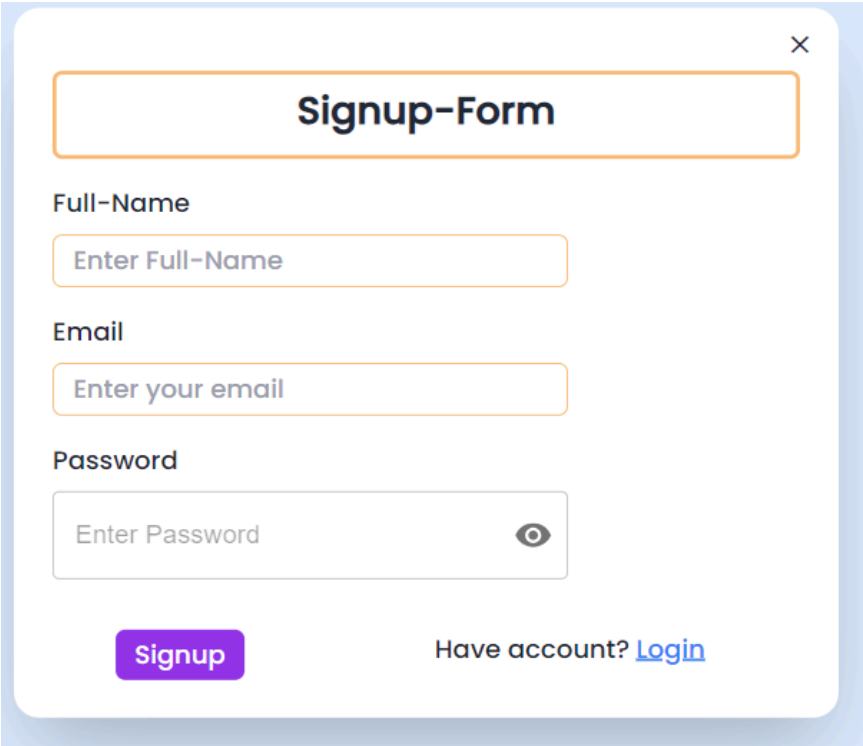


Signup: Company and Jobseeker can sign up or create a new account from the login modal.

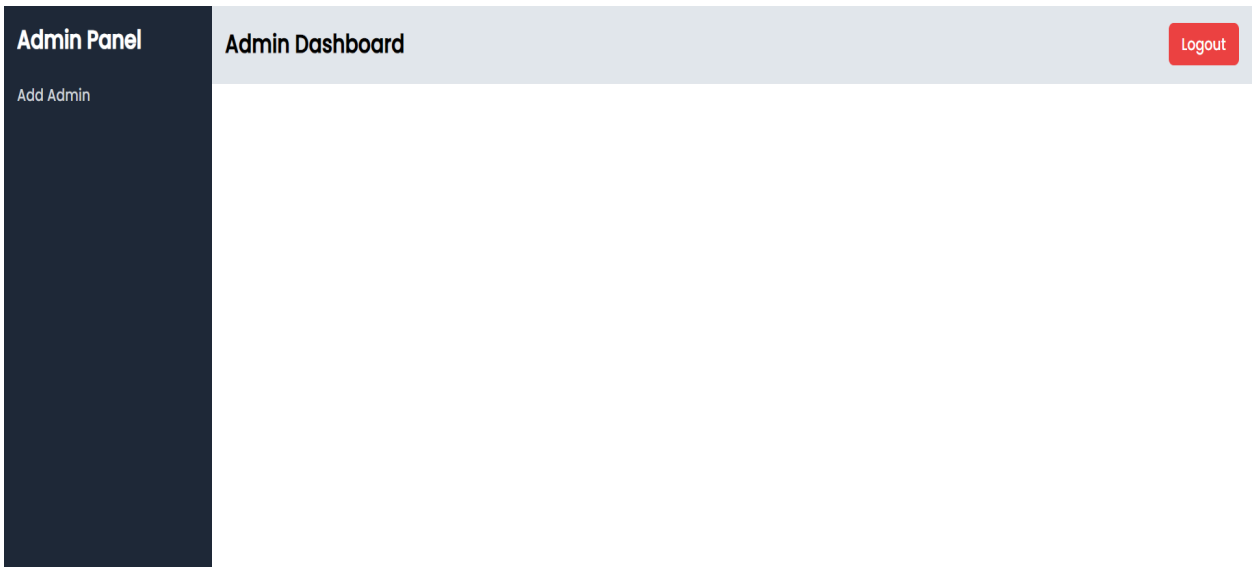
Company signup:

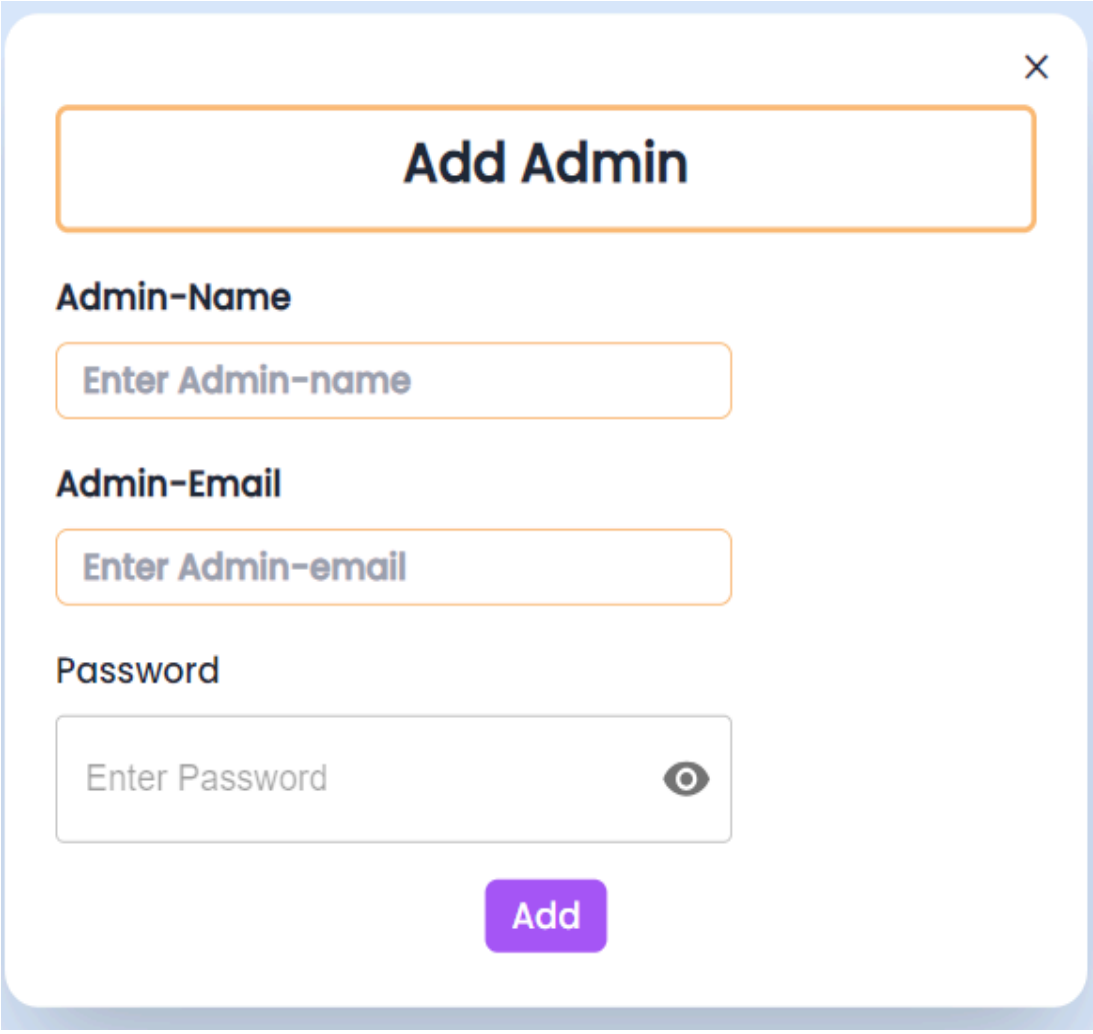


Jobseeker signup:

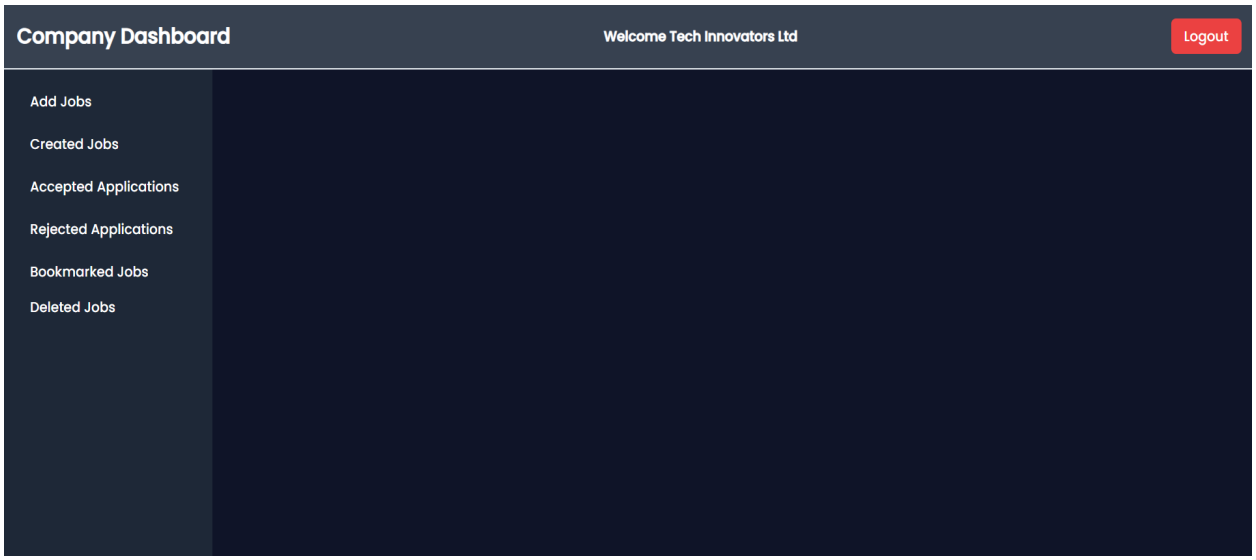


Add Admins: Admins can create an account Directly other admins can add admins from the admin dashboard. Here on the top right corner logout button.



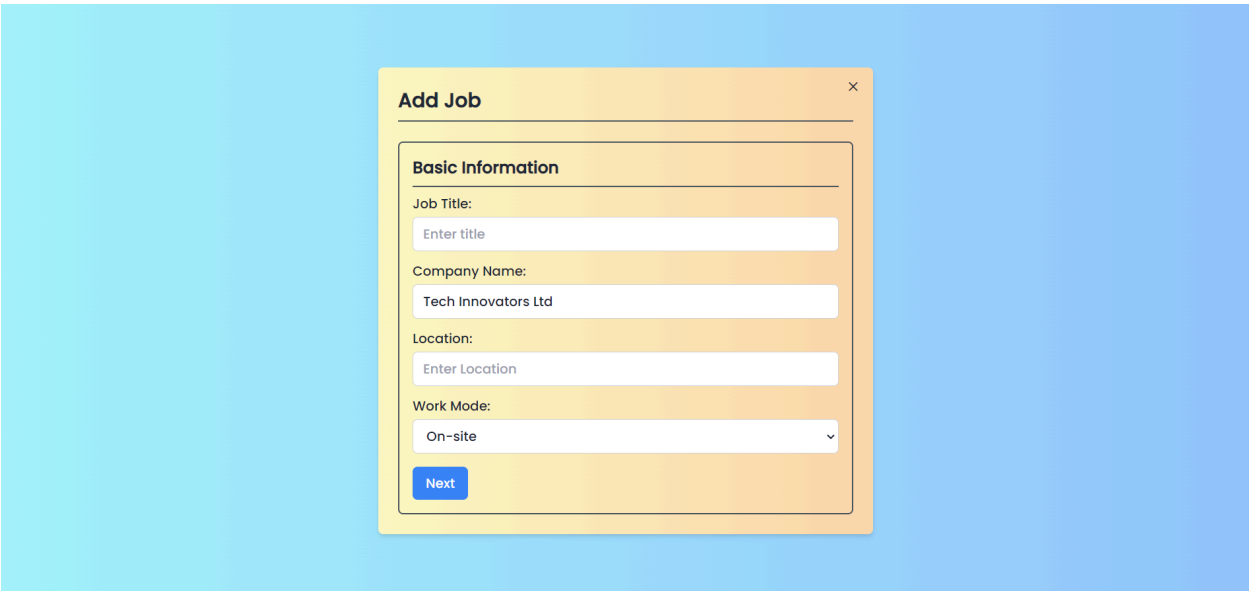


Company Dashboard: After Login as a company this dashboard will be shown.

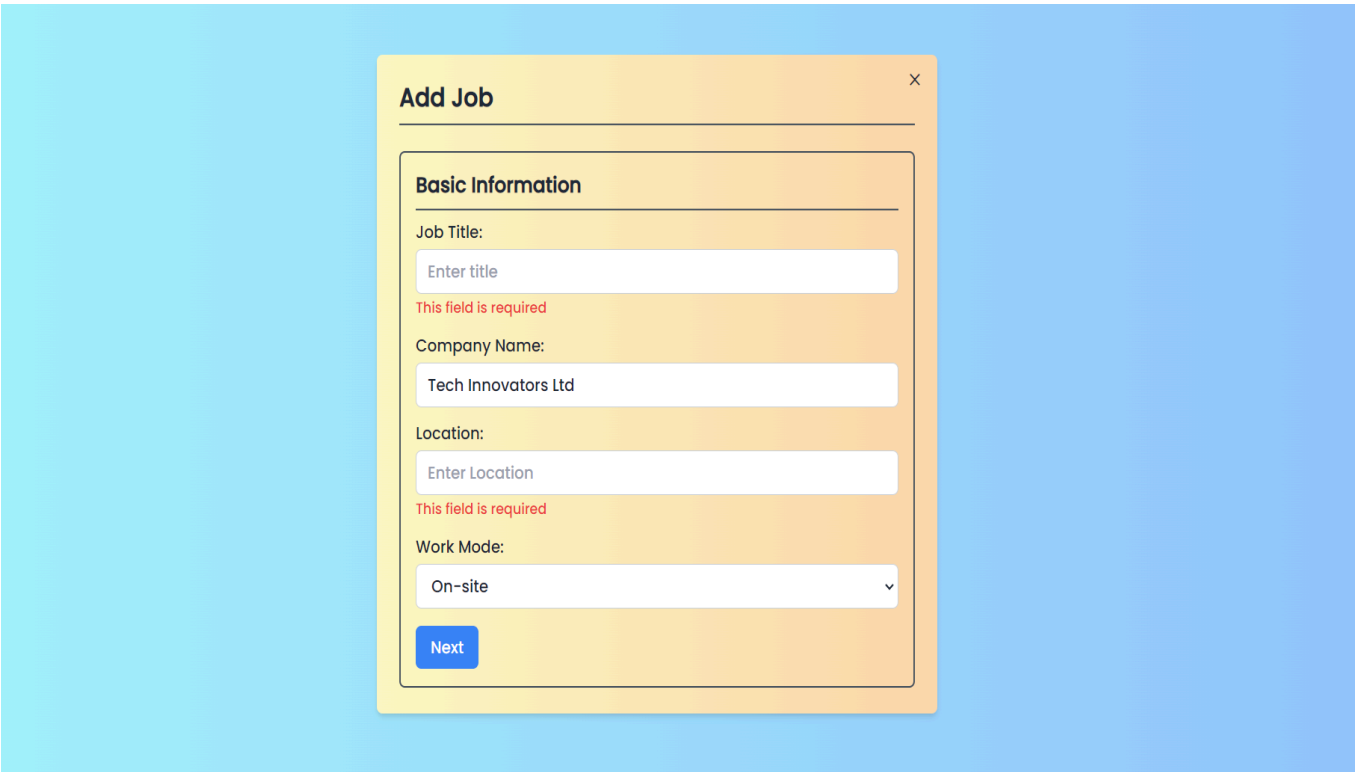


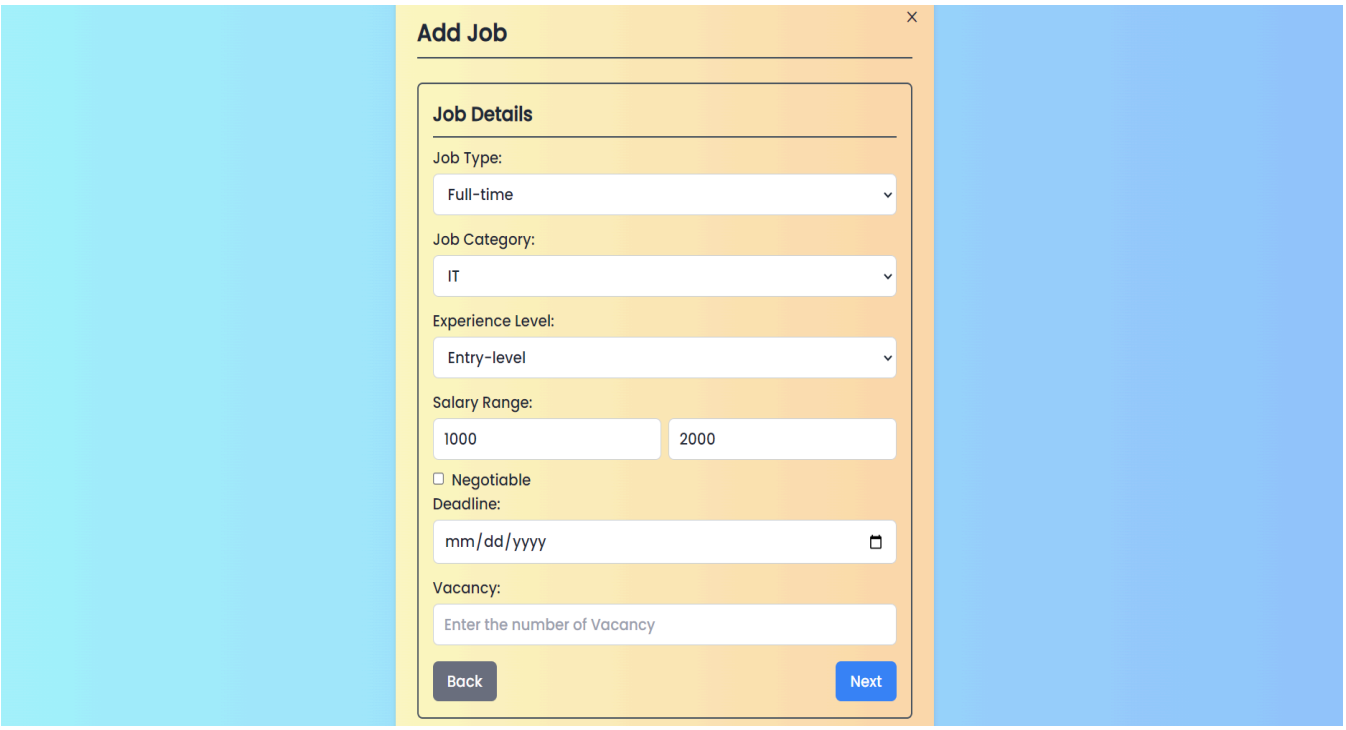
Each of the buttons are described below:

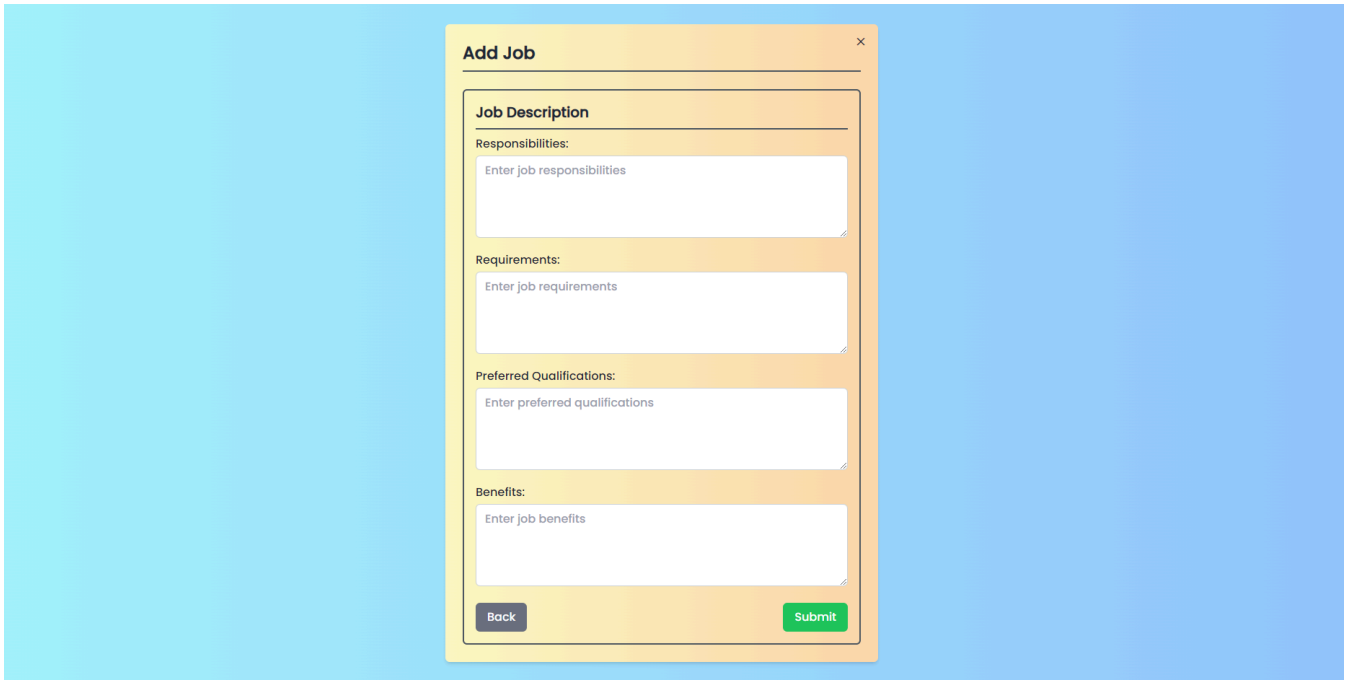
Add Jobs: to add a new job the button is shown at the top of the sidebar.(Light mode)



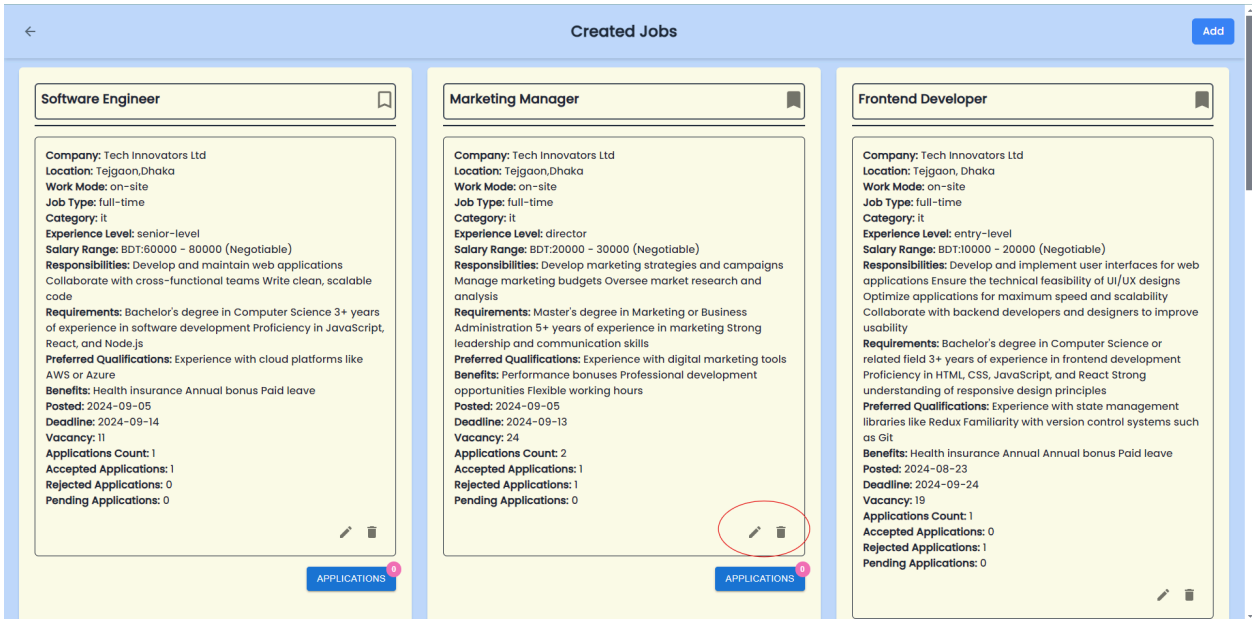
All the fields are required if any of them is not given and the next button is pressed it will show this field is required.


Add Job
Basic Information
Job Title:
Enter title
This field is required
Company Name:
Tech Innovators Ltd
Location:
Enter Location
This field is required
Work Mode:
On-site
Next

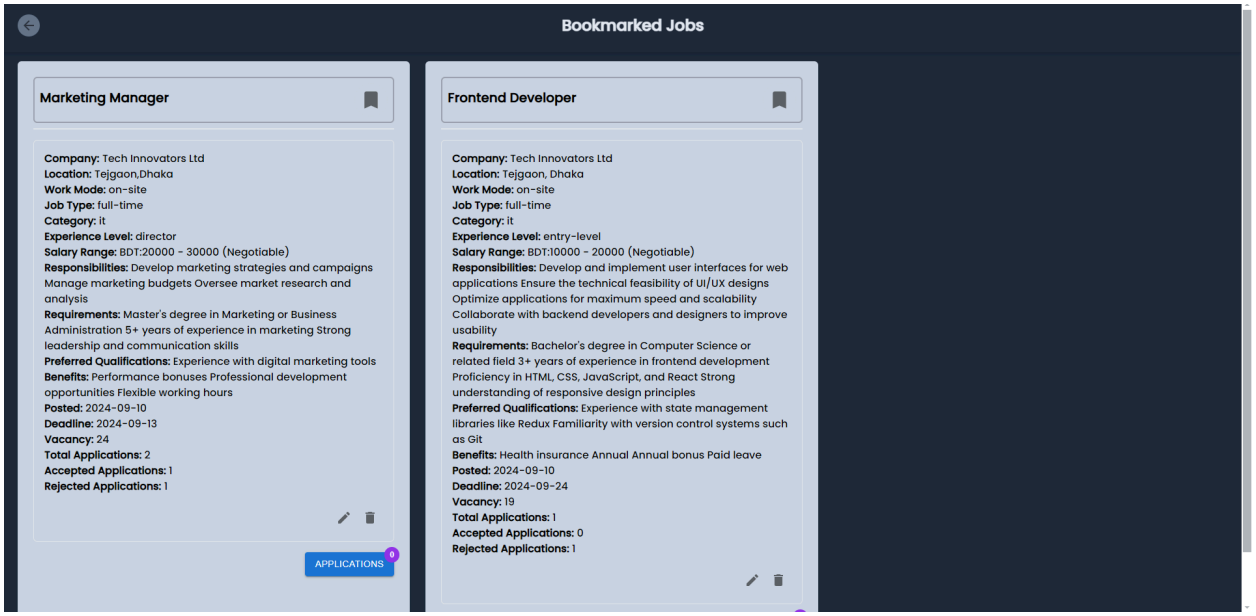

Add Job
Job Details
Job Type:
Full-time
Job Category:
IT
Experience Level:
Entry-level
Salary Range:
1000
2000
Negotiable
Deadline:
mm/dd/yyyy
Vacancy:
Enter the number of Vacancy
Back
Next



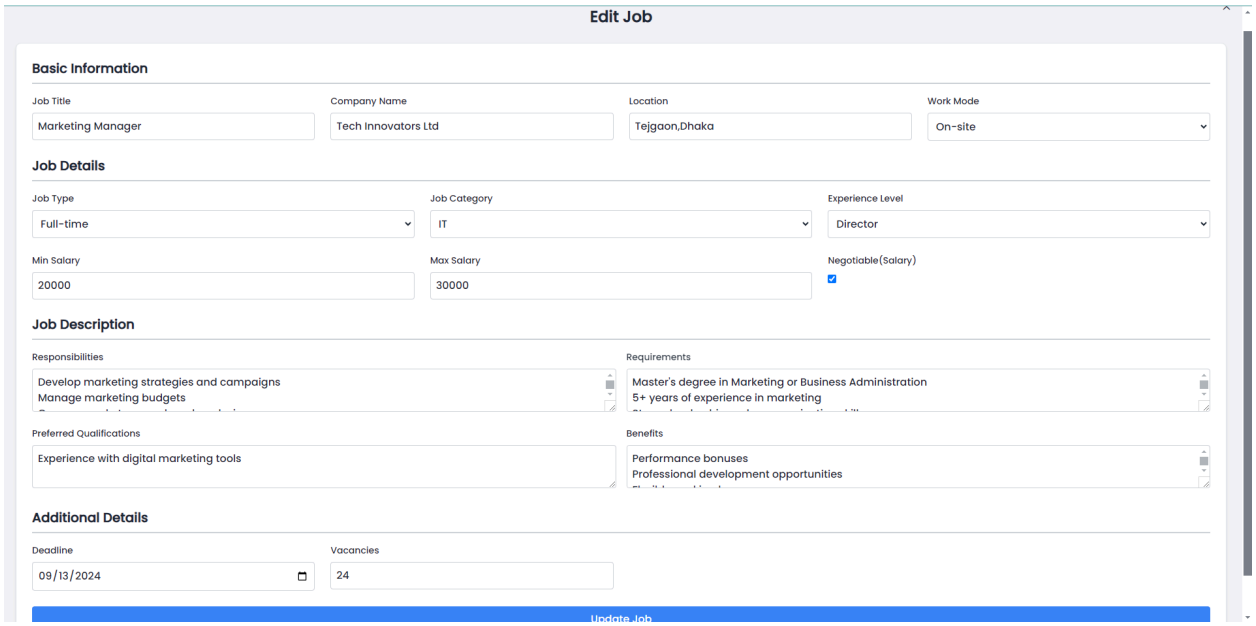
Created Jobs: here all the created jobs will be shown there are edit and delete buttons also given.



Bookmarked jobs: All the bookmarked jobs will be shown here.(Dark mode)

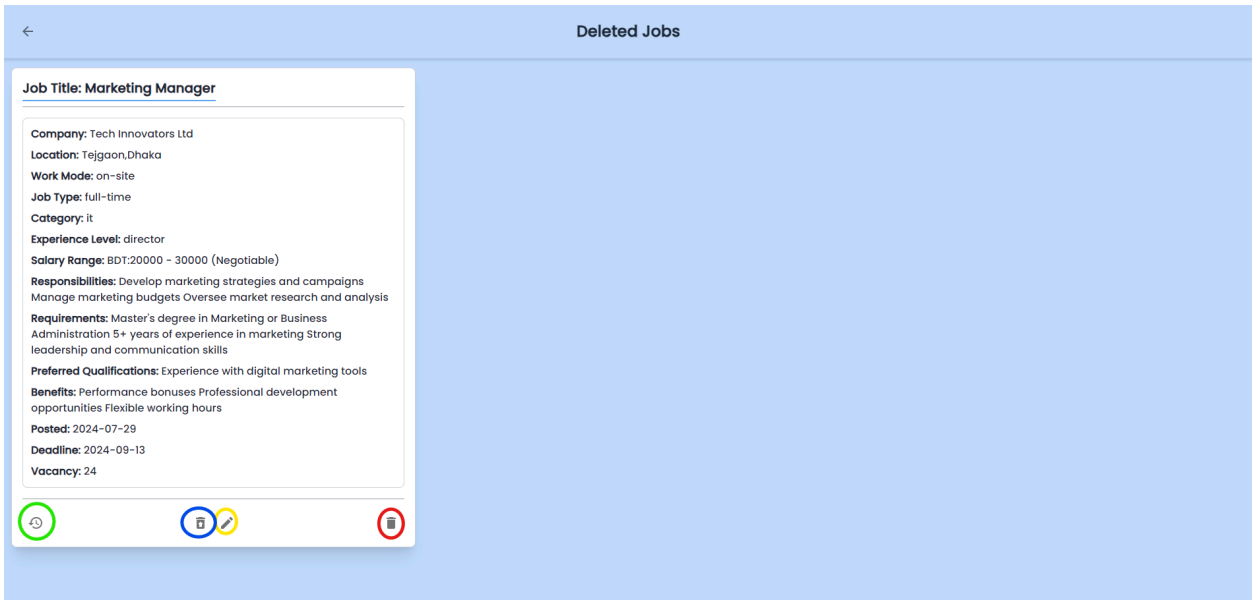


Edit Jobs:

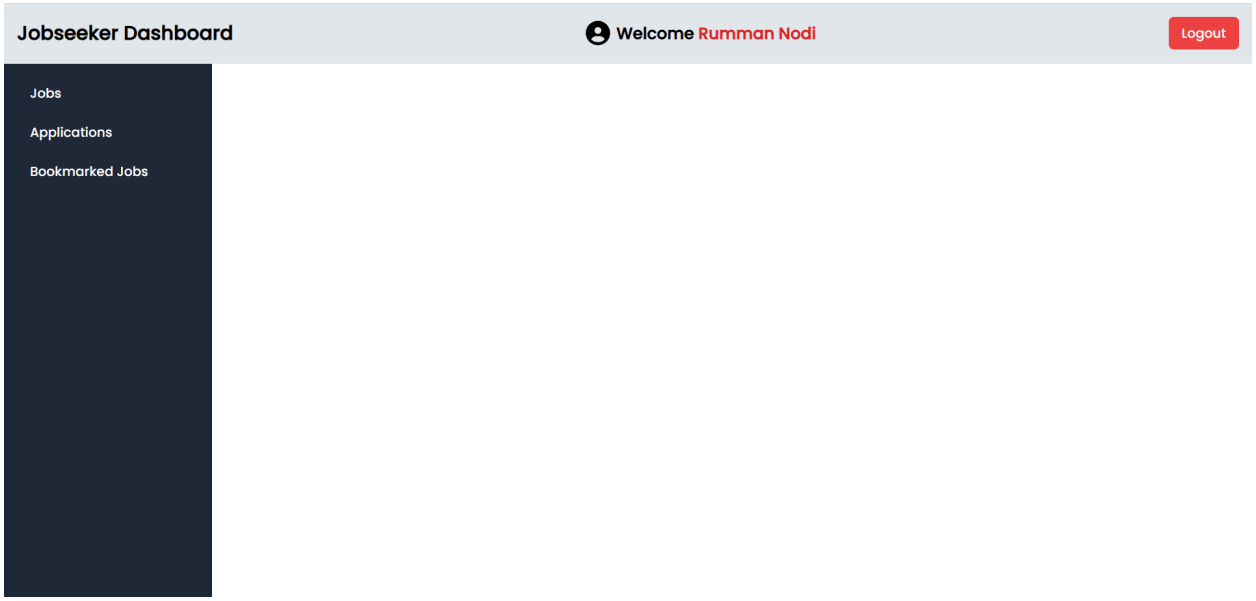


Deleted Jobs: By clicking on the delete button the job will store in the deleted jobs page.Here the green circled icon is for restoring the job which means by clicking on this icon the job will be posting as it is will all its applications that are presented.The blue circled icon is for repost that job as a new job here all the applications will be removed and jobseekers will be able to apply for this job again.the red circled icon for deleting the job permanently and also edit button also

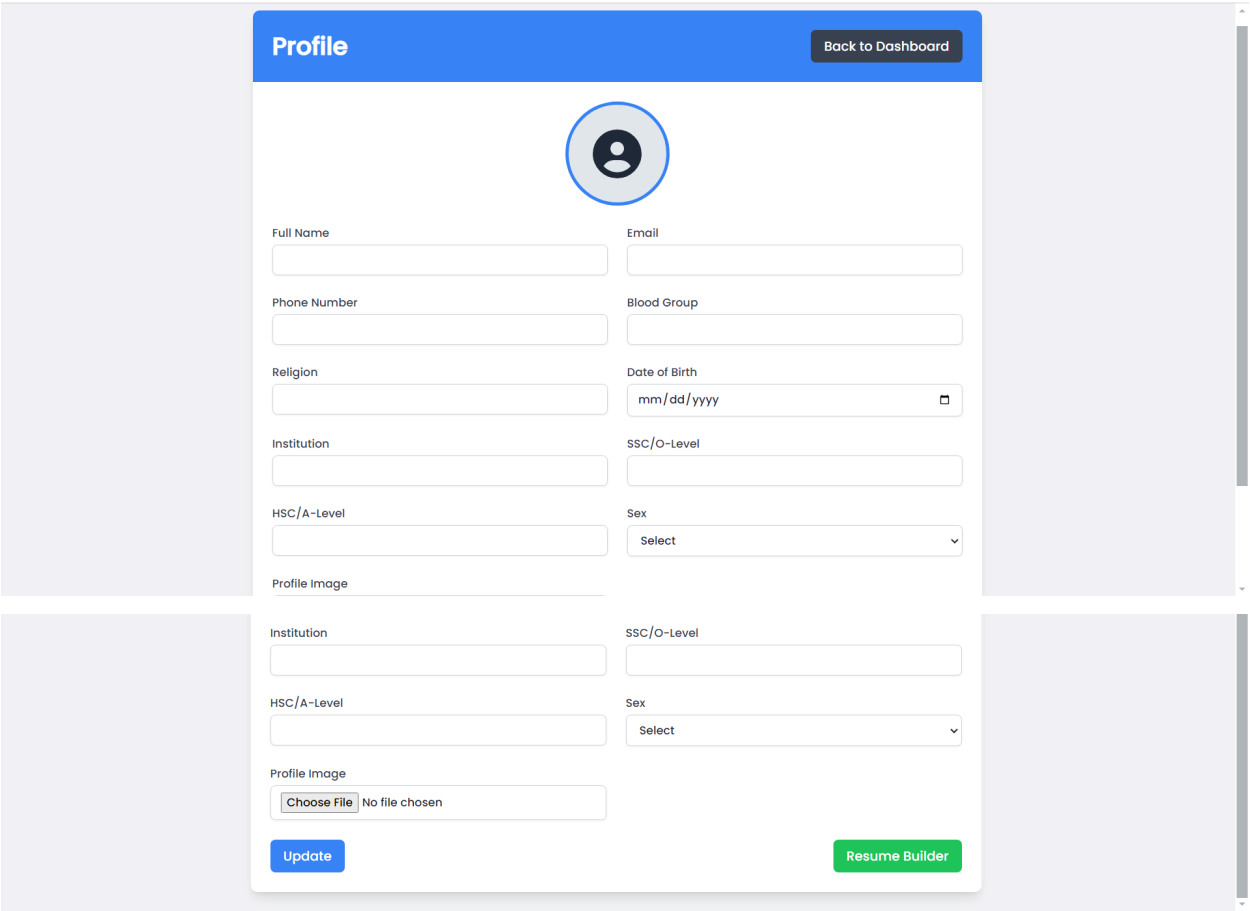
present.



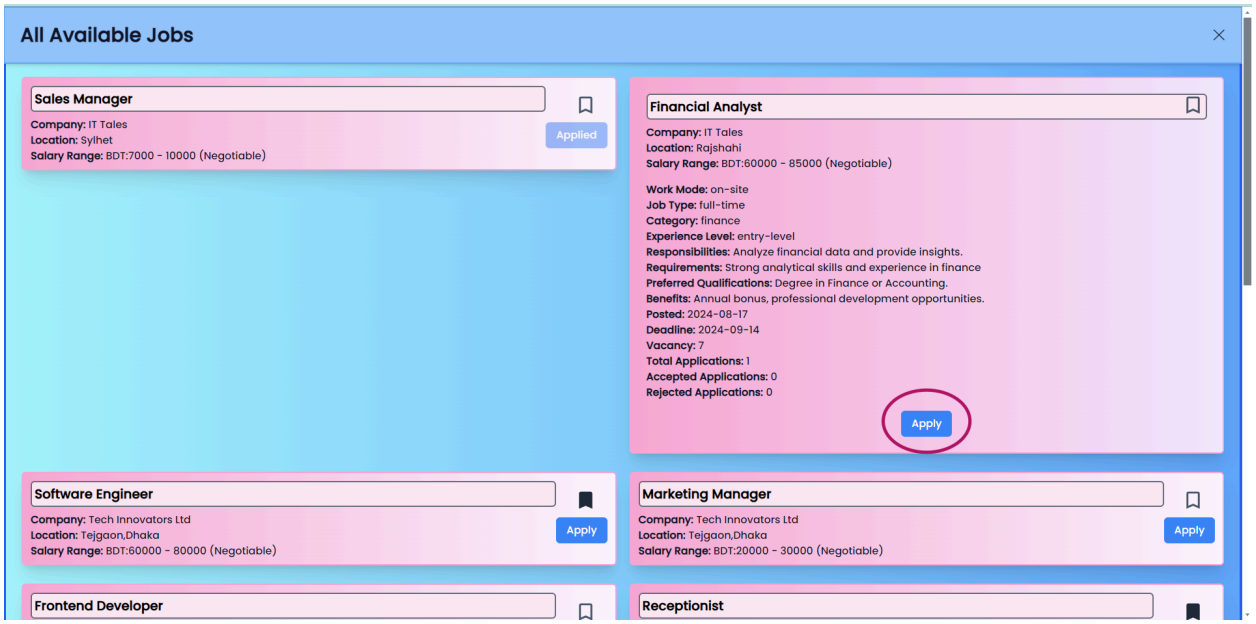
Jobseekers Dashboard:



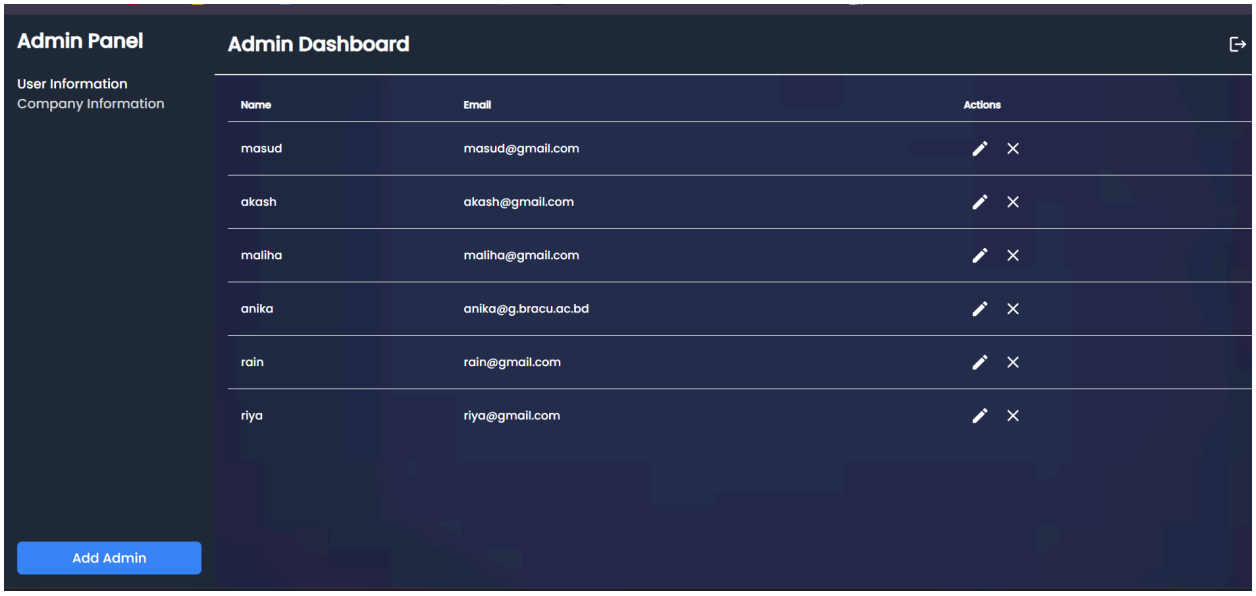
User profile :



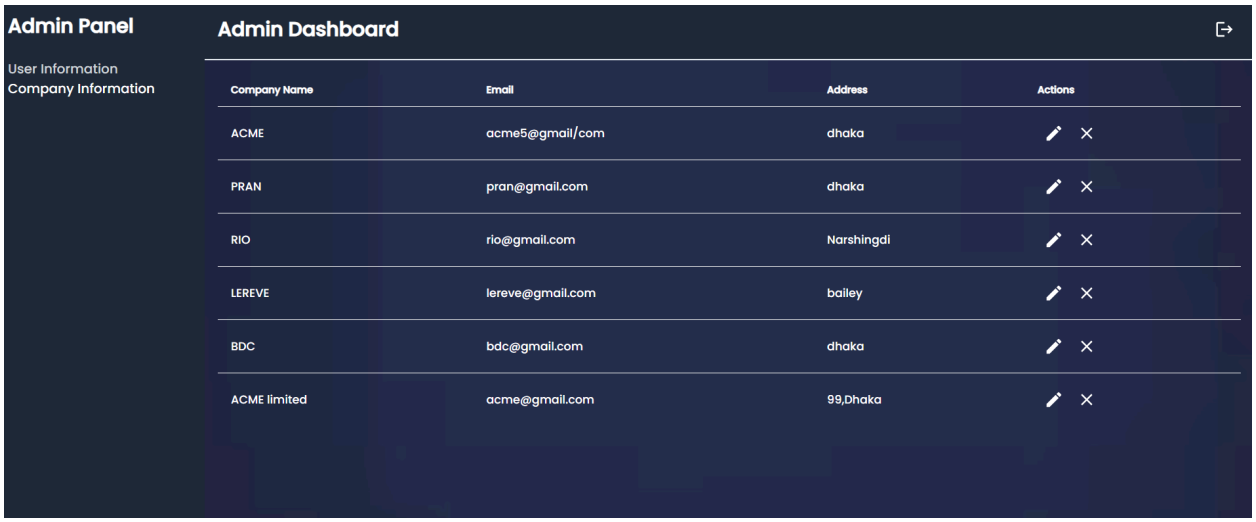
All Jobs: All posted jobs will be shown in the Jobs button .The apply button will become applied if this user has applied
for that job.



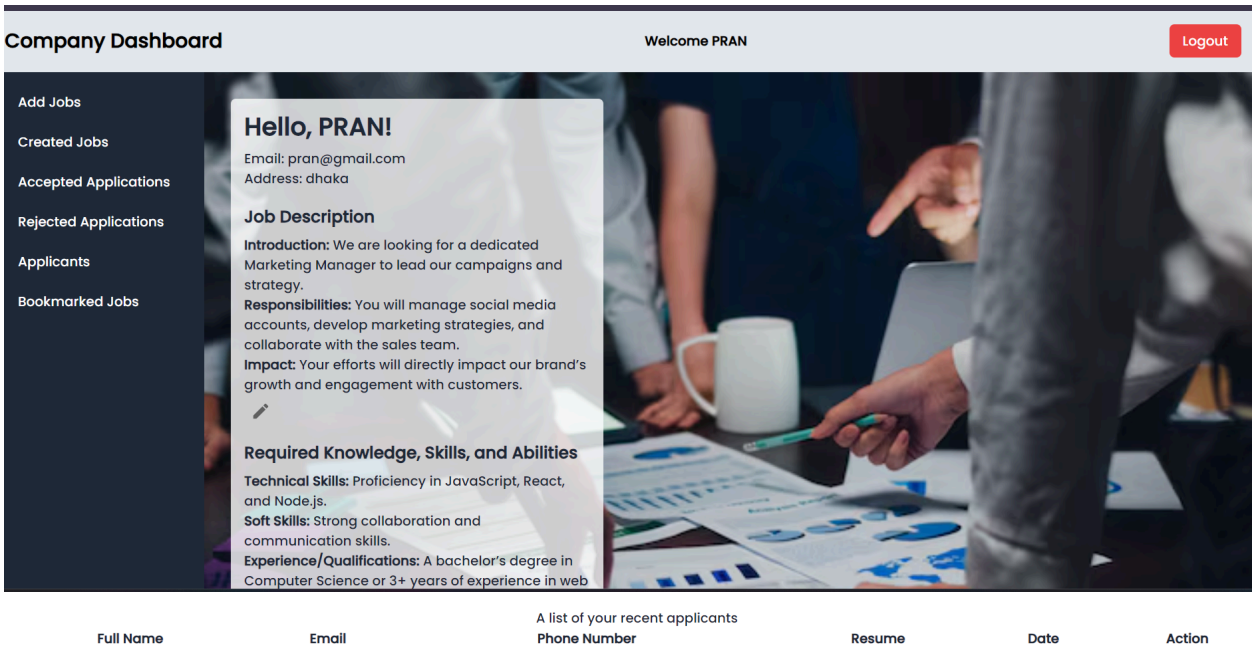
Admin dashboard(user info)



Admin dashboard(companyinfo)

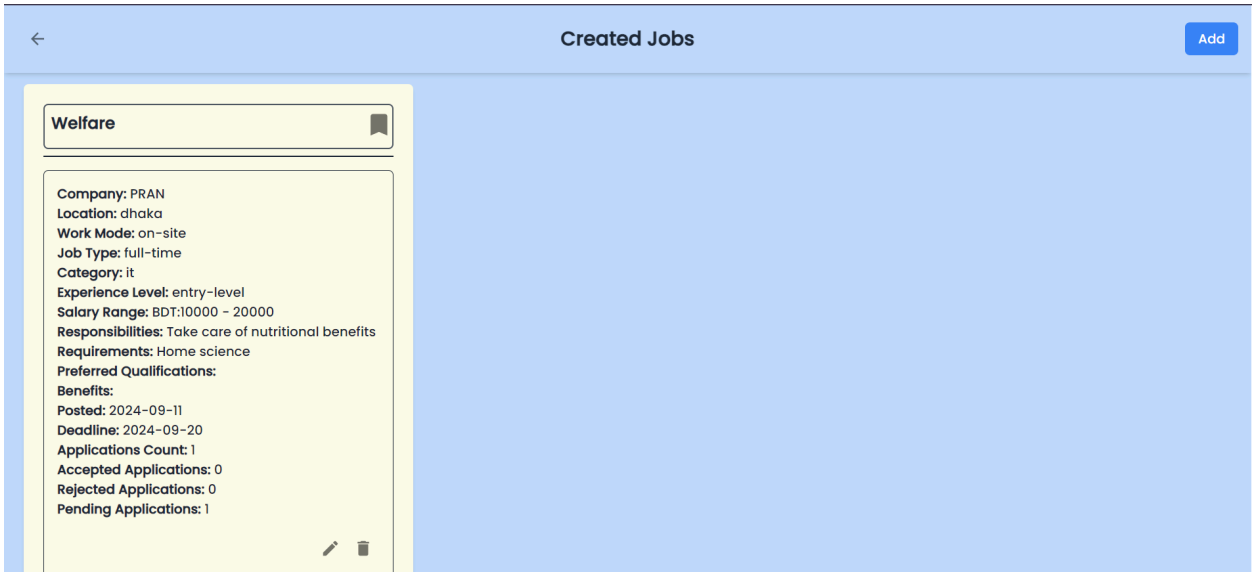


Company dashboard

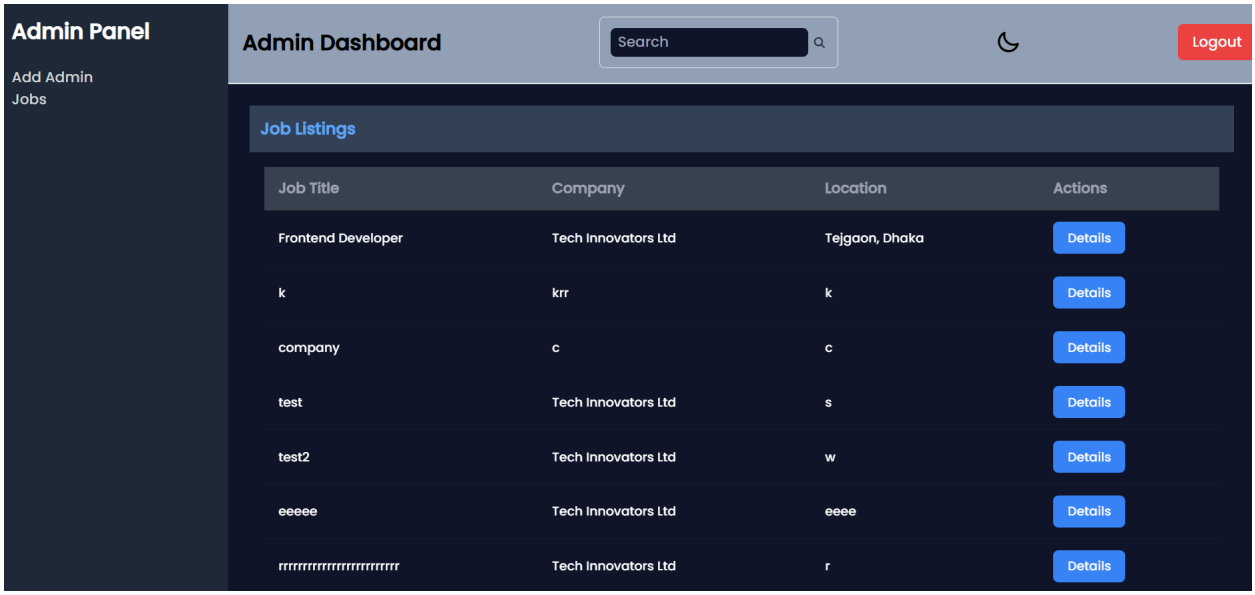


A list of your recent applicants

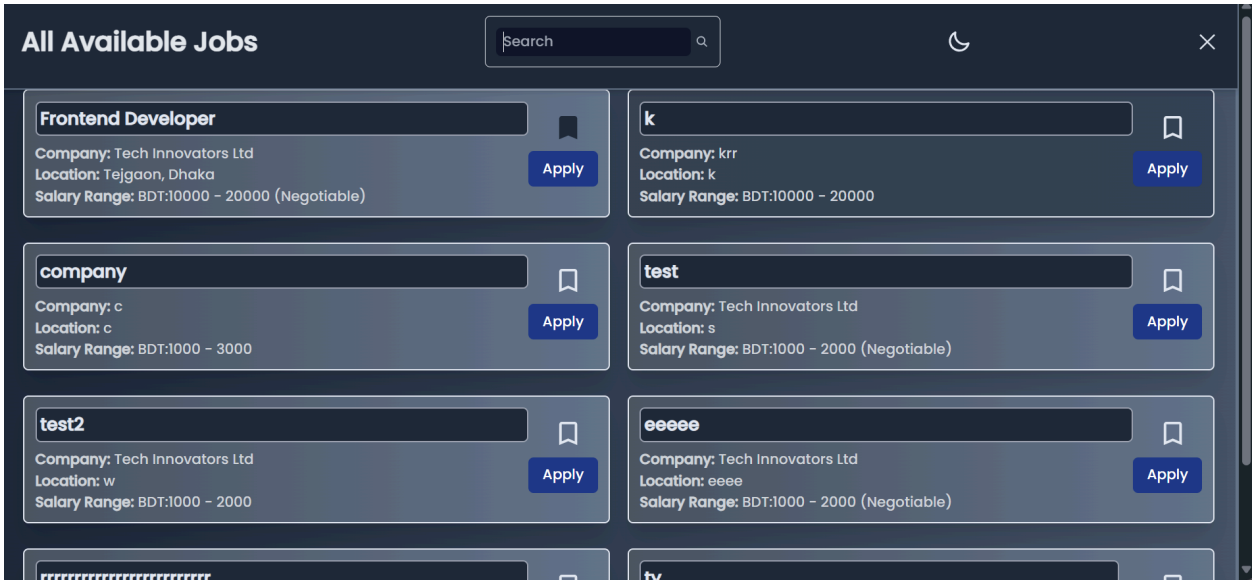
Full Name	Email	Phone Number	Resume	Date	Action

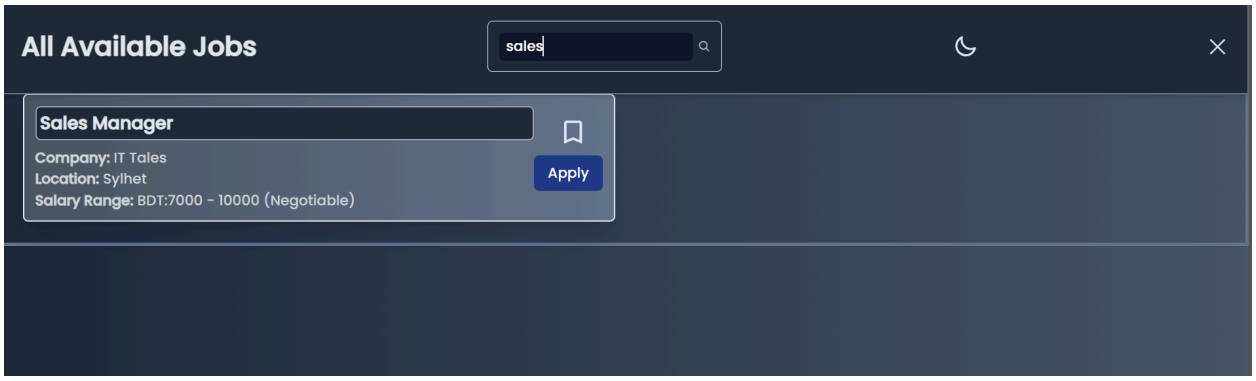


Admin Dashboard (Jobs)



Search Job





Technology (Framework, Languages)

Framework: Mern Stack

Frontend Development

Contribution of ID :21201114, Name : Tasnim Rahman
Home page:
Navbar:

```
import React, { useState, useEffect } from 'react';
```

```
import Login_Jobseeker from './Login_Jobseeker';
import Login_Company from './Login_Company';
import Login_Admin from './Login_Admin';
function Navbar() {
  const [theme, setTheme] = useState(localStorage.getItem("theme") ?
localStorage.getItem("theme") : "light");
  const element = document.documentElement;
  useEffect(() => {
    if (theme === "dark") {
      element.classList.add("dark");
      localStorage.setItem("theme", "dark");
      document.body.classList.add("dark");
    } else {
      element.classList.remove("dark");
      localStorage.setItem("theme", "light");
      document.body.classList.remove("dark");
    }
  }, [theme]);
  const navItemStyle = {
    color: theme === "dark" ? "white" : "black"
  };
  const [sticky,setSticky]=useState(false);
  useEffect(()=> {
    const handleScroll=()=>{
    if(window.scrollY>0){
      setSticky(true)
    }
    else{
      setSticky(false)
    }
  }
    window.addEventListener("scroll",handleScroll)
    return ()=>{
      window.addEventListener("scroll",handleScroll)
    }
  },[])
  const navItems = (
    <>
      <li>
        <a className=" dark:text-black " href="/">Home</a>
```

```
      </li>
      <li>
        <a className=" dark:text-black " href="/contact-us">Contact Us</a>
      </li>
      <li>
        <a className=" dark:text-black " href="/about">About</a>
      </li>
      <li>
        <a
          className="btn-admin dark:text-white dark:bg-slate-600"
          role="button"
          tabIndex={0}
          onClick={() =>
document.getElementById("my_modal_admin").showModal()}
        >
          Are you an Admin?
        </a>
        <Login_Admin />
      </li>
    </>
  );
  return (
    <>
      <div className={`max-w-screen-2xl container mx-auto md:px-1 py-3
bg-slate-100 dark:bg-slate-600 dark:text-white fixed top-0 left-0 right-0
z-50 ${sticky ? "sticky-navbar shadow-md bg-base-300 duration-300
transition-all ease-in-out" : ""}`}>

        <div className="navbar ">
          <div className="navbar-start">
            <div className="dropdown">
              <div tabIndex={0} role="button" className="btn btn-ghost
lg:hidden">
                <svg
                  xmlns="http://www.w3.org/2000/svg"
                  className="h-5 w-5"
                  fill="none"
                  viewBox="0 0 24 24"
                  stroke="currentColor">
                  <path
```

```
                    strokeLinecap="round"
                    strokeLinejoin="round"
                    strokeWidth="2"
                    d="M4 6h16M4 12h8m-8 6h16" />
                </svg>
              </div>
              <ul
                tabIndex={0}
                className="menu menu-sm dropdown-content bg-base-100
rounded-box z-[1] mt-3 w-52 p-2 shadow  ">
                {navItems}
              </ul>
            </div>
            <a className="btn btn-ghost text-4xl font-bold
cursor-pointer">Workforce-Enroll</a>
          </div>
          <div className="navbar-end">
            <div className="navbar-end flex items-center gap-4 lg:gap-8">
              <div className="navbar-center hidden lg:flex">
              <ul className="menu menu-horizontal px-1">{navItems}</ul>
              </div>
              <div className='hidden md:block'>
                <label className="px-3 py-3 border rounded-md flex
items-center gap-1">
                  <input type="text" className="grow outline-none px-2
py-1 border-3 rounded-md  dark:bg-slate-900 dark:text-white"
placeholder="Search" />
                  <svg
                    xmlns="http://www.w3.org/2000/svg"
                    viewBox="0 0 16 16"
                    fill="currentColor"
                    className="h-4 w-4 opacity-70">
                    <path
                      fillRule="evenodd"
                      d="M9.965 11.026a5 5 0 1 1 1.06-1.06l2.755
2.754a.75.75 0 1 1-1.06 1.06l-2.755-2.754ZM10.5 7a3.5 3.5 0 1 1-7 0 3.5
3.5 0 0 1 7 0Z"
                      clipRule="evenodd" />
                  </svg>
                </label>
```

```
              </div>
              <label className="swap swap-rotate">
                <input type="checkbox" checked={theme === "dark"}
onChange={() => setTheme(theme === "light" ? "dark" : "light")} />
                <svg
                  className="swap-off h-7 w-7 fill-current"
                  xmlns="http://www.w3.org/2000/svg"
                  viewBox="0 0 24 24">
                  <path

d="M5.64,17l-.71.71a1,1,0,0,0,0,1.41,1,1,0,0,0,1.41,0l.71-.71A1,1,0,0,0,5.
64,17ZM5,12a1,1,0,0,0-1-1H3a1,1,0,0,0,0,2H4A1,1,0,0,0,5,12Zm7-7a1,1,0,0,0,
1-1V3a1,1,0,0,0-2,0V4A1,1,0,0,0,12,5ZM5.64,7.05a1,1,0,0,0,.7.29,1,1,0,0,0,
.71-.29,1,1,0,0,0,0-1.41l-.71-.71A1,1,0,0,0,4.93,6.34Zm12,.29a1,1,0,0,0,.7
-.29l.71-.71a1,1,0,1,0-1.41-1.41L17,5.64a1,1,0,0,0,0,1.41A1,1,0,0,0,17.66,
7.34ZM21,11H20a1,1,0,0,0,0,2h1a1,1,0,0,0,0-2Zm-9,8a1,1,0,0,0-1,1v1a1,1,0,0
,0,2,0V20A1,1,0,0,0,12,19ZM18.36,17A1,1,0,0,0,17,18.36l.71.71a1,1,0,0,0,1.
41,0,1,1,0,0,0,0-1.41ZM12,6.5A5.5,5.5,0,1,0,17.5,12,5.51,5.51,0,0,0,12,6.5
Zm0,9A3.5,3.5,0,1,1,15.5,12,3.5,3.5,0,0,1,12,15.5Z" />
                </svg>
                <svg
                  className="swap-on h-7 w-7 fill-current"
                  xmlns="http://www.w3.org/2000/svg"
                  viewBox="0 0 24 24">
                  <path

d="M21.64,13a1,1,0,0,0-1.05-.14,8.05,8.05,0,0,1-3.37.73A8.15,8.15,0,0,1,9.
08,5.49a8.59,8.59,0,0,1,.25-2A1,1,0,0,0,8,2.36,10.14,10.14,0,1,0,22,14.05,
1,1,0,0,0,21.64,13Zm-9.5,6.69A8.14,8.14,0,0,1,7.08,5.22v.27A10.15,10.15,0,
0,0,17.22,15.63a9.79,9.79,0,0,0,2.1-.22A8.11,8.11,0,0,1,12.14,19.73Z" />
                </svg>
              </label>
              <div className="">
                <a className="btn bg-black text-white py-2 px-6
hover:bg-slate-600 duration-500 cursor-pointer" role="button"
                  tabIndex={0} onClick={() =>
document.getElementById("my_modal_company").showModal()}>Login as
Company</a>
                <Login_Company />
              </div>
```

```
              <div className="">
                <a className="btn bg-black text-white py-2 px-6
hover:bg-slate-600 duration-500 cursor-pointer" role="button"
                  tabIndex={0} onClick={() =>
document.getElementById("my_modal_jobseeker").showModal()}>Login as
Jobseeker</a>
                <Login_Jobseeker />
              </div>
            </div>
          </div>
        </div>
      </div>
    </>
  )
}
export default Navbar;
```

Banner:

```
import React from 'react';
import banner from "../../public/Banner.jpg";
function Banner() {
  return (
    <>
    <div className='max-w-screen-2xl container mx-auto md:px-10 px-4 flex
md:flex-row my-20'>
      <div className='w-1/2  md:w-1/2 mt-12 md:mt-32 pr-8'>
      <div className='space-y-12'>
      <h1 className='text-4xl font-bold'>YOUR DREAM JOB IS WAITING FOR
<span className='text-blue-400'>YOU!!!
      </span>
      </h1>
      <p className="text-xl">
      Experience the future of job hunting with our intelligent platform,
offering real-time updates, detailed company insights, and career growth
tools.
      </p>
      <label className="input input-bordered flex items-center gap-2">
  <svg
    xmlns="http://www.w3.org/2000/svg"
    viewBox="0 0 16 16"
```

```
    fill="currentColor"
    className="h-4 w-4 opacity-70">
    <path
      d="M2.5 3A1.5 1.5 0 0 0 1 4.5v.793c.026.009.051.02.076.032L7.674
8.51c.206.1.446.1.652 0l6.598-3.185A.755.755 0 0 1 15 5.293V4.5A1.5 1.5 0
0 0 13.5 3h-11Z" />
    <path
      d="M15 6.954 8.978 9.86a2.25 2.25 0 0 1-1.956 0L1 6.954V11.5A1.5 1.5
0 0 0 2.5 13h11a1.5 1.5 0 0 0 1.5-1.5V6.954Z" />
  </svg>
  <input type="text" className="grow px-2" placeholder="Email" />
</label>
</div>
<button className="btn mt-4  bg-blue-500 text-white">Get Started</button>
      </div>
      <div className=' w-full md:w-1/2 pt-16 md:pt-32'>
      <img src={banner} className="w-100 h-100" alt=""/>
      </div>
    </div>
    </>
  )
}
export default Banner
```

Home:

```
import React from 'react'
import Navbar from '../components/Navbar'
import Banner from '../components/Banner'
import Footer from '../components/Footer'
import Freebook from '../components/Freebook'
function Home() {
  return (
    <>
    <Navbar />
    <Banner/>
    <Freebook/>
    <Footer/>
    </>
  )
}
export default Home
```

Signup Company,Jobseeker and add admin parts looks almost same so for demonstration only showing Singup company:

```
import React,{ useState } from 'react';
import { Link, useNavigate } from 'react-router-dom';
import Login_Company from './Login_Company';
import { useForm } from "react-hook-form";
import axios from "axios";
import toast from 'react-hot-toast';
import { IconButton, InputAdornment, TextField } from '@mui/material';
import Visibility from '@mui/icons-material/Visibility';
import VisibilityOff from '@mui/icons-material/VisibilityOff';
function Signup_Company() {
  const { register, handleSubmit, formState: { errors } } = useForm();
  const navigate = useNavigate();
  const [showPassword, setShowPassword] = useState(false);
  const onSubmit = async (data) => {
    const userInfo = {
      company_name: data.company_name,
      company_email: data.company_email,
      company_address: data.company_address,
      password: data.password,
    };
    try {
      const response = await
axios.post("http://localhost:4001/user1/signup_company", userInfo);
      console.log(response.data);
      if (response.data) {
        toast.success('Signup Successful');
        localStorage.setItem("Users", JSON.stringify(response.data));
        navigate('/');
      }
    } catch (error) {
      console.error("Error:", error);
      toast.error('Already Exists');
    }
  };
  return (
    <>
```

```
      <div className="flex h-screen items-center justify-center
bg-blue-100 dark:bg-slate-600">
        <div className='w-[600px]'>
          <div className='modal-box dark:bg-slate-900 dark:text-white'>
            <form onSubmit={handleSubmit(onSubmit)}>
              <p><Link to="/" className="btn btn-sm btn-circle btn-ghost
absolute right-2 top-2">X</Link></p>
              <div className="flex justify-center mt-4 border-[3px]
border-orange-300 py-3 rounded-md">
                <h3 className="font-bold text-2xl cursor-default
mt-[-5px]">Signup-Form</h3>
              </div>
              <div className='mt-4 space-y-2'>
                <span>Company-Name</span>
                <br />
                <input
                  type='text'
                  placeholder='Enter Company-Name'
                  className='w-80 px-3 py-1 border border-orange-300
rounded-md outline-none dark:text-slate-900'
                  {...register("company_name", { required: true })}
                />
                <br />
                {errors.company_name && <span className="text-sm
text-red-500">This field is required</span>}
              </div>
              <div className='mt-4 space-y-2'>
                <span>Company Email</span>
                <br />
                <input
                  type='email'
                  placeholder='Enter Company email'
                  className='w-80 px-3 py-1 border border-orange-300
rounded-md outline-none dark:text-slate-900'
                  {...register("company_email", { required: true })}
                />
                <br />
                {errors.company_email && <span className="text-sm
text-red-500">This field is required</span>}
              </div>
```

```
              <div className='mt-4 space-y-2'>
                <span>Address</span>
                <br />
                <input
                  type='text'
                  placeholder='Head Office Address'
                  className='w-80 px-3 py-1 border border-orange-300
rounded-md outline-none dark:text-slate-900'
                  {...register("company_address", { required: true })}
                />
                <br />
                {errors.company_address && <span className="text-sm
text-red-500">This field is required</span>}
              </div>

              <div className='mt-4 space-y-2'>
              <span>Password</span>
              <br />
              <TextField
                type={showPassword ? 'text' : 'password'}
                placeholder='Enter Password'
                className='w-80 px-3 py-1 border border-orange-300
rounded-md outline-none  dark:bg-slate-200 dark:text-slate-900'
                {...register("password", { required: true })}
                InputProps={{
                  endAdornment: (
                    <InputAdornment position="end">
                      <IconButton
                        aria-label="toggle password visibility"
                        onClick={() => setShowPassword(!showPassword)}
                        edge="end"
                      >
                        {showPassword ? <VisibilityOff /> : <Visibility
/>}
                      </IconButton>
                    </InputAdornment>
                  ),
                }}
              />
```

```
              <br />
              {errors.password && <span className="text-sm
text-red-500">This field is required</span>}
            </div>
              <div className='flex justify-around mt-8'>
                <button className='bg-purple-600 text-white rounded-md
px-3 py-1 hover:bg-purple-800 duration-300 ml-[-20px]'>Signup</button>
                <p>
                  Have an account? <button className='underline
text-blue-500 cursor-pointer' onClick={() =>
document.getElementById("my_modal_company").showModal()}>Login</button>
                  <Login_Company />
                </p>
              </div>
            </form>
          </div>
        </div>
      </div>
    </>
  );
}
export default Signup_Company;
```

Login Company,Jobseeker,admin parts looks almost same so for demonstration only showing Login company:

```
import React, { useState } from 'react';
import { Link, useNavigate } from 'react-router-dom';
import { useForm } from 'react-hook-form';
import axios from 'axios';
import toast from 'react-hot-toast';
import { IconButton, InputAdornment, TextField } from '@mui/material';
import Visibility from '@mui/icons-material/Visibility';
import VisibilityOff from '@mui/icons-material/VisibilityOff';
function Login_Company() {
  const { register, handleSubmit, formState: { errors } } = useForm();
  const navigate = useNavigate();
  const [showPassword, setShowPassword] = useState(false);
  const onSubmit = async (data) => {
    try {
      const userInfo = {
        company_name: data.company_name,
```

```
        company_email: data.company_email,
        password: data.password,
      };
      const response = await
axios.post("http://localhost:4001/user1/login_company", userInfo);
      console.log(response.data);
      if (response.data) {
        toast.success('Login Successful');
        localStorage.setItem("Users", JSON.stringify(response.data));
        navigate('/company_dashboard');
      }
    } catch (error) {
      console.error("Error:", error);
      toast.error('Wrong Email or Password');
    }
  };
  const handleCloseModal = (event) => {
    event.preventDefault();
    document.getElementById("my_modal_company").close();
    navigate("/");
  };
  return (
    <div>
      <dialog id="my_modal_company" className="modal">
        <div className="modal-box dark:bg-slate-900 dark:text-white">
          <form onSubmit={handleSubmit(onSubmit)} method='dialog'>
            <button
              type="button"
              className='btn btn-sm btn-circle btn-ghost absolute right-2
top-2'
              onClick={handleCloseModal}
            >
              ✕
            </button>
            <h3 className="font-bold text-lg cursor-default">Company
Login</h3>

            <div className='mt-4 space-y-2'>
              <span>Company-Name</span>
              <br />
```

```
              <input
                type='text'
                placeholder='Enter Company-name'
                className='w-80 px-3 py-1 border rounded-md outline-none
dark:text-slate-900'
                {...register("company_name", { required: true })}
              />
              <br />
              {errors.company_name && <span className="text-sm
text-red-500">This field is required</span>}
            </div>
            <div className='mt-4 space-y-2'>
              <span>Company-Email</span>
              <br />
              <input
                type='email'
                placeholder='Enter Company-email'
                className='w-80 px-3 py-1 border rounded-md outline-none
dark:text-slate-900'
                {...register("company_email", { required: true })}
              />
              <br />
              {errors.company_email && <span className="text-sm
text-red-500">This field is required</span>}
            </div>

            <div className='mt-4 space-y-2'>
              <span>Password</span>
              <br />
              <TextField
                type={showPassword ? 'text' : 'password'}
                placeholder='Enter Password'
                className='w-80 px-3 py-1 border rounded-md outline-none
dark:text-slate-900 dark:bg-slate-200'
                {...register("password", { required: true })}
                InputProps={{
                  endAdornment: (
                    <InputAdornment position="end">
                      <IconButton
                        aria-label="toggle password visibility"
```

```
                        onClick={() => setShowPassword(!showPassword)}
                        edge="end"
                      >
                        {showPassword ? <VisibilityOff /> : <Visibility
/>}
                      </IconButton>
                    </InputAdornment>
                  ),
                }}
              />
              <br />
              {errors.password && <span className="text-sm
text-red-500">This field is required</span>}
            </div>

            <div className='flex justify-around mt-4'>
              <button className='bg-purple-500 text-white rounded-md px-3
py-1 hover:bg-purple-700 duration-300'>Login</button>
              <p>
                Not registered? <Link to="/signup_company"
className='underline text-blue-500 cursor-pointer'>Signup</Link>
              </p>
            </div>
          </form>
        </div>
      </dialog>
    </div>
  );
}
export default Login_Company;
```

Add New Jobs:(job form)

```
import React, { useState, useEffect } from 'react';
import { useForm } from 'react-hook-form';
import toast from 'react-hot-toast';
import { Link, useNavigate } from 'react-router-dom';

function Add_Jobs() {
    const [page, setPage] = useState(1);
    const [companyName, setCompanyName] = useState('');
```

```
    const { register, handleSubmit, formState: { errors }, trigger,
setValue, getValues } = useForm();
    const navigate = useNavigate();

    useEffect(() => {
        const user = JSON.parse(localStorage.getItem('Users'));
        if (user && user.user1) {
            setCompanyName(user.user1.company_name);
            setValue('companyName', user.user1.company_name);
        } else {
            console.error('Failed to fetch company details from
localStorage');
        }
    }, [setValue]);

    const handleSubmitForm = async (data) => {
        const user = JSON.parse(localStorage.getItem('Users'));
        const companyId = user && user.user1 ? user.user1._id : null;

        if (!companyId) {
            toast.error('Company ID is missing');
            return;
        }

        const jobData = { ...data, company: companyId };

        try {
            const response = await
fetch('http://localhost:4001/jobs/addjobs', {
                method: 'POST',
                headers: {
                    'Content-Type': 'application/json',
                },
                body: JSON.stringify(jobData),
            });

            if (response.ok) {
                const result = await response.json();
                toast.success('Job added successfully!');
                navigate('/company_dashboard');
```

```
            } else {
                const errorData = await response.json();
                toast.error(errorData.message || 'Failed to add job');
            }
        } catch (error) {
            toast.error('Failed to add job');
            console.error('Error:', error);
        }
    };

    const handleNext = async () => {
        const isValid = await trigger();
        if (isValid) {
            setPage(page + 1);
        }
    };

    const handleBack = () => {
        setPage(page - 1);
    };

    return (
        <div className="flex h-screen items-center justify-center
bg-gradient-to-r from-cyan-200 to-blue-300 dark:bg-gradient-to-r
dark:from-gray-600 dark:to-gray-700">
            <div className="relative w-[600px] bg-gradient-to-r
from-yellow-100 to-orange-200 p-6 rounded-md shadow-md
dark:bg-gradient-to-r dark:from-slate-900 dark:to-slate-500
dark:text-white">
                <p className="absolute right-2 top-2">
                    <Link to="/company_dashboard" className='btn btn-sm
btn-circle btn-ghost hover:bg-gray-200 dark:hover:bg-slate-600'>
                        ×
                    </Link>
                </p>

                <form onSubmit={handleSubmit(handleSubmitForm)}>
                    <h2 className="text-2xl font-bold mb-6 border-b-2 pb-2
border-gray-600 dark:border-slate-700">Add Job</h2>
```

```
                    {page === 1 && (
                        <div className="border-2 p-4 rounded-md
border-gray-600 dark:border-slate-700">
                            <h3 className="text-xl font-semibold mb-2
border-b-2 pb-2 border-gray-600 dark:border-slate-700">Basic
Information</h3>
                            <div className="mb-3">
                                <label className="block mb-1">Job
Title:</label>
                                <input
                                    type="text"
                                    placeholder='Enter title'
                                    {...register("jobTitle", { required:
true })}
                                    className="w-full px-3 py-2 border
border-gray-300 rounded-md dark:text-black"
                                />
                                {errors.jobTitle && <span
className="text-red-500 text-sm">This field is required</span>}
                            </div>
                            <div className="mb-3">
                                <label className="block mb-1">Company
Name:</label>
                                <input
                                    type="text"
                                    {...register("companyName")}
                                    value={companyName}
                                    readOnly
                                    className="w-full px-3 py-2 border
border-gray-300 rounded-md dark:text-black"
                                />
                            </div>
                            <div className="mb-3">
                                <label className="block
mb-1">Location:</label>
                                <input
                                    type="text"
                                    placeholder='Enter Location'
                                    {...register("location", { required:
true })}
```

```
                                    className="w-full px-3 py-2 border
border-gray-300 rounded-md dark:text-black"
                                />
                                {errors.location && <span
className="text-red-500 text-sm">This field is required</span>}
                            </div>
                            <div className="mb-3">
                                <label className="block mb-1">Work
Mode:</label>
                                <select
                                    {...register("workMode")}
                                    className="w-full px-3 py-2 border
border-gray-300 rounded-md dark:text-slate-500"
                                >
                                    <option
value="on-site">On-site</option>
                                    <option value="remote">Remote</option>
                                    <option value="hybrid">Hybrid</option>
                                </select>
                            </div>
                            <div className="flex justify-between mt-4">
                                <button
                                    type="button"
                                    onClick={handleNext}
                                    className="bg-blue-500 text-white px-4
py-2 rounded-md hover:bg-blue-700 transition duration-200 dark:bg-blue-700
dark:hover:bg-blue-900"
                                >
                                    Next
                                </button>
                            </div>
                        </div>
                    )}

                    {page === 2 && (
                        <div className="border-2 p-4 rounded-md
border-gray-600 dark:border-slate-700">
                            <h3 className="text-xl font-semibold mb-2
border-b-2 pb-2 border-gray-600 dark:border-slate-700">Job Details</h3>
                            <div className="mb-3">
```

```
                                <label className="block mb-1">Job
Type:</label>
                                <select
                                    {...register("jobType", { required:
true })}
                                    className="w-full px-3 py-2 border
border-gray-300 rounded-md dark:text-slate-500"
                                >
                                    <option
value="full-time">Full-time</option>
                                    <option
value="part-time">Part-time</option>
                                    <option
value="contract">Contract</option>
                                    <option
value="internship">Internship</option>
                                    <option
value="freelance">Freelance</option>
                                </select>
                                {errors.jobType && <span
className="text-red-500 text-sm">This field is required</span>}
                            </div>
                            <div className="mb-3">
                                <label className="block mb-1">Job
Category:</label>
                                <select
                                    {...register("jobCategory", {
required: true })}
                                    className="w-full px-3 py-2 border
border-gray-300 rounded-md dark:text-slate-500"
                                >
                                    <option value="it">IT</option>
                                    <option
value="marketing">Marketing</option>
                                    <option value="sales">Sales</option>
                                    <option
value="finance">Finance</option>
                                    <option value="hr">HR</option>
                                </select>
```

```
                                {errors.jobCategory && <span
classNameName="text-red-500 text-sm">This field is required</span>}
                            </div>
                            <div classNameName="mb-3">
                                <label classNameName="block mb-1">Experience
Level:</label>
                                <select
                                    {...register("experienceLevel", {
required: true })}
                                    classNameName="w-full px-3 py-2 border
border-gray-300 rounded-md dark:text-slate-500"
                                >
                                    <option
value="entry-level">Entry-level</option>
                                    <option
value="mid-level">Mid-level</option>
                                    <option
value="senior-level">Senior-level</option>
                                    <option
value="manager">Manager</option>
                                    <option
value="director">Director</option>
                                </select>
                                {errors.experienceLevel && <span
classNameName="text-red-500 text-sm">This field is required</span>}
                            </div>
                            <div classNameName="mb-3">
                                <label classNameName="block mb-1">Salary
Range:</label>
                                <div classNameName="flex gap-2">
                                    <input
                                        type="number"
                                        placeholder="Min"
                                        {...register("minSalary", { min:
1000, max: 100000, step: 1000 })}
                                        defaultValue={1000}
                                        classNameName="w-1/2 px-3 py-2 border
border-gray-300 rounded-md dark:text-black"
                                    />
                                    <input
```

```
                                        type="number"
                                        placeholder="Max"
                                        {...register("maxSalary", { min:
1000, max: 100000, step: 1000 })}
                                        defaultValue={2000}
                                        className="w-1/2 px-3 py-2 border
border-gray-300 rounded-md dark:text-black"
                                    />
                                </div>
                                <div className="mt-2">
                                    <input
                                        type="checkbox"
                                        {...register("negotiable")}
                                    />
                                    <span
className="ml-2">Negotiable</span>
                                </div>
                                <div className="mb-3">
                               <label className="block
mb-1">Deadline:</label>
                                <input
                                     type="date"
                                     placeholder='Select deadline'
                                     {...register("deadline", { required:
true })}
                                     className="w-full px-3 py-2 border
border-gray-300 rounded-md dark:text-black"
                                    />
                                    {errors.deadline && <span
className="text-red-500 text-sm">This field is required</span>}
                            </div>
                            <div className="mb-3">
                                <label className="block mb-1
dark:text-gray-300">Vacancy:</label>
                                <input type="number" placeholder='Enter
the number of Vacancy' {...register('vacancies', { required: true })}
className="w-full px-3 py-2 border border-gray-300 rounded-md
dark:bg-slate-800 dark:text-gray-300" />
                                {errors.vacancies && <p
className="text-red-500 text-sm">This field is required</p>}
```

```
                            </div>
                            </div>
                            <div className="flex justify-between mt-4">
                                <button
                                    type="button"
                                    onClick={handleBack}
                                    className="bg-gray-500 text-white px-4
py-2 rounded-md hover:bg-gray-700 transition duration-200 dark:bg-gray-700
dark:hover:bg-gray-900"
                                >
                                    Back
                                </button>
                                <button
                                    type="button"
                                    onClick={handleNext}
                                    className="bg-blue-500 text-white px-4
py-2 rounded-md hover:bg-blue-700 transition duration-200 dark:bg-blue-700
dark:hover:bg-blue-900">Next</button>
                            </div>
                        </div>
                    )}
                    {page === 3 && (
                        <div className="border-2 p-4 rounded-md
border-gray-600 dark:border-slate-700">
                            <h3 className="text-xl font-semibold mb-2
border-b-2 pb-2 border-gray-600 dark:border-slate-700">Job
Description</h3>
                            <div className="mb-3">
                                <label className="block
mb-1">Responsibilities:</label>
                                <textarea
                                    rows="4"
                                    placeholder='Enter job
responsibilities'
                                    {...register("responsibilities", {
required: true })}
                                    className="w-full px-3 py-2 border
border-gray-300 rounded-md dark:text-black"
                                />
```

```
                                {errors.responsibilities && <span
classNameName="text-red-500 text-sm">This field is required</span>}
                            </div>
                            <div className="mb-3">
                                <label className="block
mb-1">Requirements:</label>
                                <textarea
                                    rows="4"
                                    placeholder='Enter job requirements'
                                    {...register("requirements", {
required: true })}
                                    className="w-full px-3 py-2 border
border-gray-300 rounded-md dark:text-black"
                                />
                                {errors.requirements && <span
classNameName="text-red-500 text-sm">This field is required</span>}
                            </div>
                            <div className="mb-3">
                                <label className="block mb-1">Preferred
Qualifications:</label>
                                <textarea
                                    rows="4"
                                    placeholder='Enter preferred
qualifications'

{...register("preferredQualifications")}
                                    className="w-full px-3 py-2 border
border-gray-300 rounded-md dark:text-black"
                                />
                            </div>
                            <div className="mb-3">
                                <label className="block
mb-1">Benefits:</label>
                                <textarea
                                    rows="4"
                                    placeholder='Enter job benefits'
                                    {...register("benefits")}
                                    className="w-full px-3 py-2 border
border-gray-300 rounded-md dark:text-black"
                                />
```

```
                            </div>
                            <div className="flex justify-between mt-4">
                                <button
                                    type="button"
                                    onClick={handleBack}
                                    className="bg-gray-500 text-white px-4
py-2 rounded-md hover:bg-gray-700 transition duration-200 dark:bg-gray-700
dark:hover:bg-gray-900"
                                >
                                    Back
                                </button>
                                <button
                                    type="submit"
                                    className="bg-green-500 text-white
px-4 py-2 rounded-md hover:bg-green-700 transition duration-200
dark:bg-green-700 dark:hover:bg-green-900"
                                >
                                    Submit
                                </button>
                            </div>
                        </div>
                    )}
                </form>
            </div>
        </div>
    );
}
export default Add_Jobs;
```

Created jobs:Company will get to see its job here:

```
import React, { useEffect, useState } from 'react';
import { Link } from 'react-router-dom';
import { IconButton, Button } from '@mui/material';
import EditIcon from '@mui/icons-material/Edit';
import DeleteIcon from '@mui/icons-material/Delete';
import ArrowBackIcon from '@mui/icons-material/ArrowBack';
import BookmarkBorderIcon from '@mui/icons-material/BookmarkBorder';
import BookmarkIcon from '@mui/icons-material/Bookmark';
import { useNavigate } from 'react-router-dom';
function Created_Jobs() {
  const [jobs, setJobs] = useState([]);
```

```
  const navigate = useNavigate();
  const [bookmarkedJobs, setBookmarkedJobs] = useState(new Set());
  const [pendingCounts, setPendingCounts] = useState({});
  const [applicationCounts, setApplicationCounts] = useState({});
  const [acceptedCounts, setAcceptedCounts] = useState({});
  const [rejectedCounts, setRejectedCounts] = useState({});
  useEffect(() => {
    const fetchJobs = async () => {
      const user = JSON.parse(localStorage.getItem('Users'));
      if (user && user.user1) {
        const companyId = user.user1._id;
        try {
          const response = await
fetch(`http://localhost:4001/jobs?company=${companyId}`);
          if (response.ok) {
            let data = await response.json();
            data = data.filter(job => !job.isDeleted);
            setJobs(data);
            const bookmarks = new Set(data.filter(job =>
job.bookmarkedByCompany).map(job => job._id));
            setBookmarkedJobs(bookmarks);
            const counts = await fetchPendingCounts(data.map(job =>
job._id));
            setPendingCounts(counts);
            const appCounts = await fetchApplicationCounts(data.map(job =>
job._id));
            setApplicationCounts(appCounts);
            const acceptedAndRejectedCounts = await
fetchAcceptedAndRejectedCounts(data.map(job => job._id));
            setAcceptedCounts(acceptedAndRejectedCounts.accepted);
            setRejectedCounts(acceptedAndRejectedCounts.rejected);
          } else {
            console.error('Failed to fetch jobs');
          }
        } catch (error) {
          console.error('Error:', error);
        }
      }
    };
    const fetchPendingCounts = async (jobIds) => {
```

```
      const counts = {};
      for (const jobId of jobIds) {
        try {
          const response = await
fetch(`http://localhost:4001/applications/job/${jobId}?status=Pending`);
          if (response.ok) {
            const data = await response.json();
            counts[jobId] = data.applications.length;
          } else {
            counts[jobId] = 0;
          }
        } catch (error) {
          counts[jobId] = 0;
        }
      }
      return counts;
    };
    const fetchAcceptedAndRejectedCounts = async (jobIds) => {
      const counts = {
        accepted: {},
        rejected: {},
      };
      for (const jobId of jobIds) {
        try {
          const [acceptedResponse, rejectedResponse] = await Promise.all([
fetch(`http://localhost:4001/applications/jobs/${jobId}/accepted-count`),

fetch(`http://localhost:4001/applications/jobs/${jobId}/rejected-count`)
          ]);
          if (acceptedResponse.ok) {
            const acceptedData = await acceptedResponse.json();
            counts.accepted[jobId] = acceptedData.acceptedCount;
          } else {
            counts.accepted[jobId] = 0;
          }
          if (rejectedResponse.ok) {
            const rejectedData = await rejectedResponse.json();
            counts.rejected[jobId] = rejectedData.rejectedCount;
          } else {
            counts.rejected[jobId] = 0;
```

```
          }
        } catch (error) {
          counts.accepted[jobId] = 0;
          counts.rejected[jobId] = 0;
        }
      }
      return counts;
    };
    const fetchApplicationCounts = async (jobIds) => {
      const counts = {};
      for (const jobId of jobIds) {
        try {
          const response = await
fetch(`http://localhost:4001/applications/job/${jobId}/counts`);
          if (response.ok) {
            const data = await response.json();
            counts[jobId] = data.totalCount;
          } else {
            counts[jobId] = 0;
          }
        } catch (error) {
          counts[jobId] = 0;
        }
      }
      return counts;
    };
    fetchJobs();
  }, []);
  const handleDeleteJob = async (jobId) => {
    try {
      const response = await
fetch(`http://localhost:4001/jobs/${jobId}/soft-delete`, {
        method: 'PUT',
      });
      if (response.ok) {
        setJobs(jobs.filter(job => job._id !== jobId));
      } else {
        const errorText = await response.text();
        alert(`Error: ${errorText}`);
      }
```

```
    } catch (error) {
      console.error("Error:", error);
      alert("An error occurred while deleting the job. Please try
again.");
    }
  };
  const handleBookmarkClick = async (jobId) => {
    try {
      const response = await
fetch('http://localhost:4001/jobs/bookmark/company', {
        method: 'POST',
        headers: {
          'Content-Type': 'application/json',
        },
        body: JSON.stringify({ jobId }),
      });
      if (response.ok) {
        const data = await response.json();
        setBookmarkedJobs(prev => {
          const newSet = new Set(prev);
          if (newSet.has(jobId)) {
            newSet.delete(jobId);
          } else {
            newSet.add(jobId);
          }
          return newSet;
        });
      } else {
        console.error('Failed to toggle bookmark status');
      }
    } catch (error) {
      console.error('Error:', error);
    }
  };
  const formatDate1 = (dateString) => {
        if (!dateString) {
            return "N/A";
        }
        const date = new Date(dateString);
        return date.toISOString().split('T')[0];
```

```
    };
  const formatDate = (dateString) => {
    if (!dateString) {
        return "N/A";
    }
    const date = new Date(dateString);
    const today = new Date();
    const tomorrow = new Date(today);
    today.setHours(0, 0, 0, 0);
    tomorrow.setDate(today.getDate() + 1);
    tomorrow.setHours(0, 0, 0, 0);
    const formattedDate = date.toISOString().split('T')[0];
    console.log(`Date: ${formattedDate}, Today:
${today.toISOString().split('T')[0]}, Tomorrow:
${tomorrow.toISOString().split('T')[0]}`); // Debugging line

    if (formattedDate === today.toISOString().split('T')[0]) {
        return `Today is the deadline (${formattedDate})`;
    }
    if (formattedDate === tomorrow.toISOString().split('T')[0]) {
        return `Tomorrow is the deadline (${formattedDate})`;
    }
    return formattedDate;
};
  return (
    <div className="min-h-screen bg-blue-200 dark:text-slate-200
dark:bg-slate-800">
<div className="fixed top-0 left-0 right-0 bg-blue-200 dark:bg-slate-800
p-6 z-10 flex items-center shadow-md">
  <Link to="/company_dashboard">
    <IconButton size="small" className="hover:bg-gray-100
dark:hover:bg-slate-600 dark:bg-slate-500">
      <ArrowBackIcon />
    </IconButton>
  </Link>
  <h2 className="text-2xl font-bold absolute left-1/2 transform
-translate-x-1/2">
    Created Jobs
  </h2>
  <button
```

```
    onClick={() => navigate('/addjobs')}
    className="bg-blue-500 text-white px-4 py-2 rounded-md
hover:bg-blue-600 dark:bg-blue-700 dark:hover:bg-blue-600 absolute
right-4"
  >
    Add
  </button>
</div>
      <div className="pt-24 p-6">
        <div className="grid gap-6 md:grid-cols-2 lg:grid-cols-3">
          {jobs.length > 0 ? (
            jobs.map((job, index) => (
              <div key={index} className="relative bg-yellow-50 p-6
rounded-md shadow-md dark:bg-slate-300 dark:text-black">
                {/* Bookmark icon positioned in the top-right corner */}
                <div className="absolute top-6 right-4">
                  <IconButton onClick={() => handleBookmarkClick(job._id)}
style={{ fontSize: 30 }}>
                    {bookmarkedJobs.has(job._id) ? <BookmarkIcon
fontSize="large"/> : <BookmarkBorderIcon fontSize="large"/>}
                  </IconButton>
                </div>
                <div className="border-b-2 border-gray-800
dark:border-gray-200 mb-4 pb-2">
                  <div className="border-2 border-gray-600
dark:border-gray-400 p-2 rounded-md">
                    <h3 className="text-xl font-semibold
mb-2">{job.jobTitle}</h3>
                  </div>
                </div>
                <div className="border border-gray-800
dark:border-gray-200 p-4 rounded-md mb-4">
                  <p><strong>Company:</strong> {job.companyName}</p>
                  <p><strong>Location:</strong> {job.location}</p>
                  <p><strong>Work Mode:</strong> {job.workMode}</p>
                  <p><strong>Job Type:</strong> {job.jobType}</p>
                  <p><strong>Category:</strong> {job.jobCategory}</p>
                  <p><strong>Experience Level:</strong>
{job.experienceLevel}</p>
                  <p>
```

```
                    <strong>Salary Range:</strong> BDT:{job.minSalary} -
{job.maxSalary}{' '}
                    {job.negotiable ? '(Negotiable)' : ''}
                  </p>
                  <p><strong>Responsibilities:</strong>
{job.responsibilities}</p>
                  <p><strong>Requirements:</strong> {job.requirements}</p>
                  <p><strong>Preferred Qualifications:</strong>
{job.preferredQualifications}</p>
                  <p><strong>Benefits:</strong> {job.benefits}</p>
                  <p><strong>Posted:</strong>
{formatDate1(job.updatedAt)}</p>
                  <p><strong>Deadline:</strong>
{formatDate(job.deadline)}</p>
                  <p><strong>Vacancy:</strong> {job.vacancies}</p>
                  <p><strong>Applications Count:</strong>
{applicationCounts[job._id] || 0}</p> {/* New field */}
                  <p><strong>Accepted Applications:</strong>
{acceptedCounts[job._id] || 0}</p>
                  <p><strong>Rejected Applications:</strong>
{rejectedCounts[job._id] || 0}</p>
                  <p><strong>Pending Applications:</strong>
{pendingCounts[job._id] || 0}</p>
                  <div className="flex justify-end items-center mt-4">
                    <div className="flex items-center">
                      <Link to={`/edit-job/${job._id}`}>
                        <IconButton>
                          <EditIcon />
                        </IconButton>
                      </Link>
                      <IconButton onClick={() =>
handleDeleteJob(job._id)}>
                        <DeleteIcon />
                      </IconButton>
                    </div>
                  </div>
                </div>
                <div className="flex justify-end relative">
                <Link to={`/applications/${job._id}/pending`}>
```

```
                    <Button variant="contained" color="primary"
className="relative">
                      Applications
                      <span className="absolute -top-2 -right-2
bg-pink-400 dark:bg-purple-600 text-white text-xs font-bold rounded-full
px-2 py-1">
                        {pendingCounts[job._id] || 0}
                      </span>
                    </Button>
                  </Link>
                </div>
              </div>
            ))
          ) : (
            <p>No jobs found.</p>
          )}
        </div>
      </div>
    </div>
  );
}
export default Created_Jobs;
```

All Jobs : jobseekers will get the jobs in this code

```
import React, { useState, useEffect } from 'react';
import { useNavigate } from 'react-router-dom';
import BookmarkBorderIcon from '@mui/icons-material/BookmarkBorder';
import BookmarkIcon from '@mui/icons-material/Bookmark';
import { toast } from 'react-toastify';
const getUserId = () => {
    const user = JSON.parse(localStorage.getItem('Users'));
    return user ? user._id : null;
};
function All_Jobs() {
    const [jobs, setJobs] = useState([]);
    const [expandedJob, setExpandedJob] = useState(null);
    const [bookmarkedJobs, setBookmarkedJobs] = useState([]);
    const [userAppliedJobs, setUserAppliedJobs] = useState({});
    const [applicationCounts, setApplicationCounts] = useState({});
```

```
    const navigate = useNavigate();
    const userId = getUserId();
    const [acceptedCounts, setAcceptedCounts] = useState({});
    const [rejectedCounts, setRejectedCounts] = useState({});
    useEffect(() => {
        const fetchJobs = async () => {
            try {
                const response = await
fetch(`http://localhost:4001/jobs/all?userId=${userId}`);
                let data = await response.json();
                data=data.filter(job => !job.isDeleted);
                setJobs(data);
                const bookmarked = data.filter(job =>
job.isBookmarked).map(job => job._id);
                setBookmarkedJobs(bookmarked);
                const appliedJobs = await
fetchUserAppliedJobs(data.map(job => job._id));
                setUserAppliedJobs(appliedJobs);
                const counts = await fetchApplicationCounts(data.map(job
=> job._id));
                setApplicationCounts(counts);
                const acceptedAndRejectedCounts = await
fetchAcceptedAndRejectedCounts(data.map(job => job._id));
                setAcceptedCounts(acceptedAndRejectedCounts.accepted);
                setRejectedCounts(acceptedAndRejectedCounts.rejected);
            } catch (error) {
                console.error('Error fetching jobs:', error);
            }
        };
        const fetchUserAppliedJobs = async (jobIds) => {
            const appliedJobs = {};
            for (const jobId of jobIds) {
                try {
                    const response = await
fetch(`http://localhost:4001/applications/check-application/${userId}/${jo
bId}`);
                    if (response.ok) {
                        const data = await response.json();
                        appliedJobs[jobId] = data.hasApplied === 'yes';
                    } else {
```

```
                        appliedJobs[jobId] = false;
                    }
                } catch (error) {
                    appliedJobs[jobId] = false;
                }
            }
            return appliedJobs;
        };
        const fetchAcceptedAndRejectedCounts = async (jobIds) => {
            const counts = {
              accepted: {},
              rejected: {},
            };
            for (const jobId of jobIds) {
              try {
                const [acceptedResponse, rejectedResponse] = await
Promise.all([

fetch(`http://localhost:4001/applications/jobs/${jobId}/accepted-count`),

fetch(`http://localhost:4001/applications/jobs/${jobId}/rejected-count`)
                ]);

                if (acceptedResponse.ok) {
                  const acceptedData = await acceptedResponse.json();
                  counts.accepted[jobId] = acceptedData.acceptedCount;
                } else {
                  counts.accepted[jobId] = 0;
                }
                if (rejectedResponse.ok) {
                  const rejectedData = await rejectedResponse.json();
                  counts.rejected[jobId] = rejectedData.rejectedCount;
                } else {
                  counts.rejected[jobId] = 0;
                }
              } catch (error) {
                counts.accepted[jobId] = 0;
                counts.rejected[jobId] = 0;
              }
            }
```

```
            return counts;
          };
        const fetchApplicationCounts = async (jobIds) => {
            const counts = {};
            for (const jobId of jobIds) {
                try {
                    const response = await
fetch(`http://localhost:4001/applications/job/${jobId}/counts`);
                    if (response.ok) {
                        const data = await response.json();
                        counts[jobId] = data.totalCount;
                    } else {
                        counts[jobId] = 0;
                    }
                } catch (error) {
                    counts[jobId] = 0;
                }
            }
            return counts;
        };
        fetchJobs();
    }, [userId]);
    const handleToggle = (index) => {
        setExpandedJob(expandedJob === index ? null : index);
    };
    const handleApply = (job) => {
        if (userAppliedJobs[job._id]) {
            toast.info('You have already applied for this job');
        } else {
            navigate(`/apply/${job._id}`, { state: { job } });
        }
    };
    const formatDate1 = (dateString) => {
        if (!dateString) {
            return "N/A";
        }
        const date = new Date(dateString);
        return date.toISOString().split('T')[0];
    };
    const formatDate = (dateString) => {
```

```
        if (!dateString) {
            return "N/A";
        }
        const date = new Date(dateString);
        const today = new Date();
        const tomorrow = new Date(today);
        today.setHours(0, 0, 0, 0);
        tomorrow.setDate(today.getDate() + 1);
        tomorrow.setHours(0, 0, 0, 0);

        console.log(`Date: ${date.toISOString().split('T')[0]}, Today:
${today.toISOString().split('T')[0]}, Tomorrow:
${tomorrow.toISOString().split('T')[0]}`); // Debugging line

        if (date.toISOString().split('T')[0] ===
today.toISOString().split('T')[0]) {
            return "Today is the deadline";
        }
        if (date.toISOString().split('T')[0] ===
tomorrow.toISOString().split('T')[0]) {
            return "Tomorrow is the deadline";
        }
        return date.toISOString().split('T')[0];
    };
    const handleBookmarkToggle = async (jobId) => {
        try {
            const response = await
fetch('http://localhost:4001/jobs/bookmark', {
                method: 'POST',
                headers: {
                    'Content-Type': 'application/json',
                },
                body: JSON.stringify({ userId, jobId }),
            });
            if (response.ok) {
                setJobs((prevJobs) => {
                    return prevJobs.map((job) => {
                        if (job._id === jobId) {
                            return { ...job, isBookmarked:
!job.isBookmarked };
```

```
                        }
                        return job;
                    });
                });
                setBookmarkedJobs((prevBookmarkedJobs) => {
                    if (prevBookmarkedJobs.includes(jobId)) {
                        return prevBookmarkedJobs.filter(id => id !==
jobId);
                    } else {
                        return [...prevBookmarkedJobs, jobId];
                    }
                });
            } else {
                console.error('Failed to toggle bookmark');
            }
        } catch (error) {
            console.error('Error toggling bookmark:', error);
        }
    };
    return (
        <div className="min-h-screen bg-gradient-to-r from-cyan-200
to-blue-400  text-slate-900 dark:bg-gradient-to-r dark:from-slate-800
dark:to-slate-600 dark:text-slate-200">
            <div className="fixed top-0 left-0 right-0 bg-blue-300
dark:bg-slate-800 p-6 z-10 flex justify-between items-center shadow-md
border-2 border-blue-400 dark:border-slate-500">
                <h2 className="text-3xl font-bold hover:bg-blue-400
dark:hover:bg-slate-900 dark:text-gray-200">All Available Jobs</h2>
                <button
                    className='text-2xl hover:text-black
dark:hover:text-gray-400'
                    onClick={() => navigate('/jobseeker_dashboard')}
                >
                    ×
                </button>
            </div>
            <div className="pt-24 p-6 border-4 border-blue-600
dark:border-slate-500">
                <div className="grid gap-5 md:grid-cols-2 lg:grid-cols-2">
                    {jobs.map((job, index) => (
```

```
                        <div
                            key={index}
                            className={`relative rounded-md border-2
border-pink-300 dark:border-slate-200 shadow-xl cursor-pointer
transition-all duration-300 ease-in-out overflow-hidden mt-2 ${
                                expandedJob === index
                                    ? 'bg-gradient-to-r from-pink-300
to-purple-100 hover:from-pink-400 hover:to-orange-200 p-6
dark:bg-gradient-to-r dark:from-gray-400 dark:to-slate-600 dark:text-black
h-auto'
                                    : 'bg-gradient-to-r from-pink-300
to-purple-50 hover:from-pink-400 hover:to-orange-200 p-3
dark:bg-gradient-to-r dark:from-gray-600 dark:to-slate-500
dark:hover:from-gray-700 dark:hover:to-slate-700 dark:text-gray-300 h-36'
                            }`}
                            onClick={() => handleToggle(index)}
                        >
                            <div className={`flex justify-between
items-start ${expandedJob === index ? 'mb-4' : 'mb-2'}`}>
                                <div className="flex-1">
                                    <h3 className="text-xl font-semibold
text-black bg-pink-100 dark:bg-slate-800 dark:text-slate-200 mb-2 border-2
border-gray-500 dark:border-gray-400 p-1 rounded-md">{job.jobTitle}</h3>
                                    <p><strong>Company:</strong>
{job.companyName}</p>
                                    <p><strong>Location:</strong>
{job.location}</p>
                                    <p><strong>Salary Range:</strong>
BDT:{job.minSalary} - {job.maxSalary} {job.negotiable ? "(Negotiable)" :
""}</p>
                                </div>
                                <button
                                    className="absolute top-0 right-1 p-6"
                                    onClick={(e) => {
                                        e.stopPropagation();
                                        handleBookmarkToggle(job._id);
                                    }}
                                >
                                    {bookmarkedJobs.includes(job._id) ? (
```

```
                                        <BookmarkIcon
className="text-slate-800" fontSize="large" />
                                    ) : (
                                        <BookmarkBorderIcon
className="text-slate-600 dark:text-slate-200" fontSize="large" />
                                    )}
                                </button>
                                {expandedJob !== index && (
                                    <button
                                        className={`dark:bg-blue-900
bg-blue-500 hover:bg-blue-700 dark:hover:bg-blue-700 text-white px-4 py-2
rounded-md  transition duration-200 mt-14 ${userAppliedJobs[job._id] ?
'opacity-50 cursor-not-allowed' : ''}`}
                                        onClick={(e) => {
                                            e.stopPropagation();
                                            handleApply(job);
                                        }}

disabled={userAppliedJobs[job._id]}
                                    >
                                        {userAppliedJobs[job._id] ?
'Applied' : 'Apply'}
                                    </button>
                                )}
                            </div>
                            {expandedJob === index && (
                                <div className="mt-4">
                                    <p><strong>Work Mode:</strong>
{job.workMode}</p>
                                    <p><strong>Job Type:</strong>
{job.jobType}</p>
                                    <p><strong>Category:</strong>
{job.jobCategory}</p>
                                    <p><strong>Experience Level:</strong>
{job.experienceLevel}</p>
                                    <p><strong>Responsibilities:</strong>
{job.responsibilities}</p>
                                    <p><strong>Requirements:</strong>
{job.requirements}</p>
```

```
                                    <p><strong>Preferred
Qualifications:</strong> {job.preferredQualifications}</p>
                                    <p><strong>Benefits:</strong>
{job.benefits}</p>
                                    <p><strong>Posted:</strong>
{formatDate1(job.updatedAt)}</p>
                                    <p><strong>Deadline:</strong>
{formatDate(job.deadline)}</p>
                                    <p><strong>Vacancy:</strong>
{job.vacancies}</p>
                                    <p><strong>Total
Applications:</strong> {applicationCounts[job._id] || 0}</p>
                                    <p><strong>Accepted
Applications:</strong> {acceptedCounts[job._id] || 0}</p>
                                    <p><strong>Rejected
Applications:</strong> {rejectedCounts[job._id] || 0}</p>
                                    <button
                                        className={`dark:bg-blue-900
bg-blue-500 hover:bg-blue-700 dark:hover:bg-blue-700 text-white px-4 py-2
rounded-md  transition duration-200 mt-4 block mx-auto
${userAppliedJobs[job._id] ? 'opacity-50 cursor-not-allowed' : ''}`}
                                        onClick={(e) => {
                                            e.stopPropagation();
                                            handleApply(job);
                                        }}

disabled={userAppliedJobs[job._id]}
                                    >
                                        {userAppliedJobs[job._id] ?
'Applied' : 'Apply'}
                                    </button>
                                </div>
                            )}
                        </div>
                    ))}
                </div>
            </div>
        </div>
    );
}
```

```
export default All_Jobs;
```

Edit jobs:

```
import React, { useEffect, useState } from 'react';
import { Link, useParams, useNavigate } from 'react-router-dom';
import toast from 'react-hot-toast';
const formatDate = (dateString) => {
    if (!dateString) {
        return "N/A";
    }
    const date = new Date(dateString);
    return date.toISOString().split('T')[0];
};
function EditJob() {
    const { jobId } = useParams();
    const navigate = useNavigate();
    const [job, setJob] = useState(null);
    const [formData, setFormData] = useState({
        jobTitle: '',
        companyName: '',
        location: '',
        workMode: '',
        jobType: '',
        jobCategory: '',
        experienceLevel: '',
        minSalary: '',
        maxSalary: '',
        negotiable: false,
        responsibilities: '',
        requirements: '',
        preferredQualifications: '',
        benefits: '',
        deadline: '',
        vacancies: ''
    });
    useEffect(() => {
        const fetchJob = async () => {
            try {
```

```
                const response = await
fetch(`http://localhost:4001/jobs/${jobId}`);
                if (response.ok) {
                    const data = await response.json();
                    setJob(data);
                    setFormData({
                        jobTitle: data.jobTitle || '',
                        companyName: data.companyName || '',
                        location: data.location || '',
                        workMode: data.workMode || '',
                        jobType: data.jobType || '',
                        jobCategory: data.jobCategory || '',
                        experienceLevel: data.experienceLevel || '',
                        minSalary: data.minSalary || '',
                        maxSalary: data.maxSalary || '',
                        negotiable: data.negotiable || false,
                        responsibilities: data.responsibilities || '',
                        requirements: data.requirements || '',
                        preferredQualifications:
data.preferredQualifications || '',
                        benefits: data.benefits || '',
                        deadline: data.deadline || '',
                        vacancies: data.vacancies || ''
                    });
                } else {
                    toast.error('Failed to fetch job details');
                }
            } catch (error) {
                toast.error('Error fetching job details');
                console.error('Error:', error);
            }
        };
        fetchJob();
    }, [jobId]);

    const handleChange = (e) => {
        const { name, value, type, checked } = e.target;
        setFormData({
            ...formData,
            [name]: type === 'checkbox' ? checked : value,
```

```
        });
    };
    const handleSubmit = async (e) => {
        e.preventDefault();
        try {
            const response = await
fetch(`http://localhost:4001/jobs/${jobId}`, {
                method: 'PUT',
                headers: {
                    'Content-Type': 'application/json',
                },
                body: JSON.stringify(formData),
            });
            if (response.ok) {
                toast.success('Job updated successfully');
                navigate('/created_jobs');
            } else {
                const errorData = await response.json();
                toast.error(errorData.message || 'Failed to update job');
            }
        } catch (error) {
            toast.error('Failed to update job');
            console.error('Error:', error);
        }
    };
    if (!job) {
        return <div className="p-6 bg-gray-100 dark:bg-slate-800
min-h-screen">Loading...</div>;
    }
    return (
        <div className="p-6 bg-gray-100 dark:bg-slate-900 min-h-screen">
            <p><Link to="/company_dashboard" className="btn btn-sm
btn-circle btn-ghost absolute right-2 top-2 hover:bg-gray-200
dark:hover:bg-slate-700">X</Link></p>
            <h2 className="text-2xl font-bold mb-6 text-center
dark:text-white">Edit Job</h2>
            <form onSubmit={handleSubmit} className="bg-white
dark:bg-slate-800 p-6 rounded-lg shadow-md border border-gray-200
dark:border-slate-700">
```

```
                <h3 className="text-xl font-bold mb-4 border-b-2
border-gray-300 pb-2 dark:text-white">Basic Information</h3>
                <div className="grid gap-6 md:grid-cols-2 lg:grid-cols-4">
                    <div>
                        <label className="block mb-2 text-sm font-medium
dark:text-white">Job Title</label>
                        <input
                            type="text"
                            name="jobTitle"
                            value={formData.jobTitle}
                            onChange={handleChange}
                            className="w-full p-2 border border-gray-300
rounded dark:bg-slate-700 dark:text-white"
                        />
                    </div>
                    <div>
                        <label className="block mb-2 text-sm font-medium
dark:text-white">Company Name</label>
                        <input
                            type="text"
                            name="companyName"
                            value={formData.companyName}
                            onChange={handleChange}
                            className="w-full p-2 border border-gray-300
rounded dark:bg-slate-700 dark:text-white"
                        />
                    </div>
                    <div>
                        <label className="block mb-2 text-sm font-medium
dark:text-white">Location</label>
                        <input
                            type="text"
                            name="location"
                            value={formData.location}
                            onChange={handleChange}
                            className="w-full p-2 border border-gray-300
rounded dark:bg-slate-700 dark:text-white"
                        />
                    </div>
                    <div>
```

```
                        <label className="block mb-2 text-sm font-medium
dark:text-white">Work Mode</label>
                        <select
                            name="workMode"
                            value={formData.workMode}
                            onChange={handleChange}
                            className="w-full p-2 border border-gray-300
rounded dark:bg-slate-700 dark:text-white"
                        >
                            <option value="">Select Work Mode</option>
                            <option value="on-site">On-site</option>
                            <option value="remote">Remote</option>
                            <option value="hybrid">Hybrid</option>
                        </select>
                    </div>
                </div>
                <h3 className="text-xl font-bold mt-6 mb-4 border-b-2
border-gray-300 pb-2 dark:text-white">Job Details</h3>
                <div className="grid gap-6 md:grid-cols-2 lg:grid-cols-3">
                    <div>
                        <label className="block mb-2 text-sm font-medium
dark:text-white">Job Type</label>
                        <select
                            name="jobType"
                            value={formData.jobType}
                            onChange={handleChange}
                            className="w-full p-2 border border-gray-300
rounded dark:bg-slate-700 dark:text-white"
                        >
                            <option value="">Select Job Type</option>
                            <option value="full-time">Full-time</option>
                            <option value="part-time">Part-time</option>
                            <option value="contract">Contract</option>
                            <option value="internship">Internship</option>
                        </select>
                    </div>
                    <div>
                        <label className="block mb-2 text-sm font-medium
dark:text-white">Job Category</label>
                        <select
```

```
                            name="jobCategory"
                            value={formData.jobCategory}
                            onChange={handleChange}
                            className="w-full p-2 border border-gray-300
rounded dark:bg-slate-700 dark:text-white"
                        >
                            <option value="">Select Job Category</option>
                            <option value="it">IT</option>
                            <option value="marketing">Marketing</option>
                            <option value="finance">Finance</option>
                            <option value="hr">HR</option>
                            <option value="sales">Sales</option>
                        </select>
                    </div>
                    <div>
                        <label className="block mb-2 text-sm font-medium
dark:text-white">Experience Level</label>
                        <select
                            name="experienceLevel"
                            value={formData.experienceLevel}
                            onChange={handleChange}
                            className="w-full p-2 border border-gray-300
rounded dark:bg-slate-700 dark:text-white"
                        >
                            <option value="">Select Experience
Level</option>
                            <option
value="entry-level">Entry-level</option>
                            <option value="mid-level">Mid-level</option>
                            <option
value="senior-level">Senior-level</option>
                            <option value="manager">Manager</option>
                            <option value="director">Director</option>
                        </select>
                    </div>
                    <div>
                        <label className="block mb-2 text-sm font-medium
dark:text-white">Min Salary</label>
                        <input
                            type="number"
```

```
                            name="minSalary"
                            value={formData.minSalary}
                            onChange={handleChange}
                            className="w-full p-2 border border-gray-300
rounded dark:bg-slate-700 dark:text-white"
                        />
                    </div>
                    <div>
                        <label className="block mb-2 text-sm font-medium
dark:text-white">Max Salary</label>
                        <input
                            type="number"
                            name="maxSalary"
                            value={formData.maxSalary}
                            onChange={handleChange}
                            className="w-full p-2 border border-gray-300
rounded dark:bg-slate-700 dark:text-white"
                        />
                    </div>
                    <div>
                        <label className="block mb-2 text-sm font-medium
dark:text-white">Negotiable(Salary)</label>
                        <input
                            type="checkbox"
                            name="negotiable"
                            checked={formData.negotiable}
                            onChange={handleChange}
                            className="mr-2 leading-tight"
                        />
                    </div>
                </div>
                <h3 className="text-xl font-bold mt-6 mb-4 border-b-2
border-gray-300 pb-2 dark:text-white">Job Description</h3>
                <div className="grid gap-4 md:grid-cols-2 lg:grid-cols-2">
                    <div>
                        <label className="block mb-2 text-sm font-medium
dark:text-white">Responsibilities</label>
                        <textarea
                            name="responsibilities"
                            value={formData.responsibilities}
```

```
                            onChange={handleChange}
                            className="w-full p-2 border border-gray-300
rounded dark:bg-slate-700 dark:text-white"
                        />
                    </div>
                    <div>
                        <label className="block mb-2 text-sm font-medium
dark:text-white">Requirements</label>
                        <textarea
                            name="requirements"
                            value={formData.requirements}
                            onChange={handleChange}
                            className="w-full p-2 border border-gray-300
rounded dark:bg-slate-700 dark:text-white"
                        />
                    </div>
                    <div>
                        <label className="block mb-2 text-sm font-medium
dark:text-white">Preferred Qualifications</label>
                        <textarea
                            name="preferredQualifications"
                            value={formData.preferredQualifications}
                            onChange={handleChange}
                            className="w-full p-2 border border-gray-300
rounded dark:bg-slate-700 dark:text-white"
                        />
                    </div>
                    <div>
                        <label className="block mb-2 text-sm font-medium
dark:text-white">Benefits</label>
                        <textarea
                            name="benefits"
                            value={formData.benefits}
                            onChange={handleChange}
                            className="w-full p-2 border border-gray-300
rounded dark:bg-slate-700 dark:text-white"
                        />
                    </div>
                </div>
```

```
                <h3 className="text-xl font-bold mt-6 mb-4 border-b-2
border-gray-300 pb-2 dark:text-white">Additional Details</h3>
                <div className="grid gap-6 md:grid-cols-2 lg:grid-cols-4">
                    <div>
                        <label className="block mb-2 text-sm font-medium
dark:text-white">Deadline</label>
                        <input
                            type="date"
                            name="deadline"
                            value={formatDate(formData.deadline)}
                            onChange={handleChange}
                            className="w-full p-2 border border-gray-300
rounded dark:bg-slate-700 dark:text-white"
                        />
                    </div>
                    <div>
                        <label className="block mb-2 text-sm font-medium
dark:text-white">Vacancies</label>
                        <input
                            type="number"
                            name="vacancies"
                            value={formData.vacancies}
                            onChange={handleChange}
                            className="w-full p-2 border border-gray-300
rounded dark:bg-slate-700 dark:text-white"
                        />
                    </div>
                </div>
                <button
                    type="submit"
                    className="mt-6 w-full py-2 bg-blue-500 text-white
rounded hover:bg-blue-600 dark:bg-blue-700 dark:hover:bg-blue-800"
                >
                    Update Job
                </button>
            </form>
        </div>
    );
}
export default EditJob;
```

Delete Jobs:

```
import React, { useEffect, useState } from 'react';
import { Link } from 'react-router-dom';
import { IconButton,Tooltip } from '@mui/material';
import ArrowBackIcon from '@mui/icons-material/ArrowBack';
import { useNavigate } from 'react-router-dom';
import EditIcon from '@mui/icons-material/Edit';
import RestoreIcon from '@mui/icons-material/Restore';
import RestoreFromTrashIcon from '@mui/icons-material/RestoreFromTrash';
import DeleteIcon from '@mui/icons-material/Delete';
const Modal = ({ show, onClose, onConfirm, message }) => {
  if (!show) return null;

  return (
    <div className="fixed inset-0 bg-gray-800 bg-opacity-50 flex
items-center justify-center z-50">
      <div className="bg-white dark:bg-slate-700 p-6 rounded shadow-lg">
        <h3 className="text-lg font-semibold mb-4">{message}</h3>
        <div className="flex justify-end space-x-4">
          <button
            onClick={onConfirm}
            className="bg-blue-500 text-white px-4 py-2 rounded
hover:bg-blue-600"
          >
            OK
          </button>
          <button
            onClick={onClose}
            className="bg-gray-500 text-white px-4 py-2 rounded
hover:bg-gray-600"
          >
            Cancel
          </button>
        </div>
      </div>
    </div>
  );
};
function DeletedJobs() {
  const [jobs, setJobs] = useState([]);
```

```
  const [showConfirmModal, setShowConfirmModal] = useState(false);
  const [showRestoreModal, setShowRestoreModal] = useState(false);
  const [selectedJob, setSelectedJob] = useState(null);
  const navigate = useNavigate();
  useEffect(() => {
    const fetchDeletedJobs = async () => {
      const user = JSON.parse(localStorage.getItem('Users'));
      if (user && user.user1) {
        const companyId = user.user1._id;
        try {
          const response = await
fetch(`http://localhost:4001/jobs/deleted/${companyId}`);
          if (response.ok) {
            const data = await response.json();
            setJobs(data);
          } else {
            console.error('Failed to fetch deleted jobs');
          }
        } catch (error) {
          console.error('Error:', error);
        }
      }
    };
    fetchDeletedJobs();
  }, []);
  const handleDeleteApplicationsForJob = async (jobId) => {
    try {
      const response = await
fetch(`http://localhost:4001/applications/deleteApplicationsForJob/${jobId
}`, {
        method: 'DELETE',
      });
      if (response.ok) {
        console.log('All applications for the job have been marked as
Deleted');
      } else {
        const errorText = await response.text();
        alert(`Error: ${errorText}`);
      }
    } catch (error) {
```

```
      console.error('Error:', error);
      alert('An error occurred while deleting applications. Please try
again.');
    }
  };
  const handleAdvancedRestoreJob = async (jobId) => {
    try {
      const response = await
fetch(`http://localhost:4001/jobs/advancedRestore/${jobId}`, {
        method: 'PATCH',
      });
      if (response.ok) {
        setJobs(jobs.filter(job => job._id !== jobId));
      } else {
        const errorText = await response.text();
        alert(`Error: ${errorText}`);
      }
    } catch (error) {
      console.error('Error:', error);
      alert('An error occurred while restoring the job. Please try
again.');
    }
  };
  const handleOpenConfirmModal = (jobId) => {
    setSelectedJob(jobId);
    setShowConfirmModal(true);
  };
  const handleCloseConfirmModal = () => {
    setShowConfirmModal(false);
    setSelectedJob(null);
  };
  const handleConfirm = async () => {
    if (selectedJob) {
      await handleDeleteApplicationsForJob(selectedJob);
      setShowConfirmModal(false);
      setShowRestoreModal(true);
    }
  };
  const handleRestoreConfirm = async () => {
    if (selectedJob) {
```

```
      await handleAdvancedRestoreJob(selectedJob);
      setShowRestoreModal(false);
    }
  };
  const handleCloseRestoreModal = () => {
    setShowRestoreModal(false);
    setSelectedJob(null);
  };
  const handleRestoreJob = async (jobId) => {
    try {
      const response = await
fetch(`http://localhost:4001/jobs/restore/${jobId}`, {
        method: 'PATCH',
      });
      if (response.ok) {
        setJobs(jobs.filter(job => job._id !== jobId));
      } else {
        const errorText = await response.text();
        alert(`Error: ${errorText}`);
      }
    } catch (error) {
      console.error('Error:', error);
      alert('An error occurred while restoring the job. Please try
again.');
    }
  };
  const handleDeleteClick = async (jobId) => {
    try {
      const response = await fetch(`http://localhost:4001/jobs/${jobId}`,
{
        method: 'DELETE',
      });
      if (response.ok) {
        setJobs(jobs.filter((job) => job._id !== jobId));
      } else {
        console.error('Failed to delete job');
      }
    } catch (error) {
      console.error('Error:', error);
    }
```

```
  };
  const formatDate = (dateString) => {
    if (!dateString) {
      return "N/A";
    }
    const date = new Date(dateString);
    const today = new Date();
    const tomorrow = new Date(today);
    today.setHours(0, 0, 0, 0);
    tomorrow.setDate(today.getDate() + 1);
    tomorrow.setHours(0, 0, 0, 0);
    const formattedDate = date.toISOString().split('T')[0];
    if (formattedDate === today.toISOString().split('T')[0]) {
      return `Today is the deadline (${formattedDate})`;
    }

    if (formattedDate === tomorrow.toISOString().split('T')[0]) {
      return `Tomorrow is the deadline (${formattedDate})`;
    }
    return formattedDate;
  };
  const formatDate1 = (dateString) => {
    if (!dateString) {
      return "N/A";
    }
    const date = new Date(dateString);
    return date.toISOString().split('T')[0];
  };
  return (
    <div className="min-h-screen bg-blue-200 dark:text-slate-200
dark:bg-slate-800">
      <div className="fixed top-0 left-0 right-0 bg-blue-200
dark:bg-slate-800 p-6 z-10 flex items-center shadow-md">
        <Link to="/company_dashboard">
          <IconButton size="small" className="hover:bg-gray-100
dark:hover:bg-slate-600 dark:bg-slate-500">
            <ArrowBackIcon />
          </IconButton>
        </Link>
```

```
        <h2 className="text-2xl font-bold absolute left-1/2 transform
-translate-x-1/2">
          Deleted Jobs
        </h2>
      </div>
      <div className="mt-20 p-4 grid grid-cols-1 gap-4 sm:grid-cols-2
lg:grid-cols-3">
        {jobs.length === 0 ? (
          <div className="text-center">No deleted jobs found.</div>
        ) : (
          jobs.map((job) => (
            <div
  key={job._id}
  className="bg-white dark:bg-slate-700 p-4 rounded-lg shadow-lg
hover:shadow-xl transition-shadow duration-300 relative border
border-gray-300 dark:border-slate-500"
>
  <div className="border-b border-gray-400 dark:border-slate-600 pb-2
mb-4">
    <h3 className="text-xl font-semibold border-b-2 border-blue-400
dark:border-blue-500 inline-block pb-1">
      Job Title: {job.jobTitle}
    </h3>
  </div>
  <div className="space-y-2 border p-3 rounded-lg border-gray-300
dark:border-slate-500">
    <p><strong>Company:</strong> {job.companyName}</p>
    <p><strong>Location:</strong> {job.location}</p>
    <p><strong>Work Mode:</strong> {job.workMode}</p>
    <p><strong>Job Type:</strong> {job.jobType}</p>
    <p><strong>Category:</strong> {job.jobCategory}</p>
    <p><strong>Experience Level:</strong> {job.experienceLevel}</p>
    <p>
      <strong>Salary Range:</strong> BDT:{job.minSalary} -
{job.maxSalary}{' '}
      {job.negotiable ? '(Negotiable)' : ''}
    </p>
    <p><strong>Responsibilities:</strong> {job.responsibilities}</p>
    <p><strong>Requirements:</strong> {job.requirements}</p>
```

```
    <p><strong>Preferred Qualifications:</strong>
{job.preferredQualifications}</p>
    <p><strong>Benefits:</strong> {job.benefits}</p>
    <p><strong>Posted:</strong> {formatDate1(job.createdAt)}</p>
    <p><strong>Deadline:</strong> {formatDate(job.deadline)}</p>
    <p><strong>Vacancy:</strong> {job.vacancies}</p>
  </div>
  <div className="flex justify-between mt-4 border-t pt-4 border-gray-400
dark:border-slate-600">
    <Tooltip title="Restore the job.Post the job with all the details and
applications.">
      <IconButton onClick={() => handleRestoreJob(job._id)}
className="text-green-500 hover:text-green-600">
        <RestoreIcon />
      </IconButton>
    </Tooltip>
    <div className="flex items-center">
      <Tooltip title="Repost this job. Note that all the previous
applications will be deleted and this job will be posted as new.">
        <IconButton onClick={() => handleOpenConfirmModal(job._id)}
className="text-blue-500 hover:text-blue-600">
          <RestoreFromTrashIcon />
        </IconButton>
      </Tooltip>
      <Link to={`/edit-job-del/${job._id}`}>
        <Tooltip title="Edit: Edit your job before post">
          <IconButton className="text-blue-500 hover:text-blue-600">
            <EditIcon />
          </IconButton>
        </Tooltip>
      </Link>
    </div>
    <Tooltip title="Delete this job permanently.">
      <IconButton onClick={() => handleDeleteClick(job._id)}
className="text-red-500 hover:text-red-600">
        <DeleteIcon />
      </IconButton>
    </Tooltip>
  </div>
</div>
```

```
          ))
        )}
      </div>
      <Modal
        show={showConfirmModal}
        onClose={handleCloseConfirmModal}
        onConfirm={handleConfirm}
        message="Are you sure you want to restore this job and mark all
applications as deleted?"
      />
      <Modal
        show={showRestoreModal}
        onClose={handleCloseRestoreModal}
        onConfirm={handleRestoreConfirm}
        message="Do you want to confirm the job restoration?"
      />
    </div>
  );
}
export default DeletedJobs;
```

Contribution of ID : 22101117, Name : Shawana Maliha

ApplicantsTable.jsx

```
import React, { useEffect, useState } from 'react';
import axios from 'axios';




const APPLICANTS_API_END_POINT =
'http://localhost:4001/applications/${userId}';
const shortlistingStatus = ["Accepted", "Rejected"];




const ApplicantsTable = () => {
    const [applicants, setApplicants] = useState([]);
```

```
    useEffect(() => {
        const fetchApplicants = async () => {
            try {
                const response = await
axios.get(APPLICANTS_API_END_POINT);
                setApplicants(response.data.applications);
            } catch (error) {
                console.error('Error fetching applicants:', error);
                alert('Failed to load applicants');
            }
        };




        fetchApplicants();
    }, []);




    const statusHandler = async (status, id) => {
        try {
            axios.defaults.withCredentials = true;
            const res = await
axios.post(`${APPLICANTS_API_END_POINT}/status/${id}/update`, { status });
            if (res.data.success) {
                alert(res.data.message);
                setApplicants(prev =>
                    prev.map(app => app._id === id ? { ...app, status } :
app)
                );
            }
        } catch (error) {
            alert('Error updating status: ' +
error.response.data.message);
        }
```

```
    };




    return (
        <div>
            <table className="min-w-full table-auto">
                <caption>A list of your recent applicants</caption>
                <thead>
                    <tr>
                        <th>Full Name</th>
                        <th>Email</th>
                        <th>Phone Number</th>
                        <th>Resume</th>
                        <th>Date</th>
                        <th>Action</th>
                    </tr>
                </thead>
                <tbody>
                    {
                        applicants && applicants.map((item) => (
                            <tr key={item._id}>
                                <td>{item?.fullname}</td>
                                <td>{item?.email}</td>
                                <td>{item?.phoneNumber}</td>
                                <td>
                                    {
                                        item?.resume ? <a
classname="text-blue-600" href={item?.resume} target="_blank"
rel="noopener noreferrer">View Resume</a> : 'NA'
                                    }
                                </td>
                                <td>{new
Date(item?.createdAt).toLocaleDateString()}</td>
                                <td>
                                    <select onChange={(e) =>
statusHandler(e.target.value, item._id)} value={item.status || "Select
Status"}>
```

```
                                        <option disabled>Select
Status</option>
                                        {shortlistingStatus.map((status,
index) => (
                                            <option key={index}
value={status}>{status}</option>
                                        ))}
                                    </select>
                                </td>
                            </tr>
                        ))
                    }
                </tbody>
            </table>
        </div>
    );
};




export default ApplicantsTable;
```

Admindashboard.jsx

```
import React, { useState, useEffect } from 'react';
import axios from 'axios';
import { useNavigate } from 'react-router-dom';
import { Modal, Box, Button, IconButton, TextField } from '@mui/material';
import EditIcon from '@mui/icons-material/Edit';
import CloseIcon from '@mui/icons-material/Close';
import LogoutIcon from '@mui/icons-material/Logout';

function Admin_Dashboard() {
  const [jobseekers, setJobseekers] = useState([]);
  const [companies, setCompanies] = useState([]);
  const [view, setView] = useState('user');
  const [open, setOpen] = useState(false);
  const [selectedItem, setSelectedItem] = useState(null);
  const [updatedName, setUpdatedName] = useState('');
```

```
  const [updatedEmail, setUpdatedEmail] = useState('');
  const [updatedAddress, setUpdatedAddress] = useState('');
  const navigate = useNavigate();

  useEffect(() => {
    // Fetch jobseekers data
    axios.get('http://localhost:4001/user/jobseekers')
      .then(response => setJobseekers(response.data))
      .catch(error => console.error('Error fetching jobseekers:', error));

    // Fetch companies data from /companies1
    axios.get('http://localhost:4001/user1/companies1')
      .then(response => setCompanies(response.data))
      .catch(error => console.error('Error fetching companies:', error));
  }, []);

  const handleOpenModal = (item) => {
    setSelectedItem(item);
    if (view === 'user') {
      setUpdatedName(item.fullname || '');
      setUpdatedEmail(item.email || '');
      setUpdatedAddress(''); // Clear address field for jobseekers
    } else {
      setUpdatedName(item.company_name || '');
      setUpdatedEmail(item.company_email || '');
      setUpdatedAddress(item.company_address || ''); // Set address for
companies
    }
    setOpen(true);
  };

  const handleCloseModal = () => {
    setOpen(false);
    setSelectedItem(null);
  };

  const handleEdit = (event) => {
    event.preventDefault();
    const updatedItem = view === 'user'
      ? { ...selectedItem, fullname: updatedName, email: updatedEmail }
```

```
      : { ...selectedItem, company_name: updatedName, company_email:
updatedEmail, company_address: updatedAddress };

    const url = view === 'user'
      ? `http://localhost:4001/user/jobseekers/${selectedItem._id}`
      : `http://localhost:4001/user1/companies1/${selectedItem._id}`;

    axios.put(url, updatedItem)
      .then(() => {
        if (view === 'user') {
          setJobseekers(jobseekers.map(item => item._id ===
updatedItem._id ? updatedItem : item));
        } else {
          setCompanies(companies.map(item => item._id === updatedItem._id
? updatedItem : item));
        }
        handleCloseModal();
      })
      .catch(error => console.error('Error updating item:', error));
  };

  const handleDelete = (id, type) => {
    const url = type === 'jobseeker'
      ? `http://localhost:4001/user/jobseekers/${id}`
      : `http://localhost:4001/user1/companies1/${id}`;

    axios.delete(url)
      .then(() => {
        if (type === 'jobseeker') {
          setJobseekers(jobseekers.filter(jobseeker => jobseeker._id !==
id));
        } else {
          setCompanies(companies.filter(company => company._id !== id));
        }
      })
      .catch(error => console.error('Error deleting item:', error));
  };

  const handleAddAdmin = () => {
    navigate('/signup_admin');
```

```
  };

  const handleLogout = () => {
    navigate('/');
  };

  return (
    <div
      className="flex h-screen"
      style={{
        backgroundImage:
'url(https://img.freepik.com/free-photo/blue-paint-with-shade_53876-94986.
jpg?ga=GA1.1.1599778208.1716446894&semt=ais_hybrid)',
        backgroundSize: 'cover',
        backgroundPosition: 'center',
        color: 'white',
      }}
    >
      {/* Sidebar */}
      <div className="bg-gray-800 text-gray-100 w-64 flex flex-col
justify-between">
        <div className="p-4">
          <h1 className="font-bold text-2xl text-white">Admin Panel</h1>
          <ul className="mt-6">
            <li
              className={`cursor-pointer ${view === 'user' ? 'text-white'
: 'text-gray-300'}`}
              onClick={() => setView('user')}
            >
              User Information
            </li>
            <li
              className={`cursor-pointer ${view === 'company' ?
'text-white' : 'text-gray-300'}`}
              onClick={() => setView('company')}
            >
              Company Information
            </li>
          </ul>
        </div>
```

```
        {/* Add Admin Button */}
        <div className="p-4">
          <button
            className="bg-blue-500 text-white px-4 py-2 rounded-md
hover:bg-blue-700 transition duration-300 w-full"
            onClick={handleAddAdmin}
          >
            Add Admin
          </button>
        </div>
      </div>

      {/* Main content */}
      <div className="flex-1 flex flex-col">
        {/* Navbar */}
        <div className="bg-gray-800 text-white p-4 flex justify-between
items-center border-b-2">
          <h1 className="font-bold text-2xl">Admin Dashboard</h1>
          <IconButton color="inherit" onClick={handleLogout}>
            <LogoutIcon style={{ color: 'white' }} />
          </IconButton>
        </div>

        {/* Table */}
        <div className="overflow-x-auto p-4">
          {view === 'user' && (
            <table className="table w-full">
              <thead>
                <tr className="text-white">
                  <th>Name</th>
                  <th>Email</th>
                  <th>Actions</th>
                </tr>
              </thead>
              <tbody>
                {jobseekers.map(jobseeker => (
                  <tr key={jobseeker._id}>
                    <td>{jobseeker.fullname}</td>
                    <td>{jobseeker.email}</td>
```

```
                    <td>
                      <IconButton
                        color="primary"
                        onClick={() => handleOpenModal(jobseeker)}
                      >
                        <EditIcon style={{ color: 'white' }} />
                      </IconButton>
                      <IconButton
                        color="secondary"
                        onClick={() => handleDelete(jobseeker._id,
'jobseeker')}
                      >
                        <CloseIcon style={{ color: 'white' }} />
                      </IconButton>
                    </td>
                  </tr>
                ))}
              </tbody>
            </table>
          )}
          {view === 'company' && (
            <table className="table w-full">
              <thead>
                <tr className="text-white">
                  <th>Company Name</th>
                  <th>Email</th>
                  <th>Address</th>
                  <th>Actions</th>
                </tr>
              </thead>
              <tbody>
                {companies.map(company => (
                  <tr key={company._id}>
                    <td>{company.company_name}</td>
                    <td>{company.company_email}</td>
                    <td>{company.company_address}</td>
                    <td>
                      <IconButton
                        color="primary"
                        onClick={() => handleOpenModal(company)}
```

```
                    >
                      <EditIcon style={{ color: 'white' }} />
                    </IconButton>
                    <IconButton
                      color="secondary"
                      onClick={() => handleDelete(company._id,
'company')}
                    >
                      <CloseIcon style={{ color: 'white' }} />
                    </IconButton>
                  </td>
                </tr>
              ))}
            </tbody>
          </table>
        )}
      </div>
    </div>

    {/* Modal for Edit */}
    <Modal open={open} onClose={handleCloseModal}>
      <Box className="modal-content" sx={{ backgroundColor: 'white',
padding: '20px', margin: 'auto', width: '300px' }}>
        <h2>Edit Information</h2>
        <form onSubmit={handleEdit}>
          <TextField
            label="Name"
            fullWidth
            value={updatedName}
            onChange={(e) => setUpdatedName(e.target.value)}
            margin="normal"
          />
          <TextField
            label="Email"
            fullWidth
            value={updatedEmail}
            onChange={(e) => setUpdatedEmail(e.target.value)}
            margin="normal"
          />
          {view === 'company' && (
```

```
              <TextField
                label="Address"
                fullWidth
                value={updatedAddress}
                onChange={(e) => setUpdatedAddress(e.target.value)}
                margin="normal"
              />
            )}
            <Button type="submit" variant="contained" color="primary"
fullWidth>
              Save Changes
            </Button>
          </form>
        </Box>
      </Modal>
    </div>
  );
}

export default Admin_Dashboard;
```

Company_Dashboard.jsx

```
import React, { useEffect, useState } from 'react';
import { useNavigate } from 'react-router-dom';
import { IconButton, Button } from '@mui/material';
import EditIcon from '@mui/icons-material/Edit';
import SaveIcon from '@mui/icons-material/Save';


function Company_Dashboard() {
  const navigate = useNavigate();


  const [companyName, setCompanyName] = useState('');
  const [companyEmail, setCompanyEmail] = useState('');
  const [companyAddress, setCompanyAddress] = useState('');


  const [jobDescription, setJobDescription] = useState({
```

```
    introduction: 'We are looking for a dedicated Marketing Manager to
lead our campaigns and strategy.',
    responsibilities: 'You will manage social media accounts, develop
marketing strategies, and collaborate with the sales team.',
    impact: 'Your efforts will directly impact our brand's growth and
engagement with customers.',
  });

  const [requiredSkills, setRequiredSkills] = useState({
    technicalSkills: 'Proficiency in JavaScript, React, and Node.js.',
    softSkills: 'Strong collaboration and communication skills.',
    experience: 'A bachelor's degree in Computer Science or 3+ years of
experience in web development.',
  });


  const [isEditingJobDesc, setIsEditingJobDesc] = useState(false);
  const [isEditingSkills, setIsEditingSkills] = useState(false);


  useEffect(() => {
    const user = JSON.parse(localStorage.getItem('Users'));
    if (user && user.user1) {
      setCompanyName(user.user1.company_name);
      setCompanyEmail(user.user1.company_email);
      setCompanyAddress(user.user1.company_address);
    }
  }, []);


  const handleLogout = () => {
    localStorage.removeItem('Users');
    navigate('/');
  };


  const toggleEditJobDesc = () => setIsEditingJobDesc(!isEditingJobDesc);
  const toggleEditSkills = () => setIsEditingSkills(!isEditingSkills);
  const saveJobDesc = () => {
    setIsEditingJobDesc(false);
```

```
  };
  const saveSkills = () => {
    setIsEditingSkills(false);
  };


  return (
    <div className="h-screen flex flex-col">
      {/* Header Section */}
      <div className="bg-gray-200 text-black dark:bg-gray-700
dark:text-white p-4 flex justify-between items-center border-b-2">
        <h1 className="font-bold text-2xl">Company Dashboard</h1>
        <h1 className="font-bold text-1xl">Welcome {companyName}</h1>
        <button
          onClick={handleLogout}
          className="bg-red-500 text-white px-4 py-2 rounded-md
hover:bg-red-700 transition duration-300"
        >
          Logout
        </button>
      </div>


      <div className="flex flex-1">
        {/* Sidebar Section */}
        <nav className="w-64 bg-gray-800 text-white p-4 flex-shrink-0">
          <ul>
            <li className="mb-2">
              <button
                onClick={() => navigate('/addjobs')}
                className="w-full text-left px-4 py-2 hover:bg-gray-700
rounded-md"
              >
                Add Jobs
              </button>
            </li>
            <li className="mb-2">
              <button
                onClick={() => navigate('/created_jobs')}
```

```
                  className="w-full text-left px-4 py-2 hover:bg-gray-700
rounded-md"
                >
                  Created Jobs
                </button>
              </li>
              <li className="mb-2">
                <button
                  onClick={() =>
navigate(`/accepted_applications/${companyId}`)}
                  className="w-full text-left px-4 py-2 hover:bg-gray-700
rounded-md"
                >
                  Accepted Applications
                </button>
              </li>
              <li className="mb-2">
                <button
                  onClick={() =>
navigate(`/rejected_applications/${companyId}`)}
                  className="w-full text-left px-4 py-2 hover:bg-gray-700
rounded-md"
                >
                  Rejected Applications
                </button>
              </li>
              <li className="mb-2">
                <button
                  onClick={() => navigate('/ApplicantsTable')}
                  className="w-full text-left px-4 py-2 hover:bg-gray-700
rounded-md"
                >
                  Applicants
                </button>
              </li>
              <li className="mb-2">
                <button
                  onClick={() => navigate('/company-bookmarked-jobs')}
                  className="w-full text-left px-4 py-2 hover:bg-gray-700
rounded-md"
```

```
              >
                Bookmarked Jobs
              </button>
            </li>
          </ul>
        </nav>



        {/* Main Section */}
        <div className="flex-1 p-8 bg-cover bg-center" style={{
backgroundImage:
`url('https://img.freepik.com/free-photo/millennial-asia-businessmen-busin
esswomen-meeting-brainstorming-ideas-about-new-paperwork-project-colleague
s-working-together-planning-success-strategy-enjoy-teamwork-small-modern-n
ight-office_7861-2386.jpg?size=626&ext=jpg&ga=GA1.1.1599778208.1716446894&
semt=ais_hybrid')` }}>
          <div className="max-w-md bg-white bg-opacity-80 p-4 rounded-md">
            <h1 className="mb-2 text-3xl font-bold text-left">Hello,
{companyName}!</h1>
            <p className="mb-5 text-left">Email: {companyEmail}<br
/>Address: {companyAddress}</p>


            {/* Job Description Section */}
            <h2 className="text-xl font-bold text-left mb-2">Job
Description</h2>
            {isEditingJobDesc ? (
              <>
                <textarea value={jobDescription.introduction}
onChange={(e) => setJobDescription({ ...jobDescription, introduction:
e.target.value })} placeholder="Introduction" className="textarea
textarea-bordered textarea-xs w-full max-w-xs mb-2" />
                <textarea value={jobDescription.responsibilities}
onChange={(e) => setJobDescription({ ...jobDescription, responsibilities:
e.target.value })} placeholder="Responsibilities" className="textarea
textarea-bordered textarea-xs w-full max-w-xs mb-2" />
                <textarea value={jobDescription.impact} onChange={(e) =>
setJobDescription({ ...jobDescription, impact: e.target.value })}
```

```
placeholder="Impact" className="textarea textarea-bordered textarea-xs
w-full max-w-xs mb-2" />
                <Button onClick={saveJobDesc} variant="contained"
color="primary" startIcon={<SaveIcon />}>Save</Button>
              </>
            ) : (
              <>
                <p><strong>Introduction:</strong>
{jobDescription.introduction}</p>
                <p><strong>Responsibilities:</strong>
{jobDescription.responsibilities}</p>
                <p><strong>Impact:</strong> {jobDescription.impact}</p>
                <IconButton onClick={toggleEditJobDesc}
aria-label="edit"><EditIcon /></IconButton>
              </>
            )}


            {/* Required Knowledge, Skills and Abilities Section */}
            <h2 className="text-xl font-bold text-left mt-4 mb-2">Required
Knowledge, Skills, and Abilities</h2>
            {isEditingSkills ? (
              <>
                <textarea value={requiredSkills.technicalSkills}
onChange={(e) => setRequiredSkills({ ...requiredSkills, technicalSkills:
e.target.value })} placeholder="Technical Skills" className="textarea
textarea-bordered textarea-xs w-full max-w-xs mb-2" />
                <textarea value={requiredSkills.softSkills} onChange={(e)
=> setRequiredSkills({ ...requiredSkills, softSkills: e.target.value })}
placeholder="Soft Skills" className="textarea textarea-bordered
textarea-xs w-full max-w-xs mb-2" />
                <textarea value={requiredSkills.experience} onChange={(e)
=> setRequiredSkills({ ...requiredSkills, experience: e.target.value })}
placeholder="Experience" className="textarea textarea-bordered textarea-xs
w-full max-w-xs mb-2" />
                <Button onClick={saveSkills} variant="contained"
color="primary" startIcon={<SaveIcon />}>Save</Button>
              </>
            ) : (
              <>
```

```
                <p><strong>Technical Skills:</strong>
{requiredSkills.technicalSkills}</p>
                <p><strong>Soft Skills:</strong>
{requiredSkills.softSkills}</p>
                <p><strong>Experience/Qualifications:</strong>
{requiredSkills.experience}</p>
                <IconButton onClick={toggleEditSkills}
aria-label="edit"><EditIcon /></IconButton>
              </>
            )}
          </div>
        </div>
      </div>
    </div>
  );
}


export default Company_Dashboard;
```

CompanyDetails.jsx

```
import React, { useEffect, useState } from 'react';
import { useLocation } from 'react-router-dom';
import axios from 'axios';




function CompanyDetails() {
  const location = useLocation();
  const queryParams = new URLSearchParams(location.search);
  const companyName = queryParams.get('company_name');
  const companyEmail = queryParams.get('company_email');
  const companyAddress = queryParams.get('company_address');
```

```
  const [company, setCompany] = useState(null);




  useEffect(() => {
    axios.get('http://localhost:4001/user1/companies', {
      params: {
        company_name: companyName,
        company_email: companyEmail,
        company_address: companyAddress
      }
    })
    .then(response => {
      setCompany(response.data);
    })
    .catch(error => {
      console.error('Error fetching company details:', error);
    });
  }, [companyName, companyEmail, companyAddress]);




  if (!company) {
    return <div>Loading...</div>;
  }




  return (
    <div>
      <h2>{company.company_name}</h2>
      <p>Email: {company.company_email}</p>
      <p>Address: {company.company_address}</p>
      {/* Add more fields as necessary */}
    </div>
  );
}
```

```
export default CompanyDetails;
```

Contribution of ID : 23241082, Name : Rumman Nodi

ResetPassword.jsx:

```
function ResetPassword() {
  const location = useLocation();
  const navigate = useNavigate();
  const [password, setPassword] = useState('');
  const [confirmPassword, setConfirmPassword] = useState('');
  const [token, setToken] = useState('');


  React.useEffect(() => {
    const query = new URLSearchParams(location.search);
    setToken(query.get('token'));
  }, [location.search]);

  const handleSubmit = async (e) => {
    e.preventDefault();

    if (password !== confirmPassword) {
      toast.error('Passwords do not match.');
      return;
    }

    try {
      const response = await axios.post('http://localhost:4001/password-reset/reset-password', {
        token,
        password
      });

      if (response.data.message === 'Password reset successful') {
        toast.success('Password reset successfully. You can now log in with your new password.');
        navigate('/login');
      } else {
        toast.error(response.data.message);
      }
    } catch (error) {
      console.error(error);
      toast.error('An error occurred. Please try again.');
    }
```

```jsx
return (
  <div className="flex justify-center items-center h-screen bg-gray-100">
    <div className="w-96 p-6 shadow-lg rounded bg-white">
      <h2 className="text-2xl font-bold mb-6 text-center">Reset Password</h2>
      <form onSubmit={handleSubmit}>
        <div className="space-y-2">
          <label htmlFor="password" className="block text-sm font-medium text-gray-700">
            New Password
          </label>
          <input
            id="password"
            type="password"
            placeholder="Enter new password"
            className="w-full px-3 py-2 border rounded-md outline-none"
            value={password}
            onChange={(e) => setPassword(e.target.value)}
            required
          />
        </div>
        <div className="space-y-2 mt-4">
          <label htmlFor="confirm-password" className="block text-sm font-medium text-gray-700">
            Confirm Password
          </label>
          <input
            id="confirm-password"
            type="password"
            placeholder="Confirm new password"
            className="w-full px-3 py-2 border rounded-md outline-none"
            value={confirmPassword}
            onChange={(e) => setConfirmPassword(e.target.value)}
            required
          />
          <button
            type="submit"
            className="w-full mt-4 bg-blue-500 text-white rounded-md px-3 py-2 hover:bg-blue-700 duration-300"
          >
            Reset Password
          </button>
        </form>
      </div>
    </div>
  );
}

export default ResetPassword;
```

ProfilePage.jsx:

```
t React, { useState, useEffect } from 'react';
t { useNavigate } from 'react-router-dom';
t { FontAwesomeIcon } from '@fortawesome/react-fontawesome';
t { faUserCircle } from '@fortawesome/free-solid-svg-icons';

ion UserProfile() {
st navigate = useNavigate();
st [profile, setProfile] = useState({
ullname: '',
nail: '',
noneNumber: '',
loodGroup: '',
eligion: '',
ateOfBirth: '',
nstitution: '',
scOlevel: '',
scAlevel: '',
ex: '',
nage: null, // Initialize as null for file upload

st [isNewUser, setIsNewUser] = useState(false);

effect(() => {
/ Fetch user profile data from the server
onst fetchData = async () => {
 try {
   const response = await fetch('http://localhost:4001/api/profiles');
   if (response.ok) {
     const userData = await response.json();
     setProfile(userData);
     setIsNewUser(!userData.fullname); // Check if user is new based on profile data
   }
 } catch (error) {
   console.error('Error fetching profile:', error);
 }
;
```

```
    fetchData();
  }, []);

  const handleInputChange = (e) => {
    const { name, value } = e.target;
    setProfile({ ...profile, [name]: value });
  };

  const handleFileChange = (e) => {
    const file = e.target.files[0];
    setProfile({ ...profile, image: file });
  };

  const handleSave = async () => {
    const formData = new FormData();
    formData.append('fullname', profile.fullname);
    formData.append('email', profile.email);
    formData.append('phoneNumber', profile.phoneNumber);
    formData.append('bloodGroup', profile.bloodGroup);
    formData.append('religion', profile.religion);
    formData.append('dateOfBirth', profile.dateOfBirth);
    formData.append('institution', profile.institution);
    formData.append('sscOlevel', profile.sscOlevel);
    formData.append('hscAlevel', profile.hscAlevel);
    formData.append('sex', profile.sex);
    if (profile.image) formData.append('image', profile.image); // Append image if present

    try {
      const response = await fetch('http://localhost:4001/api/profiles', {
        method: 'POST', // POST for saving new user
        body: formData,
      });
      const result = await response.json();
      if (response.ok) {
```

```
      if (response.ok) {
        alert('Profile saved successfully');
        setIsNewUser(false); // Mark as existing user after saving
      } else {
        alert(result.message || 'Failed to save profile');
      }
    } catch (error) {
      console.error('Error saving profile:', error);
      alert('An error occurred');
    }
  };

  const handleUpdate = async () => {
    const formData = new FormData();
    formData.append('fullname', profile.fullname);
    formData.append('email', profile.email);
    formData.append('phoneNumber', profile.phoneNumber);
    formData.append('bloodGroup', profile.bloodGroup);
    formData.append('religion', profile.religion);
    formData.append('dateOfBirth', profile.dateOfBirth);
    formData.append('institution', profile.institution);
    formData.append('sscOlevel', profile.sscOlevel);
    formData.append('hscAlevel', profile.hscAlevel);
    formData.append('sex', profile.sex);
    if (profile.image) formData.append('image', profile.image); // Append image if present

    try {
      const response = await fetch('http://localhost:4001/api/profiles', {
        method: 'PUT', // PUT for updating existing user
        body: formData,
      });
      const result = await response.json();
      if (response.ok) {
        alert('Profile updated successfully');
```

```
    } else {
      alert(result.message || 'Failed to update profile');
    }
  } catch (error) {
    console.error('Error updating profile:', error);
    alert('An error occurred');
  }
};

const handleResumeBuilder = () => {
  window.open('https://app.flowcv.com/dashboard', '_blank');
};

return (
  <div className="min-h-screen bg-gray-100 p-6 flex flex-col items-center">
    <div className="w-full max-w-4xl bg-white shadow-lg rounded-lg overflow-hidden">
      <div className="bg-blue-500 text-white p-6 flex items-center justify-between">
        <h1 className="text-3xl font-bold">Profile</h1>
        <button
          onClick={() => navigate('/')}
          className="bg-gray-700 text-white px-4 py-2 rounded-md hover:bg-gray-800 transition duration-300"
        >
          Back to Dashboard
        </button>
      </div>
      <div className="p-6">
        <div className="flex items-center justify-center">
          <div className="w-32 h-32 rounded-full border-4 border-blue-500 overflow-hidden bg-gray-200 flex items-center justify-center">
            {profile.image ? (
              <img src={URL.createObjectURL(profile.image)} alt="Profile" className="w-full h-full object-cover" />
            ) : (
              <FontAwesomeIcon icon={faUserCircle} className="text-6xl" />
            )}
          </div>
```

```
        </div>
        <form className="mt-6">
          <div className="grid grid-cols-1 md:grid-cols-2 gap-6">
            <div>
              <label htmlFor="fullname" className="block text-sm font-medium text-gray-700">Full Name</label>
              <input
                type="text"
                id="fullname"
                name="fullname"
                value={profile.fullname}
                onChange={handleInputChange}
                className="mt-1 block w-full px-3 py-2 border border-gray-300 rounded-md shadow-sm focus:outline-none focus:ring-blue-500 f
              />
            </div>
            <div>
              <label htmlFor="email" className="block text-sm font-medium text-gray-700">Email</label>
              <input
                type="email"
                id="email"
                name="email"
                value={profile.email}
                onChange={handleInputChange}
                className="mt-1 block w-full px-3 py-2 border border-gray-300 rounded-md shadow-sm focus:outline-none focus:ring-blue-500 f
              />
            </div>
            <div>
              <label htmlFor="phoneNumber" className="block text-sm font-medium text-gray-700">Phone Number</label>
              <input
                type="text"
                id="phoneNumber"
                name="phoneNumber"
                value={profile.phoneNumber}
                onChange={handleInputChange}
                className="mt-1 block w-full px-3 py-2 border border-gray-300 rounded-md shadow-sm focus:outline-none focus:ring-blue-500 f
              />
            </div>
```

```
<div>
  <label htmlFor="bloodGroup" className="block text-sm font-medium text-gray-700">Blood Group</label>
  <input
    type="text"
    id="bloodGroup"
    name="bloodGroup"
    value={profile.bloodGroup}
    onChange={handleInputChange}
    className="mt-1 block w-full px-3 py-2 border border-gray-300 rounded-md shadow-sm focus:outline-none focus:ring-blue-500 focus:border
  />
</div>
<div>
  <label htmlFor="religion" className="block text-sm font-medium text-gray-700">Religion</label>
  <input
    type="text"
    id="religion"
    name="religion"
    value={profile.religion}
    onChange={handleInputChange}
    className="mt-1 block w-full px-3 py-2 border border-gray-300 rounded-md shadow-sm focus:outline-none focus:ring-blue-500 focus:border
  />
</div>
<div>
  <label htmlFor="dateOfBirth" className="block text-sm font-medium text-gray-700">Date of Birth</label>
  <input
    type="date"
    id="dateOfBirth"
    name="dateOfBirth"
    value={profile.dateOfBirth}
    onChange={handleInputChange}
    className="mt-1 block w-full px-3 py-2 border border-gray-300 rounded-md shadow-sm focus:outline-none focus:ring-blue-500 focus:border
  />
</div>
<div>
  <label htmlFor="institution" className="block text-sm font-medium text-gray-700">Institution</label>
```

```
<div>
  <label htmlFor="institution" className="block text-sm font-medium text-gray-700">Institution</label>
  <input
    type="text"
    id="institution"
    name="institution"
    value={profile.institution}
    onChange={handleInputChange}
    className="mt-1 block w-full px-3 py-2 border border-gray-300 rounded-md shadow-sm focus:outline-none focus:ring-blue-500 focus:border-b
  />
</div>
<div>
  <label htmlFor="sscOlevel" className="block text-sm font-medium text-gray-700">SSC/O-Level</label>
  <input
    type="text"
    id="sscOlevel"
    name="sscOlevel"
    value={profile.sscOlevel}
    onChange={handleInputChange}
    className="mt-1 block w-full px-3 py-2 border border-gray-300 rounded-md shadow-sm focus:outline-none focus:ring-blue-500 focus:border-b
  />
</div>
<div>
  <label htmlFor="hscAlevel" className="block text-sm font-medium text-gray-700">HSC/A-Level</label>
  <input
    type="text"
    id="hscAlevel"
    name="hscAlevel"
    value={profile.hscAlevel}
    onChange={handleInputChange}
    className="mt-1 block w-full px-3 py-2 border border-gray-300 rounded-md shadow-sm focus:outline-none focus:ring-blue-500 focus:border-b
  />
</div>
<div>
  <label htmlFor="sex" className="block text-sm font-medium text-gray-700">Sex</label>
```

```
            <label htmlFor="image" className="block text-sm font-medium text-gray-700">Profile Image</label>
            <input
              type="file"
              id="image"
              name="image"
              accept="image/*"
              onChange={handleFileChange}
              className="mt-1 block w-full px-3 py-2 border border-gray-300 rounded-md shadow-sm focus:outline-none focus:ring-blue-500 f
            />
          </div>
        </div>
        <div className="mt-6 flex justify-between">
          <button
            type="button"
            onClick={isNewUser ? handleSave : handleUpdate}
            className="bg-blue-500 text-white px-4 py-2 rounded-md hover:bg-blue-600 transition duration-300"
          >
            {isNewUser ? 'Save' : 'Update'}
          </button>
          <button
            type="button"
            onClick={handleResumeBuilder}
            className="bg-green-500 text-white px-4 py-2 rounded-md hover:bg-green-600 transition duration-300"
          >
            Resume Builder
          </button>
        </div>
      </form>
    </div>
  </div>
</div>
);
}

export default UserProfile;
```

Contribution of ID : 2101201, Name : Samia Tabassum

All_jobs

```
import { useNavigate, useLocation } from 'react-router-dom';
```

```
    const location = useLocation();
```

```
    const [searchTerm, setSearchTerm] = useState("");
    const [results, setResults] = useState([])

    const handleSubmit = (e) => {
        e.preventDefault();

        if (searchTerm.trim() === "") {
            navigate('/all_jobs');
        } else {
            const urlParams = new URLSearchParams();
            urlParams.set("searchTerm", searchTerm);
            const searchQuery = urlParams.toString();
            navigate(`/all_jobs?${searchQuery}`);
        }
```

```
    }

    useEffect(() => {
        const urlParams = new URLSearchParams(location.search);
        const searchTermFromUrl = urlParams.get('searchTerm');
        if (searchTermFromUrl) {
            setSearchTerm(searchTermFromUrl);
        }
```

```
  //fetch search results
        const fetchResults = async () => {
            try {
                const response = await
fetch(`http://localhost:4001/jobs/search?searchTerm=${encodeURIComponent(s
earchTermFromUrl)}`);
                if (!response.ok) throw new Error('Network response was
not ok');
                const data = await response.json();
                setResults(data);
            } catch (error) {
                console.error('Error fetching jobs:', error);;
            }
      };

      if (searchTermFromUrl) {
        fetchResults();
      } else {
        fetchJobs();
      }
    }, [location.search, userId]);

    const dispJobs = searchTerm ? results : jobs;
```

```
                <form onSubmit={handleSubmit}>
                <div className='hidden md:block'>
                    <label className="px-3 py-3 border rounded-md flex
items-center gap-1">
                    <input type="text"
```

```
                    className="grow outline-none px-2 py-1 border-3
rounded-md  dark:bg-slate-900 dark:text-white"
                    placeholder="Search"
                    value = {searchTerm}
                    onChange={(e) => setSearchTerm(e.target.value)}
                    />
                    <svg
                        xmlns="http://www.w3.org/2000/svg"
                        viewBox="0 0 16 16"
                        fill="currentColor"
                        className="h-4 w-4 opacity-70">
                        <path
                        fillRule="evenodd"
                        d="M9.965 11.026a5 5 0 1 1 1.06-1.06l2.755
2.754a.75.75 0 1 1-1.06 1.06l-2.755-2.754ZM10.5 7a3.5 3.5 0 1 1-7 0 3.5
3.5 0 0 1 7 0Z"
                        clipRule="evenodd" />
                    </svg>
                    </label>

                </div>
                </form>
```

Admin dashboard (Jobs)

```
            <li>
            <NavLink to="/admin_dashboard/ad_jobs"
className="text-gray-300 hover:text-white">
                Jobs
            </NavLink></li>
```

Ad_jobs

```
import React, { useEffect, useState } from 'react';
import { useNavigate } from 'react-router-dom';
import axios from "axios";

function Ad_Jobs() {
  const navigate = useNavigate();
```

```
  const [jobs, setJobs] = useState([]);

  useEffect(() => {
    const fetchJobs = async () => {
      try {
        const res = await axios.get('http://localhost:4001/jobs/all');
        console.log(res.data)
        setJobs(res.data);
      } catch(error){
        console.log(error)
      }
    };

    fetchJobs();
  }, []);

  return (
          <div className="flex h-screen">

          {/* Main content */}
          <div className="flex-1 flex flex-col">

          {/* Additional content or components */}
          <div className="bg-blue-400 dark:bg-slate-700 p-3 items-center">
            <h2 className="font-bold text-indigo-900 dark:text-blue-400
text-lg">Job Listings</h2>
           </div>
           {/* Table */}
           <div className="overflow-x-auto p-4">
              <table className="table w-full">
                  <thead className="bg-gray-700">
                    <tr>
                      <th className="text-base text-gray-400 px-4">Job
Title</th>
                      <th className="text-base text-gray-400 px-4">
Company</th>
                      <th className="text-base text-gray-400
px-4">Location</th>
```

```
                        <th className="text-base text-gray-400
px-4">Actions</th>
                    </tr>
                    </thead>

                    <tbody>
                    {jobs.map(job => (
                      <tr key={job._id}>
                        <td>{job.jobTitle}</td>
                        <td>{job.companyName}</td>
                        <td>{job.location}</td>
                        <td>
                          <button
                            className="bg-blue-500 text-white px-4 py-2
rounded-md hover:bg-blue-700 transition duration-300"
                            onClick={() =>
navigate(`/admin_dashboard/ad_jobs/${job._id}`)}>
                            Details
                          </button>
                        </td>
                      </tr>
                    ))}
                    </tbody>

                </table>
                </div>
            </div>
        </div>

    )
}

export default Ad_Jobs;
```

Ad_job_det

```
import {React, useState, useEffect} from 'react'
import { useParams, Link } from 'react-router-dom';
import { Button } from '@mui/material';
import { useNavigate } from 'react-router-dom';
```

```
function Ad_Job_Details() {
    const navigate = useNavigate();
    const { jobId } = useParams();
    const [job, setJob] = useState(null);
    const [loading, setLoading] = useState(true);
    const [error, setError] = useState(null);
    const [applicationCounts, setApplicationCounts] = useState(0);
    const [acceptedCounts, setAcceptedCounts] = useState(0);
    const [rejectedCounts, setRejectedCounts] = useState(0);

    const handleDeleteClick = async (jobId) => {
      try {
        const response = await
fetch(`http://localhost:4001/jobs/${jobId}`, {
          method: 'DELETE',
        });
        if (response.ok) {
          setJob(null);
          navigate('/admin_dashboard/ad_jobs');
        } else {
          console.error('Failed to delete job');
        }
      } catch (error) {
        console.error('Error:', error);
      }
    };

    useEffect(() => {
      const fetchJob = async () => {
        try {
          const response = await
fetch(`http://localhost:4001/jobs/${jobId}`);
          if (response.ok) {
            const data = await response.json();
            setJob(data);

            // Fetch total application counts
            const appCount = await fetchApplicationCounts(jobId);
            setApplicationCounts(appCount);
```

```
            // Fetch accepted and rejected counts
            const acceptedAndRejectedCounts = await
fetchAcceptedAndRejectedCounts(data._id);
            setAcceptedCounts(acceptedAndRejectedCounts.accepted);
            setRejectedCounts(acceptedAndRejectedCounts.rejected);

          } else {
            setError('Failed to fetch job details');
          }
        } catch (error) {
          setError('Error fetching job details');
          console.error('Error:', error);
        } finally {
          setLoading(false);
        }
      };

      const fetchAcceptedAndRejectedCounts = async (jobId) => {
        const counts = {
          accepted: 0,
          rejected: 0,
        };

        try {
          const [acceptedResponse, rejectedResponse] = await Promise.all([

fetch(`http://localhost:4001/applications/jobs/${jobId}/accepted-count`),

fetch(`http://localhost:4001/applications/jobs/${jobId}/rejected-count`)
          ]);

          if (acceptedResponse.ok) {
            const acceptedData = await acceptedResponse.json();
            counts.accepted = acceptedData.acceptedCount;
          }

          if (rejectedResponse.ok) {
            const rejectedData = await rejectedResponse.json();
            counts.rejected = rejectedData.rejectedCount;
```

```
          }
        } catch (error) {
          console.error('Error fetching accepted and rejected counts:',
error);
        }

        return counts;
      };

      const fetchApplicationCounts = async (jobId) => {
        try {
          const response = await
fetch(`http://localhost:4001/applications/job/${jobId}/counts`);
          if (response.ok) {
            const data = await response.json();
            return data.totalCount;
          } else {
            return 0;
          }
        } catch (error) {
          console.error('Error fetching application counts:', error);
          return 0;
        }
      };

      fetchJob();
    }, [jobId]);

    if (loading) return (
      <div className="flex justify-center items-center h-40">
        <span className="text-xl">Loading...</span>
      </div>
    );

    if (error) return <p className="text-red-600">{error}</p>;

    const formatDate = (dateString) => {
      if (!dateString) {
          return "N/A";
      }
```

```
      const date = new Date(dateString);
      const today = new Date();
      const tomorrow = new Date(today);

      // Reset hours for today and tomorrow
      today.setHours(0, 0, 0, 0);
      tomorrow.setDate(today.getDate() + 1);
      tomorrow.setHours(0, 0, 0, 0);

      if (date.toISOString().split('T')[0] ===
today.toISOString().split('T')[0]) {
          return `Today is the deadline
(${date.toISOString().split('T')[0]})`;
      }

      if (date.toISOString().split('T')[0] ===
tomorrow.toISOString().split('T')[0]) {
          return `Tomorrow is the deadline
(${date.toISOString().split('T')[0]})`;
      }

      return date.toISOString().split('T')[0];
    };

    return (
      <div className="p-6 bg-blue-200 min-h-screen dark:text-slate-200
dark:bg-slate-800 flex items-center justify-center">
        <div className="max-w-4xl w-full bg-yellow-50 p-4 rounded-md
shadow-md dark:bg-slate-300 dark:text-black">
        <p style={{ position: 'absolute', top: '125px', right: '55px' }}>
          <button
          className='btn-sm btn-circle btn-ghost bg-gray-200
hover:bg-gray-100 dark:bg-slate-500 dark:hover:bg-slate-600 text-2xl
hover:text-black dark:hover:text-gray-400'
          onClick={() => navigate('/admin_dashboard/ad_jobs')}>
                    ×
          </button>
          </p>
          {job ? (
```

```
            <>
              <h2 className="text-2xl font-bold mb-4">{job.jobTitle}</h2>
              <p><strong>Company:</strong> {job.companyName}</p>
              <p><strong>Location:</strong> {job.location}</p>
              <p><strong>Work Mode:</strong> {job.workMode}</p>
              <p><strong>Job Type:</strong> {job.jobType}</p>
              <p><strong>Category:</strong> {job.jobCategory}</p>
              <p><strong>Experience Level:</strong>
{job.experienceLevel}</p>
              <p>
                <strong>Salary Range:</strong> BDT {job.minSalary} -
{job.maxSalary}
                {job.negotiable ? ' (Negotiable)' : ''}
              </p>
              <p><strong>Responsibilities:</strong>
{job.responsibilities}</p>
              <p><strong>Requirements:</strong> {job.requirements}</p>
              <p><strong>Preferred Qualifications:</strong>
{job.preferredQualifications}</p>
              <p><strong>Benefits:</strong> {job.benefits}</p>
              <p><strong>Posted:</strong> {formatDate(job.createdAt)}</p>
              <p><strong>Deadline:</strong> {formatDate(job.deadline)}</p>
              <p><strong>Applications Count:</strong> {applicationCounts
|| 0}</p>
              <p><strong>Accepted Applications:</strong> {acceptedCounts
|| 0}</p>
              <p><strong>Rejected Applications:</strong> {rejectedCounts
|| 0}</p>
              <div className="flex justify-end mt-4">
                {/*<Link to={`/edit-job/${job._id}/`}>
                <Button
                  variant="contained"
                  color="primary"
                  style={{ marginRight: '10px' }}>
                  Update
                </Button>
                </Link>*/}

                  <Button onClick={() => handleDeleteClick(job._id)}
variant="contained" color="primary" className="relative">
```

```
                    Delete
                  </Button>

              </div>
            </>
          ) : (
            <p>Job not found.</p>
          )}
        </div>
      </div>
    );
  }

export default Ad_Job_Details
```

Backend Development

Contribution of ID :21201114, Name : Tasnim Rahman
Admin Login and Signup:
Model:

```
import mongoose from "mongoose";
const user2Schema= new mongoose.Schema({
    admin_name:{
        type:String,
        required:true
    },
    admin_email:{
        type:String,
        required:true,
        unique:true
    },
    password:{
        type:String,
        required:true
    },
})
const User2=mongoose.model("User2",user2Schema);
export default User2;
```

Controller:

```
import User2 from "../model/admin1.model.js";
import bcryptjs from "bcryptjs";
export const signup2=async(req,res)=>{
    try{
        const {admin_name,admin_email,password }=req.body;
        const user2= await User2.findOne({admin_email});
        if(user2){
            return res.status(400).json({message:"User already exists"})
        }
        const hashPassword= await bcryptjs.hash(password,8)
        const createdUser1=new User2({
            admin_name:admin_name,
            admin_email:admin_email,
            password:hashPassword,
        })
```

```
        await createdUser1.save()
        res.status(201).json({message:"User created successfully"})
    }catch(error){
        console.log("Error: " + error.message);
        res.status(500).json({message:"Internal server error"});
    }
};
export const login2=async(req,res)=>{
    try{
        const {admin_email, password,admin_name} = req.body;
        const user2 = await User2.findOne({ admin_email });
        if (!user2) {
            return res.status(400).json({ message: "Invalid email or
password" });
        }
        if (user2.admin_name !== admin_name) {
            return res.status(400).json({ message: "Invalid company name"
});
        }
        const isMatch = await bcryptjs.compare(password, user2.password);
        if (!isMatch) {
            return res.status(400).json({ message: "Invalid email or
password" });
        }
        res.status(200).json({
            message: "Login successful",
            user2: {
                _id: user2._id,
                admin_name: user2.admin_name,
                admin_email: user2.admin_email,
            },
        });
    }catch(error){
        console.log("Error:"+error.message)
        res.status(500).json({message:"Internal server error"});
    }
}
```

Route:

```
import express from "express";
import { login2, signup2 } from "../controller/admin1.controller.js";
```

```
const router=express.Router()
router.post("/signup_admin",signup2)
router.post("/login_admin",login2)
export default router;
```

Company Login and Signup:

Model:

```
import mongoose from "mongoose";
const user1Schema= new mongoose.Schema({
    company_name:{
        type:String,
        required:true
    },
    company_email:{
        type:String,
        required:true,
        unique:true
    },

company_address:{
    type:String,
    required:true,

},
    password:{
        type:String,
        required:true
    },

})
const User1=mongoose.model("User1",user1Schema);
export default User1;
```

Controller:

```
import User1 from "../model/company.model.js";
import bcryptjs from "bcryptjs";


export const signup1 = async (req, res) => {
  try {
```

```
    const { company_name, company_email, company_address, password } =
req.body;
    const user1 = await User1.findOne({ company_email });
    if (user1) {
      return res.status(400).json({ message: "User already exists" });
    }
    const hashPassword = await bcryptjs.hash(password, 8);
    const createdUser1 = new User1({
      company_name,
      company_email,
      company_address,
      password: hashPassword,
    });
    await createdUser1.save();
    res.status(201).json({ message: "User created successfully",
company_name });
  } catch (error) {
    console.log("Error: " + error.message);
    res.status(500).json({ message: "Internal server error" });
  }
};

export const login1 = async (req, res) => {
  try {
    const { company_name, company_email, password } = req.body;
    const user1 = await User1.findOne({ company_email });
    if (!user1) {
      return res.status(400).json({ message: "Invalid email or password"
});
    }

    if (user1.company_name !== company_name) {
      return res.status(400).json({ message: "Invalid company name" });
    }

    const isMatch = await bcryptjs.compare(password, user1.password);
    if (!isMatch) {
      return res.status(400).json({ message: "Invalid email or password"
});
    }
```

```
    res.status(200).json({
      message: "Login successful",
      user1: {
        _id: user1._id,
        company_name: user1.company_name,
        company_email: user1.company_email,
      },
    });
  } catch (error) {
    console.log("Error:" + error.message);
    res.status(500).json({ message: "Internal server error" });
  }
};
```

Route:

```
import express from "express";
import { signup1,login1 } from "../controller/company.controller.js";

const router=express.Router()

router.post("/signup_company",signup1)
router.post("/login_company",login1)

export default router;
```

Jobseeker Login and Signup:

Model:

```
import mongoose from "mongoose";
const userSchema= new mongoose.Schema({
    fullname:{
        type:String,
        required:true
    },
    email:{
        type:String,
        required:true,
        unique:true
```

```
    },
    password:{
        type:String,
        required:true
    },

})
const User=mongoose.model("User",userSchema);
export default User;
```

Controller:

```
import User from "../model/user.model.js";
import bcryptjs from "bcryptjs";

export const signup = async (req, res) => {
    try {
        const { fullname, email, password } = req.body;
        const user = await User.findOne({ email });
        if (user) {
            return res.status(400).json({ message: "User already exists"
});
        }
        const hashPassword = await bcryptjs.hash(password, 8);
        const createdUser = new User({
            fullname: fullname,
            email: email,
            password: hashPassword,
        });
        await createdUser.save();
        res.status(201).json({ message: "User created successfully" });
    } catch (error) {
        console.log("Error: " + error.message);
        res.status(500).json({ message: "Internal server error" });
    }
};

export const login = async (req, res) => {
    try {
        const { email, password } = req.body;
        const user = await User.findOne({ email });
        if (!user) {
```

```
            return res.status(400).json({ message: "Invalid email or
password" });
        }
        const isMatch = await bcryptjs.compare(password, user.password);
        if (!isMatch) {
            return res.status(400).json({ message: "Invalid email or
password" });
        }
        res.status(200).json({
            message: "Login successful",
            user: {
                _id: user._id,
                fullname: user.fullname,
                email: user.email
            }
        });
    } catch (error) {
        console.log("Error: " + error.message);
        res.status(500).json({ message: "Internal server error" });
    }
};
```

Route:

```
import express from "express";
import { signup,login } from "../controller/user.controller.js";
const router=express.Router()
router.post("/signup_jobseeker",signup)
router.post("/login_jobseeker",login)
export default router;
```

Jobs Management:add new job,edit,delete,bookmarking and other handling.
Model:

```
import mongoose from "mongoose";


const jobSchema = new mongoose.Schema({
  jobTitle: {
    type: String,
    required: true,
```

```
  },
  companyName: {
    type: String,
    required: true,
  },
  location: {
    type: String,
    required: true,
  },
  workMode: {
    type: String,
    required: true,
    enum: ["on-site", "remote", "hybrid"],
  },
  jobType: {
    type: String,
    required: true,
    enum: ["full-time", "part-time", "contract", "internship",
"freelance"],
  },
  jobCategory: {
    type: String,
    required: true,
    enum: ["it", "marketing", "sales", "finance", "hr"],
  },
  experienceLevel: {
    type: String,
    required: true,
    enum: ["entry-level", "mid-level", "senior-level", "manager",
"director"],
  },
  deadline: {
    type: Date,
    required: true,
  },
  minSalary: {
    type: Number,
    required: true,
  },
  maxSalary: {
```

```
    type: Number,
    required: true,
  },
  negotiable: {
    type: Boolean,
    default: false,
  },
  responsibilities: {
    type: String,
    required: true,
  },
  requirements: {
    type: String,
    required: true,
  },
  preferredQualifications: {
    type: String,
  },
  benefits: {
    type: String,
  },
  company: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'User1',
    //ref: 'Company',
    required: true,
  },
  //
  bookmarkedBy: [{ type: mongoose.Schema.Types.ObjectId, ref: 'User' }],
  bookmarkedByCompany: {
    type: Boolean,
    default: false,
  },
  vacancies:{
    type: Number,
    required: true,
  },
  isDeleted: {
    type: Boolean,
    default: false,  // Initially, the job is not deleted
```

```
  },
}, {
  timestamps: true,
});
const Job = mongoose.model("Job", jobSchema);
export default Job;
```

Controller:

```
import mongoose from 'mongoose';
import Job from "../model/jobs.model.js";
export const addJob = async (req, res) => {
  try {
    const {
      jobTitle,
      companyName,
      location,
      workMode,
      jobType,
      jobCategory,
      experienceLevel,
      minSalary,
      maxSalary,
      negotiable,
      responsibilities,
      requirements,
      preferredQualifications,
      benefits,
      company,
      deadline,
      vacancies
    } = req.body;
    if (!company) {
      return res.status(400).json({ message: "Company ID is required" });
    }
    const existingJob = await Job.findOne({ jobTitle, companyName });
    if (existingJob) {
      return res.status(400).json({ message: "This job already exists" });
    }
    const newJob = new Job({
      jobTitle,
```

```
      companyName,
      location,
      workMode,
      jobType,
      jobCategory,
      experienceLevel,
      minSalary,
      maxSalary,
      negotiable,
      responsibilities,
      requirements,
      preferredQualifications,
      benefits,
      company,
      deadline,
      vacancies
    });
    const savedJob = await newJob.save();
    res.status(201).json({ message: "Job created successfully", job:
savedJob });
  } catch (error) {
    console.log("Error: " + error.message);
    res.status(500).json({ message: "Internal server error" });
  }
};

// Fetch all jobs with bookmark status for a user
export const getAllJobs = async (req, res) => {
  try {
    const currentDate = new Date();
    const userId = req.query.userId;
    const jobs = await Job.find({ deadline: { $gte: currentDate } });
    if (userId) {
      const jobsWithBookmarkStatus = jobs.map(job => ({
        ...job.toObject(),
        isBookmarked: job.bookmarkedBy.includes(userId)
      }));
      res.status(200).json(jobsWithBookmarkStatus);
    } else {
      res.status(200).json(jobs);
```

```
    }
  } catch (error) {
    console.log("Error: " + error.message);
    res.status(500).json({ message: "Internal server error" });
  }
};
// Toggle bookmark status for a company
export const toggleBookmarkCompany = async (req, res) => {
  try {
    const { jobId } = req.body;
    if (!jobId) {
      return res.status(400).json({ message: "Job ID is required" });
    }
    if (!mongoose.Types.ObjectId.isValid(jobId)) {
      return res.status(400).json({ message: "Invalid Job ID format" });
    }
    const job = await Job.findById(jobId);
    if (!job) {
      return res.status(404).json({ message: "Job not found" });
    }
    job.bookmarkedByCompany = !job.bookmarkedByCompany;
    await job.save();
    res.status(200).json({ bookmarkedByCompany: job.bookmarkedByCompany
});
  } catch (error) {
    console.log("Error: " + error.message);
    res.status(500).json({ message: "Internal server error" });
  }
};
// Fetch jobs for a company and optionally toggle bookmark status
export const getJobsByCompany = async (req, res) => {
  try {
    const { company, toggleJobId } = req.query;
    if (!company) {
      return res.status(400).json({ message: "Company ID is required" });
    }
    if (toggleJobId) {
      if (!mongoose.Types.ObjectId.isValid(toggleJobId)) {
        return res.status(400).json({ message: "Invalid Job ID format" });
      }
```

```
      const job = await Job.findById(toggleJobId);
      if (!job) {
        return res.status(404).json({ message: "Job not found" });
      }
      job.bookmarkedByCompany = !job.bookmarkedByCompany;
      await job.save();
      return res.status(200).json({ bookmarkedByCompany:
job.bookmarkedByCompany });
    }
    const jobs = await Job.find({ company });
    res.status(200).json(jobs);
  } catch (error) {
    console.log("Error: " + error.message);
    res.status(500).json({ message: "Internal server error" });
  }
};
// Fetch bookmarked jobs for a company
export const getBookmarkedJobsByCompany = async (req, res) => {
  try {
    const { companyId } = req.params;
    if (!companyId) {
      return res.status(400).json({ message: "Company ID is required" });
    }
    if (!mongoose.Types.ObjectId.isValid(companyId)) {
      return res.status(400).json({ message: "Invalid Company ID format"
});
    }
    const jobs = await Job.find({
      company: companyId,
      bookmarkedByCompany: true
    });
    res.status(200).json(jobs);
  } catch (error) {
    console.log("Error: " + error.message);
    res.status(500).json({ message: "Internal server error" });
  }
};
//fetching a single job by ID
export const getJobById = async (req, res) => {
  try {
```

```
    const job = await Job.findById(req.params.id);
    if (!job) {
      return res.status(404).json({ message: "Job not found" });
    }
    res.status(200).json(job);
  } catch (error) {
    console.log("Error: " + error.message);
    res.status(500).json({ message: "Internal server error" });
  }
};
// Edit
export const updateJob = async (req, res) => {
  try {
    const {
      jobTitle,
      companyName,
      location,
      workMode,
      jobType,
      jobCategory,
      experienceLevel,
      minSalary,
      maxSalary,
      negotiable,
      responsibilities,
      requirements,
      preferredQualifications,
      benefits,
      deadline,
      vacancies
    } = req.body;
    const { id } = req.params;
    const job = await Job.findById(id);
    if (!job) {
      return res.status(404).json({ message: "Job not found" });
    }
    const duplicateJob = await Job.findOne({
      _id: { $ne: id },
      jobTitle,
      companyName,
```

```
    });
    if (duplicateJob) {
      return res.status(400).json({ message: "This job already exists" });
    }
    job.jobTitle = jobTitle;
    job.companyName = companyName;
    job.location = location;
    job.workMode = workMode;
    job.jobType = jobType;
    job.jobCategory = jobCategory;
    job.experienceLevel = experienceLevel;
    job.minSalary = minSalary;
    job.maxSalary = maxSalary;
    job.negotiable = negotiable;
    job.responsibilities = responsibilities;
    job.requirements = requirements;
    job.preferredQualifications = preferredQualifications;
    job.benefits = benefits;
    job.deadline=deadline;
    job.vacancies=vacancies;
    const updatedJob = await job.save();
    res.status(200).json({ message: "Job updated successfully", job:
updatedJob });
  } catch (error) {
    console.log("Error: " + error.message);
    res.status(500).json({ message: "Internal server error" });
  }
};
//delete Parmanently
export const deleteJob = async (req, res) => {
  try {
    const job = await Job.findByIdAndDelete(req.params.id);
    if (!job) {
      return res.status(404).json({ message: "Job not found" });
    }
    res.status(200).json({ message: "Job deleted successfully" });
  } catch (error) {
    console.log("Error: " + error.message);
    res.status(500).json({ message: "Internal server error" });
  }
```

```
};
// Toggle bookmark status for a job
export const toggleBookmark = async (req, res) => {
  try {
    const { userId, jobId } = req.body;
    if (!userId || !jobId) {
      return res.status(400).json({ message: "User ID and Job ID are
required" });
    }
    if (!mongoose.Types.ObjectId.isValid(userId) ||
!mongoose.Types.ObjectId.isValid(jobId)) {
      return res.status(400).json({ message: "Invalid User ID or Job ID
format" });
    }
    const job = await Job.findById(jobId);
    if (!job) {
      return res.status(404).json({ message: "Job not found" });
    }
    const isBookmarked = job.bookmarkedBy.includes(userId);
    if (isBookmarked) {
      job.bookmarkedBy.pull(userId);
    } else {
      job.bookmarkedBy.push(userId);
    }
    await job.save();
    res.status(200).json({ message: isBookmarked ? "Bookmark removed" :
"Job bookmarked" });
  } catch (error) {
    console.log("Error: " + error.message);
    res.status(500).json({ message: "Internal server error" });
  }
};
export const getBookmarkedJobs = async (req, res) => {
  try {
    const userId = req.params.userId;
    if (!userId) {
      return res.status(400).json({ message: "User ID is required" });
    }
    const jobs = await Job.find({ bookmarkedBy: userId });
    if (!jobs.length) {
```

```
      return res.status(404).json({ message: "No bookmarked jobs found"
});
    }
    res.status(200).json(jobs);
  } catch (error) {
    console.error("Error fetching bookmarked jobs:", error.message);
    res.status(500).json({ message: "Internal server error" });
  }
};
// Soft delete a job by setting isDeleted to true
export const softDeleteJob = async (req, res) => {
  try {
    const { jobId } = req.params;
    if (!mongoose.Types.ObjectId.isValid(jobId)) {
      return res.status(400).json({ message: "Invalid Job ID format" });
    }
    const job = await Job.findById(jobId);
    if (!job) {
      return res.status(404).json({ message: "Job not found" });
    }
    job.isDeleted = true;
    const updatedJob = await job.save();
    res.status(200).json({ message: "Job deleted successfully", job:
updatedJob });
  } catch (error) {
    console.error("Error:", error.message);
    res.status(500).json({ message: "Internal server error" });
  }
};
// Fetch deleted jobs for a specific company
export const getDeletedJobsByCompany = async (req, res) => {
  try {
    const { companyId } = req.params;
    if (!mongoose.Types.ObjectId.isValid(companyId)) {
      return res.status(400).json({ message: "Invalid Company ID format"
});
    }
    const deletedJobs = await Job.find({ company: companyId, isDeleted:
true });
    if (!deletedJobs.length) {
```

```
      return res.status(404).json({ message: "No deleted jobs found for
this company" });
    }
    res.status(200).json(deletedJobs);
  } catch (error) {
    console.error("Error fetching deleted jobs:", error.message);
    res.status(500).json({ message: "Internal server error" });
  }
};
// Restore a deleted job by setting isDeleted to false
export const restoreJob = async (req, res) => {
  try {
    const { jobId } = req.params;
    if (!mongoose.Types.ObjectId.isValid(jobId)) {
      return res.status(400).json({ message: "Invalid Job ID format" });
    }
    const job = await Job.findById(jobId);
    if (!job) {
      return res.status(404).json({ message: "Job not found" });
    }
    if (!job.isDeleted) {
      return res.status(400).json({ message: "Job is not deleted" });
    }
    job.isDeleted = false;
    const updatedJob = await job.save();
    res.status(200).json({ message: "Job restored successfully", job:
updatedJob });
  } catch (error) {
    console.error("Error:", error.message);
    res.status(500).json({ message: "Internal server error" });
  }
};
export const advancedRestoreJob = async (req, res) => {
  try {
    const { jobId } = req.params;
    if (!mongoose.Types.ObjectId.isValid(jobId)) {
      return res.status(400).json({ message: "Invalid Job ID format" });
    }
    const job = await Job.findById(jobId);
    if (!job) {
```

```
      return res.status(404).json({ message: "Job not found" });
    }
    if (!job.isDeleted) {
      return res.status(400).json({ message: "Job is not deleted" });
    }
    job.isDeleted = false;
    job.bookmarkedBy = [];
    job.bookmarkedByCompany = false;
    const updatedJob = await job.save();
    res.status(200).json({ message: "Job restored successfully with
additional fields updated", job: updatedJob });
  } catch (error) {
    console.error("Error:", error.message);
    res.status(500).json({ message: "Internal server error" });
  }
};
```

Route:

```
import express from 'express';
import { addJob, getAllJobs, getJobsByCompany, getJobById, updateJob,
deleteJob,toggleBookmark,
getBookmarkedJobs,toggleBookmarkCompany,getBookmarkedJobsByCompany,softDel
eteJob,getDeletedJobsByCompany,restoreJob,advancedRestoreJob} from
'../controller/jobs.controller.js';
const router = express.Router();
// adding new
router.post('/addjobs', addJob);
// geting all jobs
router.get('/all', getAllJobs);
// geting all jobs for a specific company
router.get('/', getJobsByCompany);
// geting a single job by ID
router.get('/:id', getJobById);
//Edit
router.put('/:id', updateJob);
//delete
router.delete('/:id', deleteJob);
```

```
// Toggle bookmark
router.post('/bookmark', toggleBookmark);
// Get bookmarked jobs for a user
router.get('/bookmarked/:userId', getBookmarkedJobs);
// Toggle bookmark status for company
router.post('/bookmark/company', toggleBookmarkCompany);
// Fetch bookmarked jobs for a company
router.get('/bookmarked-jobs/:companyId', getBookmarkedJobsByCompany);
// Route to soft delete a job by ID
router.put('/:jobId/soft-delete', softDeleteJob);
// Route to get deleted jobs by company ID
router.get('/deleted/:companyId', getDeletedJobsByCompany);
// Route to restore a soft deleted job
router.patch('/restore/:jobId', restoreJob);
// Route to advanced restore a soft deleted job with additional field
updates
router.patch('/advancedRestore/:jobId', advancedRestoreJob);
export default router;
```

Contribution of ID : 22101117, Name : Shawana Maliha

application.route.js

```
import express from 'express';
import multer from 'multer';
import path from 'path';
import {
    applyForJob,
    getUserApplications,
    updateApplication,
    deleteApplication,
    getApplicationsForJob,
    acceptApplication,
    rejectApplication,
    getAcceptedApplications,
    getRejectedApplications,
    deleteApplicationFromCompany,
    getApplicationCountsForJob,
```

```
    getAcceptedApplicationsCountForJob,
    getRejectedApplicationsCountForJob,
    hasUserAppliedForJob,
    deleteApplicationFromJobseeker,
    getApplicants


} from '../controller/applications.controller.js';

const router = express.Router();

// Configure multer storage
const storage = multer.diskStorage({
    destination: function (req, file, cb) {
        cb(null, path.join(process.cwd(), 'uploads'));
    },
    filename: function (req, file, cb) {
        cb(null, Date.now() + path.extname(file.originalname));
    }
});

// Initialize multer with the storage configuration
const upload = multer({
    storage: storage,
    limits: {
        fileSize: 5 * 1024 * 1024 // Limit file size to 5 MB
    },
    fileFilter: (req, file, cb) => {
        const allowedTypes = /pdf|doc|docx|jpg|jpeg|png/;
        const extname =
allowedTypes.test(path.extname(file.originalname).toLowerCase());
        const mimetype = allowedTypes.test(file.mimetype);

        if (mimetype && extname) {
            return cb(null, true);
        }
        cb(new Error('Only .pdf, .doc, .docx, .jpg, .jpeg, .png files are
allowed!'));
    }
});
```

```
router.post('/addapplication', upload.fields([{ name: 'coverLetter',
maxCount: 1 }, { name: 'cv', maxCount: 1 }]), applyForJob);

router.get('/:userId', getUserApplications);

router.put('/updateapplication', upload.fields([{ name: 'coverLetter',
maxCount: 1 }, { name: 'cv', maxCount: 1 }]), updateApplication);




// Delete an application
router.delete('/:applicationId', deleteApplication);


// Get all applications (optional status filter)
router.get('/job/:jobId', getApplicationsForJob);




// Accept an application
//router.put('/applications/:applicationId/accept', acceptApplication);
router.put('/:applicationId/accept', acceptApplication);
//router.get('/accepted', getAcceptedApplications);
// Get all accepted applications for jobs created by a company
//router.get('/accepted-applications/:companyId',
getAcceptedApplications);
// Add this route to get all accepted applications for a company
router.get('/company/:companyId/accepted', getAcceptedApplications);




// Reject an application
router.put('/:applicationId/reject', rejectApplication);
router.get('/company/:companyId/rejected', getRejectedApplications);
```

```
// DELETE /applications/:applicationId/delete
router.delete('/:applicationId/delete', deleteApplicationFromCompany);


// routes/applications.route.js
router.get('/job/:jobId/counts', getApplicationCountsForJob);

// Add routes for counting accepted and rejected applications
router.get('/jobs/:jobId/accepted-count',
getAcceptedApplicationsCountForJob);
router.get('/jobs/:jobId/rejected-count',
getRejectedApplicationsCountForJob);

// Define the route for checking application status
router.get('/check-application/:userId/:jobId', hasUserAppliedForJob);


// New route to update application status to 'Deletedby' by jobseeker
router.delete('/jobseeker/:applicationId',
deleteApplicationFromJobseeker);

router.get('/applicants', getApplicants);

export default router;
```

company.controller.js

```
import User1 from "../model/company.model.js";
import bcryptjs from "bcryptjs";

export const signup1 = async (req, res) => {
  try {
    const { company_name, company_email, company_address, password } =
req.body;
    const user1 = await User1.findOne({ company_email });
    if (user1) {
      return res.status(400).json({ message: "User already exists" });
    }
    const hashPassword = await bcryptjs.hash(password, 8);
    const createdUser1 = new User1({
      company_name,
```

```
      company_email,
      company_address,
      password: hashPassword,
    });
    await createdUser1.save();
    res.status(201).json({ message: "User created successfully",
company_name });
  } catch (error) {
    console.log("Error: " + error.message);
    res.status(500).json({ message: "Internal server error" });
  }
};

export const login1 = async (req, res) => {
  try {
    const { company_name, company_email, password } = req.body;
    const user1 = await User1.findOne({ company_email });
    if (!user1) {
      return res.status(400).json({ message: "Invalid email or password"
});
    }

    if (user1.company_name !== company_name) {
      return res.status(400).json({ message: "Invalid company name" });
    }

    const isMatch = await bcryptjs.compare(password, user1.password);
    if (!isMatch) {
      return res.status(400).json({ message: "Invalid email or password"
});
    }

    res.status(200).json({
      message: "Login successful",
      user1: {
        _id: user1._id,
        company_name: user1.company_name,
        company_email: user1.company_email,
        company_address: user1.company_address,
      },
```

```
    });
  } catch (error) {
    console.log("Error:" + error.message);
    res.status(500).json({ message: "Internal server error" });
  }
};

export const getCompanyDetails1 = async (req, res) => {
  try {
    const { company_name, company_email, company_address } = req.query;
    const query = {};

    if (company_name) query.company_name = company_name;
    if (company_email) query.company_email = company_email;
    if (company_address) query.company_address = company_address;
    const companies = await User1.find(query);
    if (companies.length === 0) {
      return res.status(404).json({ message: "Company not found" });
    }
    res.status(200).json(companies);
  } catch (error) {
    console.log("Error: " + error.message);
    res.status(500).json({ message: "Internal server error" });
  }
};

export const getCompanyDetails = async (req, res) => {
  try {
    const { company_email } = req.query;


    // Check if company_email is provided
    if (!company_email) {
      return res.status(400).json({ message: "company_email is required"
});
    }


    const company = await User1.findOne({ company_email });
```

```
    if (!company) {
      return res.status(404).json({ message: "Company not found" });
    }

    // Respond with company details
    res.status(200).json({
      company_email: company.company_email,
      company_address: company.company_address,
      company_name: company.company_name,
    });

  } catch (error) {
    console.log("Error: " + error.message);
    res.status(500).json({ message: "Internal server error" });
  }
};
```

company.model.js

```
import mongoose from "mongoose";
const user1Schema= new mongoose.Schema({
    company_name:{
        type:String,
        required:true
    },
    company_email:{
        type:String,
        required:true,
        unique:true
    },

company_address:{
    type:String,
    required:true,
```

```
},
    password:{
        type:String,
        required:true
    },

})
const User1=mongoose.model("User1",user1Schema);
export default User1;
```

company.route.js

```
import express from "express";
import { signup1, login1, getCompanyDetails, getCompanyDetails1 } from
"../controller/company.controller.js";

const router = express.Router();

router.post("/signup_company", signup1);
router.post("/login_company", login1);
router.get("/companies", getCompanyDetails);
router.get("/companies1", getCompanyDetails1);

export default router;
```

admin1.controller.js

```
import User2 from "../model/admin1.model.js";
import bcryptjs from "bcryptjs";
export const signup2=async(req,res)=>{
    try{
        const {admin_name,admin_email,password }=req.body;
        const user2= await User2.findOne({admin_email});
        if(user2){
            return res.status(400).json({message:"User already exists"})
        }
        const hashPassword= await bcryptjs.hash(password,8)
        const createdUser1=new User2({
            admin_name:admin_name,
            admin_email:admin_email,
            password:hashPassword,
```

```
        })
        await createdUser1.save()
        res.status(201).json({message:"User created successfully"})

    }catch(error){
        console.log("Error: " + error.message);
        res.status(500).json({message:"Internal server error"});

    }
};
export const login2=async(req,res)=>{
    try{
        const {admin_email, password,admin_name} = req.body;

        const user2 = await User2.findOne({ admin_email });

        if (!user2) {
            return res.status(400).json({ message: "Invalid email or
password" });
        }

        // Checking if the company names match
        if (user2.admin_name !== admin_name) {
            return res.status(400).json({ message: "Invalid company name"
});
        }

        const isMatch = await bcryptjs.compare(password, user2.password);
        if (!isMatch) {
            return res.status(400).json({ message: "Invalid email or
password" });
        }

        res.status(200).json({
            message: "Login successful",
            user2: {
                _id: user2._id,
                admin_name: user2.admin_name,
                admin_email: user2.admin_email,
            },
```

```
        });









        {/*const {company_email,password}=req.body;

        const user1=await User1.findOne({company_email});
        const isMatch=await bcryptjs.compare(password,user1.password)
        if(!user1|| !isMatch){
            return res.status(400).json({message:"Invalid email or
password"});

        }else{
            res.status(200).json({message:"Login successful",user1:{
                _id:user1._id,
                company_name:user1.company_name,
                company_email:user1.company_email


        }})
        }}*/}
    }catch(error){
        console.log("Error:"+error.message)
        res.status(500).json({message:"Internal server error"});
    }
}
```

admin.model.js

```
import mongoose from "mongoose";
const user2Schema= new mongoose.Schema({
    admin_name:{
        type:String,
        required:true
    },
    admin_email:{
        type:String,
        required:true,
```

```
        unique:true
    },
    password:{
        type:String,
        required:true
    },

})
const User2=mongoose.model("User2",user2Schema);
export default User2;
```

admin.route.js

```
import express from "express";
import { login2, signup2 } from "../controller/admin1.controller.js";
const router=express.Router()
router.post("/signup_admin",signup2)
router.post("/login_admin",login2)
export default router;
```

user.controller.js

```
import User from "../model/user.model.js";
import bcryptjs from "bcryptjs";

export const signup = async (req, res) => {
    try {
        const { fullname, email, password } = req.body;
        const user = await User.findOne({ email });
        if (user) {
            return res.status(400).json({ message: "User already exists"
});
        }
        const hashPassword = await bcryptjs.hash(password, 8);
        const createdUser = new User({
            fullname: fullname,
            email: email,
            password: hashPassword,
        });
        await createdUser.save();
        res.status(201).json({ message: "User created successfully" });
    } catch (error) {
        console.log("Error: " + error.message);
        res.status(500).json({ message: "Internal server error" });
```

```
    }
};

export const login = async (req, res) => {
    try {
        const { email, password } = req.body;
        const user = await User.findOne({ email });
        if (!user) {
            return res.status(400).json({ message: "Invalid email or
password" });
        }
        const isMatch = await bcryptjs.compare(password, user.password);
        if (!isMatch) {
            return res.status(400).json({ message: "Invalid email or
password" });
        }
        res.status(200).json({
            message: "Login successful",
            user: {
                _id: user._id,
                fullname: user.fullname,
                email: user.email
            }
        });
    } catch (error) {
        console.log("Error: " + error.message);
        res.status(500).json({ message: "Internal server error" });
    }
};

export const getJobseekers = async (req, res) => {
    try {
        const jobseekers = await User.find({}, 'fullname email');
        res.status(200).json(jobseekers);
    } catch (error) {
        console.log("Error: " + error.message);
        res.status(500).json({ message: "Internal server error" });
    }
};
```

Contribution of ID : 23241082, Name : Rumman Nodi
Reset Password
Model:

```
import mongoose from 'mongoose';


const passwordResetSchema = new mongoose.Schema({
  email: {
    type: String,
    required: true,
  },
  token: {
    type: String,
    required: true,
  },
  expiresAt: {
    type: Date,
    required: true,
  },
});

passwordResetSchema.index({ expiresAt: 1 }, { expireAfterSeconds: 0 });

const PasswordReset = mongoose.model('PasswordReset', passwordResetSchema);

export default PasswordReset;
```

Controller:

```
import User from '../model/user.model.js';
import jwt from 'jsonwebtoken';
import bcrypt from 'bcryptjs';
import nodemailer from 'nodemailer';


const transporter = nodemailer.createTransport({
  service: 'gmail',
  auth: {
    user: process.env.EMAIL_USER,
    pass: process.env.EMAIL_PASS,
  },
});

export const sendResetLink = async (req, res) => {
  const { email } = req.body;

  try {
    const user = await User.findOne({ email });
    if (!user) {
      return res.status(404).json({ message: 'User not found' });
    }

    // Generate a token
    const token = jwt.sign({ email: user.email }, process.env.JWT_SECRET, { expiresIn: '1h' });

    // Create a reset link
    const resetLink = `${process.env.CLIENT_URL}/reset-password?token=${token}`;

    // Send email
    await transporter.sendMail({
      from: process.env.EMAIL_USER,
      to: email,
      subject: 'Password Reset',
      text: `You requested a password reset. Click the following link to reset your password: ${resetLink}`,
    });
```

```
    res.status(200).json({ message: 'Reset link sent to your email' });
  } catch (error) {
    res.status(500).json({ message: 'Error sending reset link', error });
  }
};

export const resetPassword = async (req, res) => {
  const { token, password } = req.body;

  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    const hashedPassword = await bcrypt.hash(password, 10);
    await User.updateOne({ email: decoded.email }, { password: hashedPassword });

    res.status(200).json({ message: 'Password reset successful' });
  } catch (error) {
    res.status(500).json({ message: 'Error resetting password', error });
  }
};
```

Route:

```
import express from 'express';
import { sendResetLink, resetPassword } from '../controller/passwordReset.controller.js';

const router = express.Router();

router.post('/send-reset-link', sendResetLink);
router.post('/reset-password', resetPassword);

export default router;
```

Profile Page
Model:

```
import mongoose from "mongoose";

const profileSchema = new mongoose.Schema({
  fullname: {
    type: String,
    required: true,
    trim: true
  },
  email: {
    type: String,
    required: true,
    unique: true,
    trim: true
  },
  phoneNumber: {
    type: String,
    required: true,
    trim: true
  },
  bloodGroup: {
    type: String,
    trim: true
  },
  religion: {
    type: String,
    trim: true
  },
  dateOfBirth: {
    type: Date,
    required: true
  },
  institution: {
    type: String,
    trim: true
  },
```

```
  sscOlevel: {
    type: String,
    trim: true
  },
  hscAlevel: {
    type: String,
    trim: true
  },
  sex: {
    type: String,
    enum: ['male', 'female', 'other'],
    required: true
  },
  image: {
    type: String,
    trim: true
  }
}, { timestamps: true });

const Profile = mongoose.model("Profile", profileSchema);

export default Profile;
```

Controller:

```
import Profile from '../model/profile.model.js'; // Import the Profile model

// Save a new profile or update an existing profile
export const saveOrUpdateProfile = async (req, res) => {
  try {
    const {
      fullname, email, phoneNumber, bloodGroup, religion,
      dateOfBirth, institution, sscOlevel, hscAlevel, sex,
      image
    } = req.body;

    // Check if the profile exists based on email (unique identifier)
    let profile = await Profile.findOne({ email });

    if (profile) {
      // Update existing profile
      profile.fullname = fullname;
      profile.phoneNumber = phoneNumber;
      profile.bloodGroup = bloodGroup;
      profile.religion = religion;
      profile.dateOfBirth = dateOfBirth;
      profile.institution = institution;
      profile.sscOlevel = sscOlevel;
      profile.hscAlevel = hscAlevel;
      profile.sex = sex;
      profile.image = image;

      // Save the updated profile
      await profile.save();
      return res.status(200).json({ message: 'Profile updated successfully', profile });
    } else {
```

```
      profile = new Profile({
        fullname,
        email,
        phoneNumber,
        bloodGroup,
        religion,
        dateOfBirth,
        institution,
        sscOlevel,
        hscAlevel,
        sex,
        image
      });

      // Save the new profile
      await profile.save();
      return res.status(201).json({ message: 'Profile created successfully', profile });
    }
  } catch (error) {
    console.error('Error saving or updating profile:', error);
    return res.status(500).json({ message: 'An error occurred while saving or updating the profile' });
  }
};
```

Route:

```
import express from 'express';
import { saveOrUpdateProfile } from '../controller/profile.controller.js';

const router = express.Router();

// Route to handle profile save or update
router.put('/profiles', saveOrUpdateProfile);

export default router;
```

Contribution of ID : 21101201, Name : Samia Tabassum

jobs.controller

```
export const searchJobs = async (req, res) => {
    try {
      const page = parseInt(req.query.page) || 1;
      const limit = parseInt(req.query.limit) || 7;
      const startIdx = (page - 1) * limit;
      const searchTerm = req.query.searchTerm || "";

      const searchQuery = {
        $or: [
            { name: { $regex: searchTerm, $options: "i" } },
            { companyName: { $regex: searchTerm, $options: "i" } },
            { jobTitle: { $regex: searchTerm, $options: "i" } },
            { location: { $regex: searchTerm, $options: "i" } }
          ]
        };

      const results = await Job.find(searchQuery)
        .limit(limit)
        .skip(startIdx);

      if (results.length === 0) {
        return res.status(404).json({ message: 'No jobs found' });
      }
```

```
      res.status(200).json(results);

    } catch (error) {
      console.log(error);
      res.status(500).json({ error: 'An error occurred while searching for
jobs' });
    }
};
```

jobs.route

```
router.get('/search', searchJobs)
```

GitHub Repository

Link: https://github.com/Tasnimrahman04/Workforce_Enroll.git

Conclusion

Workforce Enroll is a thoughtfully designed platform that bridges the gap between jobseekers and employers, fostering a seamless and efficient recruitment experience. With its robust set of features—from easy job posting and management to real-time application tracking and detailed dashboards—this project empowers users to navigate the job market with ease. By focusing on user-friendly interactions, flexibility, and transparent application management, Workforce Enroll stands out as a comprehensive solution that not only simplifies job hunting for candidates but also streamlines the hiring process for companies. It's an innovative step toward making job searching and recruitment smarter, faster, and more accessible for everyone.